

RV1106B/RV1103B Linux IPC SDK Quick Start

ID: RK-JC-YF-A28

Release Version: V1.0.0

Release Date: 2024-10-15

Security Level: ☐Top-Secret ☐Secret ☐Internal ☒Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED "AS IS". ROCKCHIP ELECTRONICS CO., LTD. ("ROCKCHIP") DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip's registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2024. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

The document presents the basic usage of Rockchip IPC Linux SDK, aiming to help developers get started with IPC Linux SDK faster.

Chipset and System Support

Chip Name	Kernel Version
RV1106B/RV1103B	Linux 5.10

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Author	Date	Revision History
V0.0.1	CWW	2024-06-26	Initial version
V0.0.2	CWW	2024-09-05	1. Add RV1106B support
V1.0.0	CWW、LHY	2024-10-14	1. Update BoardConfig.mk Introduction 2. Add How to use DM-Verify and DM-Crypt 3. Add How to Use UVC to Preview 4. Add CVR Framework Introduction 5. Update Cross Toolchain Download and Installation 6. Add Instruction of Battery Fastboot IPC

Contents

RV1106B/RV1103B Linux IPC SDK Quick Start

1. Hardware Introduction
 - 1.1 RV1103B_EVB1_V10 Hardware Overview
 - 1.1.1 Power-on
 - 1.1.2 Firmware Flashing
 - 1.1.3 Keys
 - 1.1.4 Video Preview
 - 1.1.5 Audio (MIC) Testing
 - 1.1.6 Web Live (RTMP)
 - 1.1.7 Wi-Fi
 - 1.1.8 RTC
 - 1.1.9 TF Card
 - 1.2 RV1106B_EVB2_V11 Hardware Overview
 - 1.2.1 Power-on
 - 1.2.2 Firmware Flashing
 - 1.2.3 Keys
 - 1.2.4 Video Preview
 - 1.2.5 Wi-Fi
 - 1.2.6 TF Card
2. Set up a Development Environment
 - 2.1 Download the Repo Tool and Usage
 - 2.2 SDK Obtaining
 - 2.3 Update SDK Code
 - 2.4 Cross Toolchain Download and Installation
3. SDK Usage Introduction
 - 3.1 BoardConfig.mk Introduction
 - 3.2 Check the SDK Version and Build Configuration
 - 3.3 One-click Automatic Compilation
 - 3.4 Build U-Boot
 - 3.5 Build kernel
 - 3.6 Build rootfs
 - 3.7 Build media
 - 3.8 Build Reference Applications
 - 3.9 Build Kernel's Driver
 - 3.10 Packaging env.img
 - 3.11 Firmware Packaging
 - 3.12 SDK Directory Structure
 - 3.12.1 Sysdrv Directory Introduction
 - 3.12.2 Media Directory Introduction
 - 3.12.3 App Directory Introduction
 - 3.13 The Output Directory of Images
 - 3.14 Debugging Tools
 - 3.14.1 Transfer Files via Network tftp
 - 3.14.2 Debug via ADB
 - 3.14.3 Debug via Telnet
 - 3.14.4 Debug via Serial Port
4. Documents Introduction
5. Tools Introduction
 - 5.1 Driver Installation Tool
 - 5.2 Firmware Burning
 - 5.3 Update.img Related Tool
 - 5.3.1 Packaging
 - 5.3.2 Unpacking
 - 5.3.3 Update.img Flashing
 - 5.4 SD Card Upgrade and Booting Making Tool

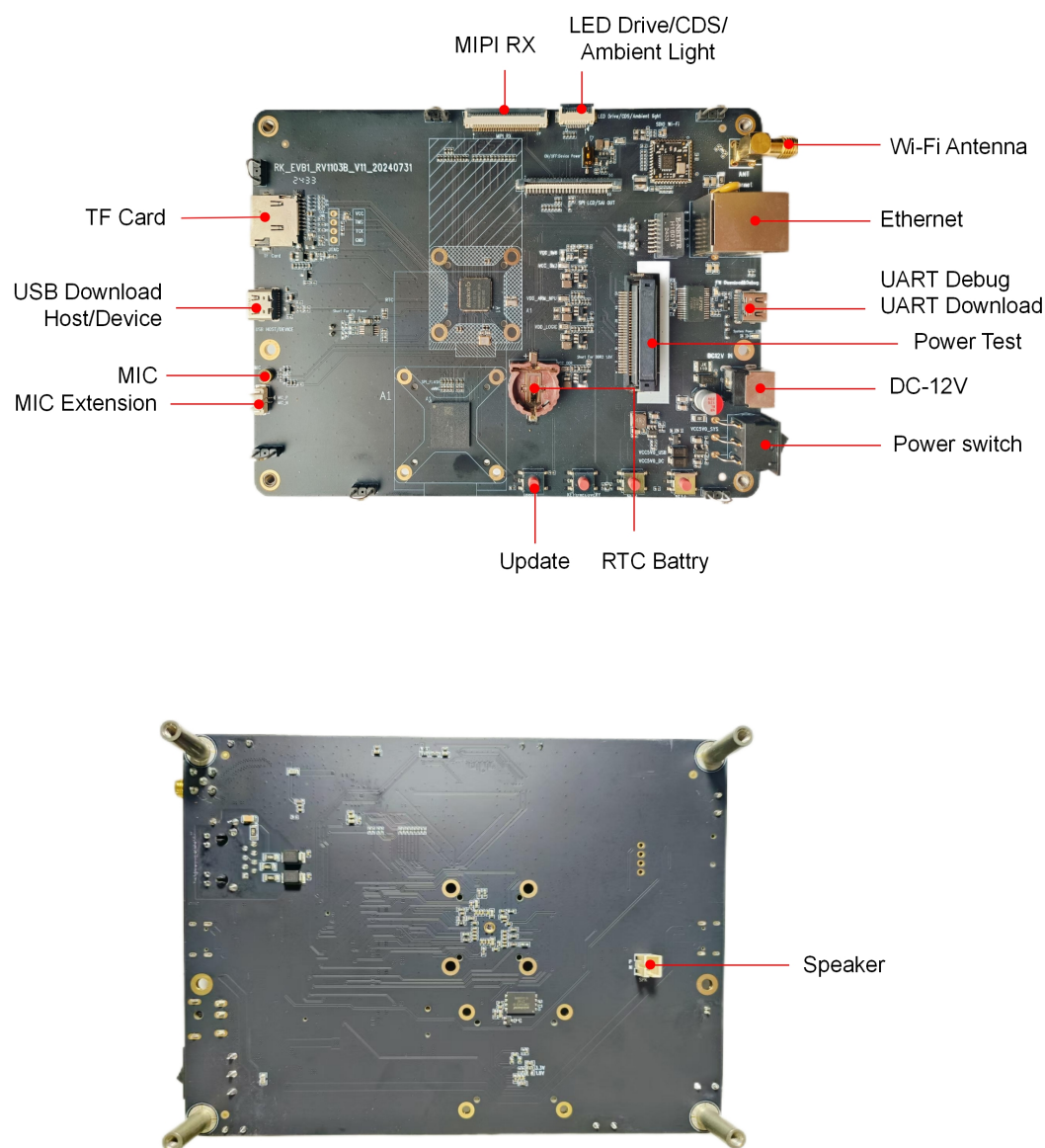
- 5.5 Factory Firmware Introduction
- 5.6 Mass Production Upgrade Tool
- 6. CVR Framework Introduction
 - 6.1 Overview of Software Functions
 - 6.2 Software Framework
 - 6.3 APP Introduction
 - 6.3.1 Version Without Screen
 - 6.3.2 Version with Screen Display
 - 6.3.3 Functional Module Introduction
 - 6.4 Introduction to LVGL
 - 6.4.1 Enable LVGL
 - 6.4.2 LVGL Configuration File
 - 6.5 rkadk Introduction
 - 6.5.1 rkadk Feature List
 - 6.5.2 rkadk API Description
- 7. Instruction of Battery Fastboot IPC
 - 7.1 Fastboot Process Introduction
 - 7.1.1 Basic Framework of Fastboot Process
 - 7.1.2 FastBoot FW config and compile
 - 7.1.3 Camera Fastboot Process Introduction
 - 7.1.3.1 Dual-camera Fastboot Demo Introduction
 - 7.1.3.2 Single Camera Boot-up Demo Introduction
 - 7.1.3.3 View the stream video
 - 7.1.4 Fastboot Log Introduction
 - 7.1.5 Introduction to the time-consuming statistics
 - 7.2 Ambient Light Sensor Introduction
 - 7.2.1 Ambient Light Sensor Type
 - 7.2.2 Ambient Light Sensor Parameters
 - 7.2.3 Analog Light Sensor Calibration Method
 - 7.2.4 Software Light Sensor Introduction
 - 7.3 The meta partition Introduction
 - 7.3.1 Meta Introduction
 - 7.3.2 Sensor Information Parameters Introduction
 - 7.3.3 Camera Sensor IQ Parameter Introduction
 - 7.3.4 AE/AWB Parameter Introduction
 - 7.3.5 Application Parameters Introduction
 - 7.3.6 Kernel CMDLINE Parameters Introduction
 - 7.3.7 Generate meta.img
 - 7.3.8 Update meta Data on Board
 - 7.3.8.1 Dump meta Partition
 - 7.3.8.2 Update meta Partition Data
 - 7.3.8.3 Update the cmdline of meta. For example, enable kernel boot log
 - 7.3.8.4 Update the cmdline of meta. Enable the camera debug AIQ log
 - 7.3.8.5 Update sensor initialization parameters
 - 7.3.8.6 Update Sensor IQ file
 - 7.3.8.7 Other parameters introduction
 - 7.4 Modify System Memory Allocation
 - 7.4.1 Modify the Kernel DTS Memory Allocation
 - 7.4.2 Modify FIT Script
 - 7.4.3 Rebuild
- 8. IPC Linux SDK Q&A
 - 8.1 How to Modify Partition Table, Add Customized Partition and Partition Read/Write
 - 8.1.1 Storage Medium and File System Type
 - 8.1.2 Partition Table Introduction
 - 8.1.3 Add Customized Partition
 - 8.1.4 OEM Partition Mounting Introduction
 - 8.2 How to Use OTA
 - 8.2.1 How to Use Recovery

- 8.3 [How to Use TFTP Upgrade in U-Boot Terminal](#)
 - 8.4 [How to Use SDcard Upgrade in U-Boot Terminal](#)
 - 8.5 [How to Use a Key to Trigger SDcard Upgrade during U-Boot Phase](#)
 - 8.6 [Cannot Detect Device When SD Card Inserted](#)
 - 8.7 [How to Add Third-Party Libraries to the sysdrv Directory](#)
 - 8.8 [How to Add New Applications in Project/app](#)
 - 8.9 [How to Download the NPU Model Transformation Tool and Runtime Library](#)
 - 8.10 [How to Change the CMA Size on Board](#)
 - 8.11 [How to Use the coredump](#)
 - 8.12 [Kernel Driver Insmod Introduction](#)
 - 8.13 [Information About Libraries and Driver for RV1106B/RV1103B](#)
 - 8.14 [How to Use NFS](#)
 - 8.15 [How to Add a New User and Set Password for Login](#)
 - 8.16 [The Usage of A/B Systems](#)
 - 8.16.1 [Introduction to Startup Schemes](#)
 - 8.16.2 [Configuration](#)
 - 8.16.3 [OTA Upgrade Tool](#)
 - 8.16.4 [A/B Systems Switching](#)
 - 8.16.5 [A/B Systems Upgrading](#)
 - 8.17 [Get the Camera Support Lists](#)
 - 8.18 [Get the Flash Support Lists](#)
 - 8.19 [The Usage of the Stress Test](#)
 - 8.19.1 [memtester Test](#)
 - 8.19.2 [stressapptest](#)
 - 8.19.3 [cpufreq Test](#)
 - 8.19.4 [Flash Stress Test](#)
 - 8.19.5 [Reboot Test](#)
 - 8.20 [How to Use rndis](#)
 - 8.21 [How to Add Non-root User Login](#)
 - 8.22 [How to Add New Camera Sensor](#)
 - 8.23 [How to Reboot into The Terminal of U-Boot](#)
 - 8.24 [How to Modify GPIO Register Configuration by Command](#)
 - 8.24.1 [U-Boot Terminal](#)
 - 8.24.2 [Root File System](#)
 - 8.25 [How to Modify the Uart Baudrate](#)
 - 8.25.1 [DDR Bin File](#)
 - 8.25.2 [Kernel DTS](#)
 - 8.26 [Enabling the sysstat Test Suite](#)
 - 8.27 [How to use DM-Verity and DM-Crypt](#)
 - 8.28 [How to Use UVC to Preview](#)
9. [Notices](#)

1. Hardware Introduction

1.1 RV1103B_EVB1_V10 Hardware Overview

The hardware layout of the RV1103B EVB is showed as follows.



1.1.1 Power-on

It can be powered via USB or a power supply.

It is recommended to use a power supply first. Only using USB power supply is unstable, and plugging and unplugging USB may affect the usage of the serial port.

1.1.2 Firmware Flashing

Refer to [Firmware Burning](#).

1.1.3 Keys

Keys	Function
Update	Flashing and upgrade
KEY1/KEY2/KEY3/RECOVERY	reserved

1.1.4 Video Preview

The device supports previewing in the same local area network. After the device is connected to the Internet, use RTSP software on the PC (such as VLC) to open the network stream, and enter the following address:

```
rtsp://(your device's IP address)/live/0
```

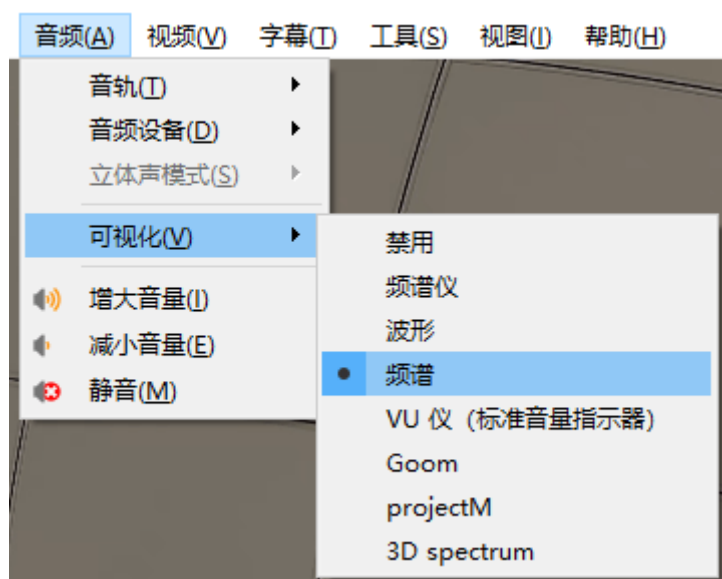


You will see the preview of the camera:

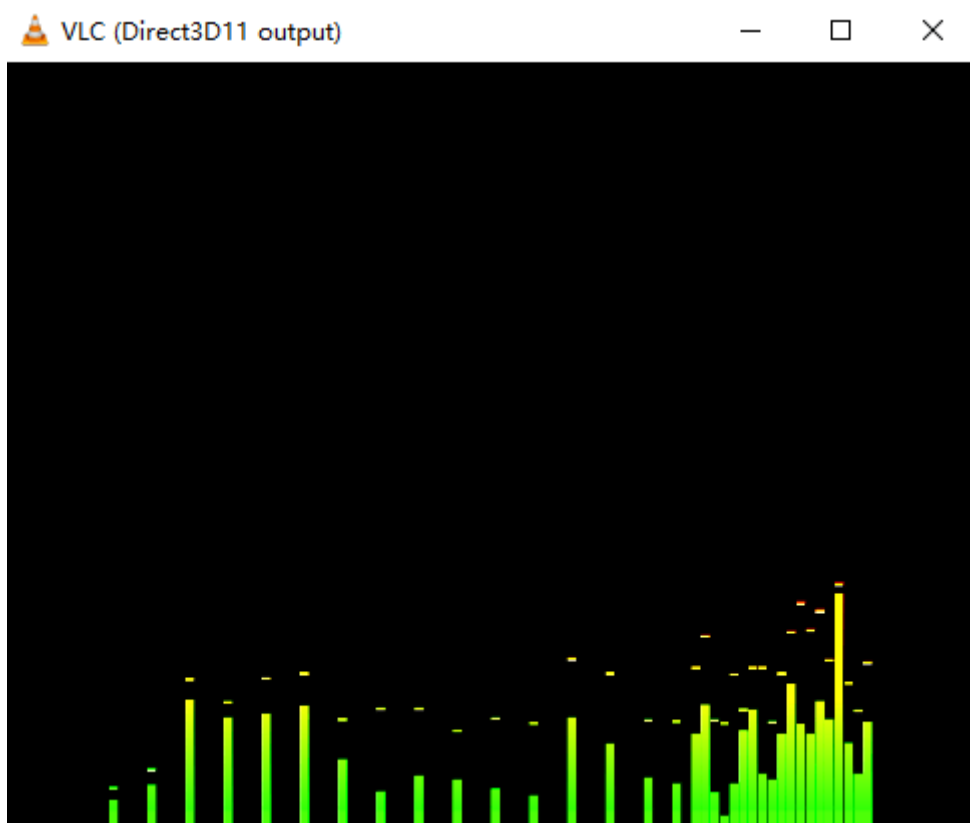


1.1.5 Audio (MIC) Testing

Accessing [RTSP Stream Preview](#) through the PC, you can get the audio at the same time. The audio can be played to the speaker directly, or can be detected by using the visual audio that comes with the software. The way to see audio spectrum is as follows:



The spectrum effect is as follows:

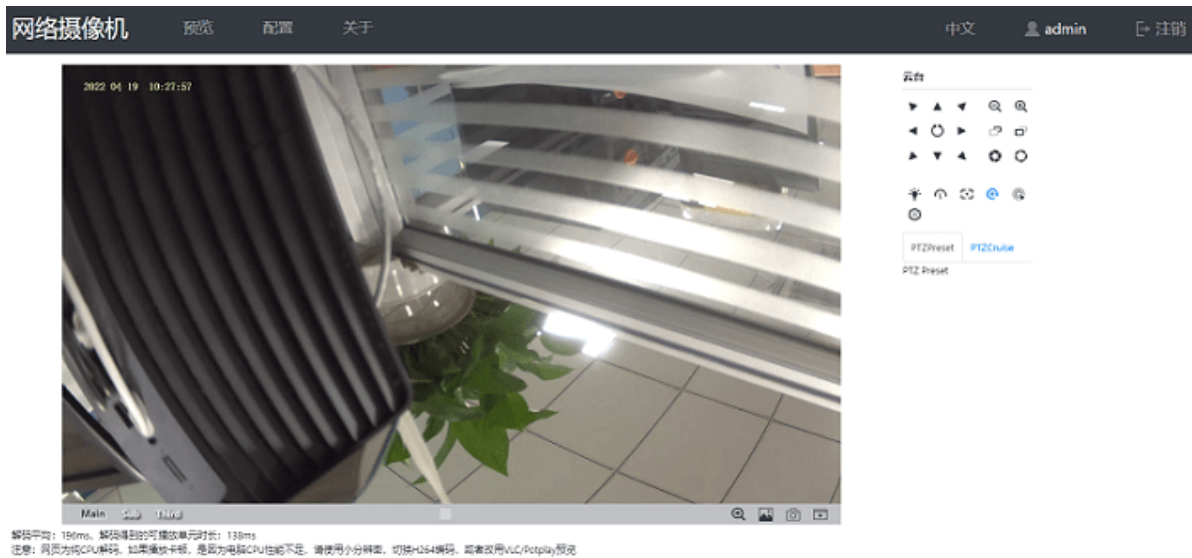


1.1.6 Web Live (RTMP)

The main stream, sub-stream, and H264/H265 supports live broadcasting on the Web side. Access the IP address of the device through a browser, enter the user name and password (both are admin by default), and enter the live preview system.



The preview effect is as follows:



1.1.7 Wi-Fi

The Wi-Fi function is not enabled by default, and the following configurations are required when you want to use it:

```
cd /oem/usr/ko/
./insmod_wifi.sh
```

After running successfully, you will get wlan0 in `ifconfig`:

```
# ifconfig
eth0      Link encap:Ethernet  HWaddr DA:54:55:F7:10:18
          inet addr:192.168.1.169  Bcast:255.255.255.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:58  errors:0  dropped:0  overruns:0  frame:0
          TX packets:6  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:6852 (6.6 KiB)  TX bytes:2052 (2.0 KiB)
          Interrupt:45

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:356100  errors:0  dropped:0  overruns:0  frame:0
          TX packets:356100  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:106376267 (101.4 MiB)  TX bytes:106376267 (101.4 MiB)

wlan0     Link encap:Ethernet  HWaddr F0:B0:40:69:53:17
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:375  errors:0  dropped:325  overruns:0  frame:0
          TX packets:2  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:107823 (105.2 KiB)  TX bytes:288 (288.0 B)
```

Get network IP:

```
rkwifi_server connect <SSID> <password>
```

Finally, get the network information:

```
wlan0    Link encap:Ethernet  HWaddr F0:B0:40:69:53:17
          inet addr:192.168.1.155  Bcast:255.255.255.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:506 errors:0 dropped:419 overruns:0 frame:0
          TX packets:4 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:141895 (138.5 KiB)  TX bytes:1028 (1.0 KiB)
```

Testing:

```
# ping www.baidu.com
PING www.baidu.com (163.177.151.109): 56 data bytes
64 bytes from 163.177.151.109: seq=0 ttl=54 time=25.981 ms
64 bytes from 163.177.151.109: seq=1 ttl=54 time=26.812 ms
64 bytes from 163.177.151.109: seq=2 ttl=54 time=50.451 ms
64 bytes from 163.177.151.109: seq=3 ttl=54 time=56.411 ms
^C
--- www.baidu.com ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 25.981/39.913/56.411 ms
```

1.1.8 RTC

The RTC needs to be installed with a battery to keep the clock unchanged after the power is powered on. (The battery models such as CR1220.)

Here are some ways to use RTC:

To check the RTC information:

```
hwclock
```

```
# hwclock
Fri Jan 1 12:52:21 2021 0.000000 seconds
```

Synchronize the system clock by RTC:

```
hwclock -s
```

```
# date
Fri Jan 1 00:00:45 UTC 2021
# hwclock -r
Tue Apr 19 15:16:14 2022 0.000000 seconds
# hwclock -s
# date
Tue Apr 19 15:16:30 UTC 2022
# hwclock -r
Tue Apr 19 15:16:35 2022 0.000000 seconds
```

Synchronize RTC by the system clock:

```
hwclock -w
```

```
# date
Tue Apr 19 15:20:04 UTC 2022
# hwclock -r
Fri Jan 1 00:00:40 2021 0.000000 seconds
# hwclock -w
# date
Tue Apr 19 15:20:14 UTC 2022
# hwclock -r
Tue Apr 19 15:20:17 2022 0.000000 seconds
#
```

1.1.9 TF Card

After inserting a TF card and power on, the TF card will be automatically mounted to `/mnt/sdcard`.

```
/dev/mmcblk1 on /mnt/sdcard type vfat (rw,noatime,uid=1000,
```

To save the video to TF card, the application configuration file `rkipc.ini` should be modified, enable the storage module and mount path.

```
vi /data/rkipc.ini
```

Take the `/mnt/sdcard` mount path as an example, modifications are shown below:

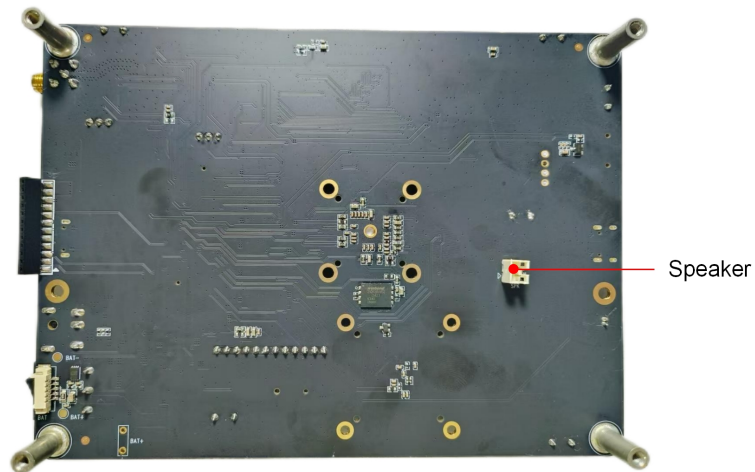
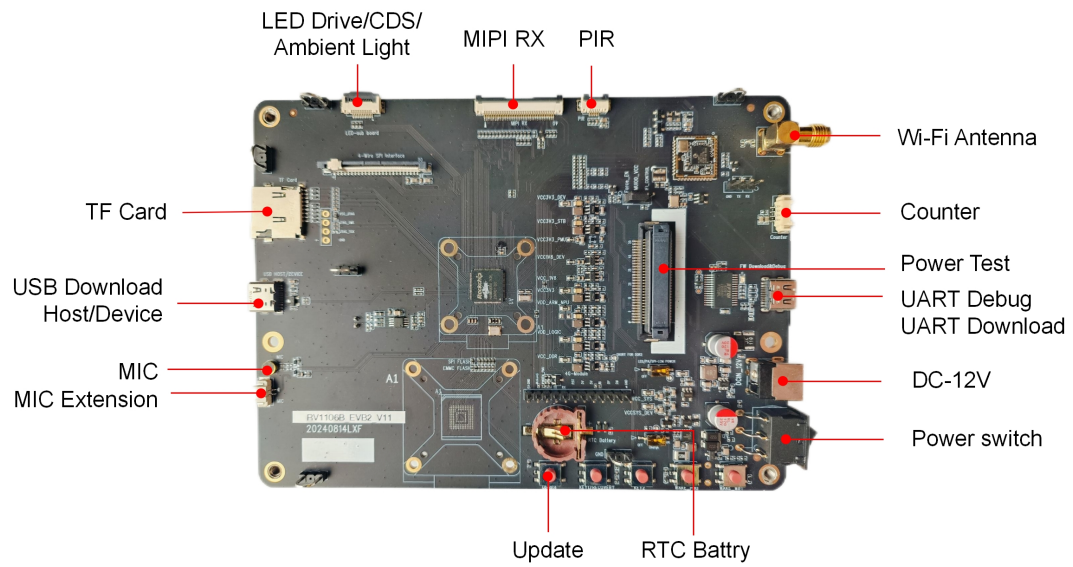
```
[storage]
mount_path           = /mnt/sdcard
free_size_del_min    = 500
free_size_del_max    = 1000

[storage.0]
enable               = 1
folder_name          = video0
file_format          = mp4
file_duration        = 60
video_quota          = 30
```

Reboot the device, videos will be written to `/mnt/sdcard/video0`.

1.2 RV1106B_EVB2_V11 Hardware Overview

The hardware layout of the RV1106B EVB is showed as follows.



1.2.1 Power-on

Requires power supply via DC-12V

1.2.2 Firmware Flashing

Refer to [Firmware Burning](#).

1.2.3 Keys

Keys	Function
Update	Flashing and upgrade
KEY1/KEY2/RECOVERYWAKE_PMU/WAKE_WIFI	reserved

1.2.4 Video Preview

Taking the FastBoot firmware as an example. (BoardConfig: BoardConfig-SPI_NOR-NONE-RV1106B_EVB2_V10_V11-BAT_IPC.mk)

1.Launch rndis

```
# rndis.sh
serialnumber is 0000000000000000
config usb0 IP...
```

2.Check the preset IP address.

```
# ifconfig
usb0      Link encap:Ethernet  HWaddr 4A:FC:84:D6:A4:1A
          inet addr:192.168.1.100  Bcast:192.168.1.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:80 errors:0 dropped:63 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:12637 (12.3 KiB)  TX bytes:0 (0.0 B)
```

3.Configuration address for the streaming software.

```
rtsp://192.168.1.100/live/0
```

1.2.5 Wi-Fi

Wi-Fi network configuration:

```
rkwifi_server start &
rkwifi_server connect "SSID" "PASSWORD"
```

1.2.6 TF Card

1.After inserting the card, the system prints out information indicating that it has been recognized.

```
6,348,299343396,-;mmc_host mmc1: Bus speed (slot 0) = 49500000Hz (slot req
50000000Hz, actual 49500000HZ div = 0)
SUBSYSTEM=mmc_host
DEVICE+=mmc_host:mmc1
6,349,299343471,-;mmc1: new high speed SDXC card at address 59b4
6,350,299343954,-;mmcblk1: mmc1:59b4 SD64G 59.4 GiB
6,351,299345963,-; mmcblk1: p1
```

2. Mount to the specified path.

```
mount /dev/mmcblk1p1 /mnt/sdcard/
```

3. Check the mount info.

```
# df -h
Filesystem                Size      Used Available Use% Mounted on
/dev/root                  7.8M        7.8M          0 100% /
devtmpfs                   4.3M          0        4.3M   0% /dev
tmpfs                      6.9M        1.1M        5.8M  15% /tmp
tmpfs                      6.9M          0        6.9M   0% /run
/dev/block/by-name/userdata
                          7.1M       408.0K        6.7M   6% /userdata
/dev/mmcblk1p1            30.0G        4.6G       25.4G  15% /mnt/sdcard
```

2. Set up a Development Environment

This SDK is developed and tested on Ubuntu system. Other Linux versions may should adjust the software package accordingly. In addition to the system requirements, there are other hardware and software requirements.

Hardware requirements: 64-bit system, hard disk space should be greater than 20G. If you do multiple builds, you will need more hard drive space

Software requirements: Ubuntu 18.04 system:

Please install software packages with below commands to setup SDK compiling environment:

```
sudo apt-get install repo git ssh make gcc \
gcc-multilib g++-multilib module-assistant \
expect g++ gawk texinfo libssl-dev \
bison flex fakeroot cmake unzip gperf autoconf \
device-tree-compiler libncurses5-dev
```

It is recommended to use Ubuntu 18.04 system or higher version for development. If you encounter an error during compilation, you can check the error message and install the corresponding software packages accordingly.

2.1 Download the Repo Tool and Usage

```
mkdir -p $HOME/repo-tool
git clone ssh://git@www.rockchip.com.cn/repo/rk/tools/repo $HOME/repo-tool

export PATH="$HOME/repo-tool:$PATH"
# Test command
repo version
```

2.2 SDK Obtaining

There are two ways to obtain the SDK: **download online** and **Offline SDK Package**.

Note: SDKs of different platforms require corresponding download permissions.

1. Download online

Please contact RK business or FAE to get the SDK Release document of related chip.

For example: The Linux IPC SDK Release document of RV1106B/RV1103B is
Rockchip_RV1106B_RV1103B_Linux_IPC_SDK_XXXXXXX_EN.pdf

2. Offline SDK Package

The offline SDK package can be obtained from RK FAE window.

```
# Take RV1106B_RV1103B_LINUX_IPC_SDK_V0.0.1_XXX.tar.bz2 as an example
mkdir rv1103b_linux_ipc_sdk
tar xf RV1106B_RV1103B_LINUX_IPC_SDK_V0.0.1_XXX.tar.bz2 -C
rv1103b_linux_ipc_sdk
cd rv1103b_linux_ipc_sdk

# Check out the local code
.repo/repo/repo sync -l
```

2.3 Update SDK Code

Before update SDK code, please make a local backup.

```
# The command of update SDK code
.repo/repo/repo sync -c --no-tags
# If some projects fail to download, add the --force-sync parameter
.repo/repo/repo sync -c --no-tags --force-sync
# After update SDK code, clean the SDK.
./build.sh clean
```

2.4 Cross Toolchain Download and Installation

The cross toolchain can be obtained from tools/linux/toolchain/ in the SDK directory.

Chip Name	Cross Toolchain	Test Commands
RV1106B/RV1103B	arm-rockchip831-linux- -uclibcgnueabihf	arm-rockchip831-linux-uclibcgnueabihf-gcc --version

```
cd tools/linux/toolchain/arm-rockchip831-linux-uclibcgnueabihf
source env_install_toolchain.sh
# or install toolchain to the dirname
# source env_install_toolchain.sh dirname
```

The cross toolchain can be obtained from tools/linux/toolchain/ in the SDK directory.

3. SDK Usage Introduction

3.1 BoardConfig.mk Introduction

- BoardConfig.mk Introduction

SDK's board config file is in the directory of `project/cfg/`, BoardConfig.mk file is an important file for SDK compilation.

The `project/cfg-all-items-introduction.txt` file will record the latest config options introduction.

Config Options	Introduction
RK_ARCH	arm or arm64 Define compiling 32 bit or 64 bit program
RK_CHIP	non-modifiable Different chips correspond to different SDKs
RK_TOOLCHAIN_CROSS	non-modifiable Define cross-compilation toolchain
RK_BOOT_MEDIUM	emmc/spi_nor/spi_nand Define the storage medium of board
RK_UBOOT_DEFCONFIG	U-Boot defconfig file name Located in sysdrv/source/uboot/u-boot/configs
RK_UBOOT_DEFCONFIG_FRAGMENT	U-Boot config file name (optional) Located in sysdrv/source/uboot/u-boot/configs Overlays the defconfig defined by RK_UBOOT_DEFCONFIG
RK_KERNEL_DEFCONFIG	Kernel defconfig file name Located in sysdrv/source/kernel/arch/\$RK_ARCH/configs
RK_KERNEL_DEFCONFIG_FRAGMENT	Kernel defconfig file name (Optional) Located in sysdrv/source/kernel/arch/\$RK_ARCH/configs Overlays the defconfig defined by RK_KERNEL_DEFCONFIG
RK_KERNEL_DTS	Kernel dts file name RK_ARCH=arm Located in sysdrv/source/kernel/arch/arm/boot/dts RK_ARCH=arm64 Located in sysdrv/source/kernel/arch/arm64/boot/dts/rockchip
RK_MISC	If recovery is enabled, Read the flag at system startup and select it into the recovery system or application system (if there is no recovery, it can be removed)
RK_CAMERA_SENSOR_IQFILES	Camera Sensor IQ file name Located in media/isp/camera_engine_rkaiq/iqfiles or media/isp/camera_engine_rkaiq/rkaiq/iqfiles If there are multi IQ files, separated by spaces. e.g. RK_CAMERA_SENSOR_IQFILES="iqfile_1 iqfile_2"
RK_PARTITION_CMD_IN_ENV	Configure partition table (IMPORTANT) The Format of Partition Table:<partdef>[,<partdef>] <partdef> format: <size>[@<offset>](part-name) For detailed configuration, please refer to Partition Table Introduction chapter

Config Options	Introduction
RK_PARTITION_FS_TYPE_CFG	Configure filesystem type of partition and mount point (IMPORTANT) Format Introduction: "partition name"@ "the mount point of partition"@ "filesystem type of partition" Notice: the mount point of root filesystem is IGNORE default (non-modifiable)
RK_SQUASHFS_COMP	Config squashfs image compression algorithm (Optional) options: lz4/lzo/lzma/xz/gzip (default xz)
RK_UBIFS_COMP	Config ubifs image compression algorithm options: lzo/zlib (default lzo)
RK_APP_TYPE	Config the reference app compilation type (Optional) run <code>./build.sh info</code> to get the options of app type
RK_APP_IPCWEB_BACKEND	Config whether to build WebServer app (Optional) y: enable
RK_BUILD_APP_TO_OEM_PARTITION	Config whether to install app to oem partition (Optional) y: enable
RK_ENABLE_RECOVERY	Config whether to enable recovery compilation (Optional) y: enable n: disable
RK_ENABLE_FASTBOOT	Config whether to enable fastboot (Optional) y: enable Needs to be combined with U-Boot and kernel modifications, refer to SDK BoardConfig-*-TB.mk
RK_ENABLE_GDB	Config whether to enable gdb compilation (Optional) y: enable n: disable
RK_ENABLE_ADBD	Config whether to enable adb (Optional) y: enable n: disable Notice: needs to be enable USB defconfig in kernel
RK_BOOTARGS_CMA_SIZE	Config CMA size (Optional)
RK_POST_BUILD_SCRIPT	The script to execute configuration before packaging rootfs.img (Located in BoardConfig dir, optional)
RK_PRE_BUILD_OEM_SCRIPT	The script to execute configuration before packaging oem.img (Located in BoardConfig dir, optional)
RK_BUILD_APP_TO_OEM_PARTITION	Config whether to enable install application to oem partition (Optional) :y: enable
RK_ENABLE_RNDIS	Config whether to enable rndis (Optional)y: enable n: disable

Config Options	Introduction
RK_META_PARAM	Conifg the parameter of meta partition (Optional, which is used for battery IPC product)

- The command to select BoardConfig

```
./build.sh lunch
```

```
You're building on Linux
Lunch menu...pick a combo:

BoardConfig-*.mk naming rules:
BoardConfig-"启动介质"-"电源方案"-"硬件版本"-"应用场景".mk
BoardConfig-"boot medium"-"power solution"-"hardware version"-"application".mk

-----

0. BoardConfig_BatteryIPC/BoardConfig-EMMC-NONE-RV1106B_EVB2_V10_V11-BAT_IPC.mk
    boot medium(启动介质): EMMC
    power solution(电源方案): NONE
    hardware version(硬件版本): RV1106B_EVB2_V10_V11
    application(应用场景): BAT_IPC
-----

-----

1. BoardConfig_BatteryIPC/BoardConfig-EMMC-NONE-RV1106B_EVB2_V10_V11-
BAT_IPC_FASTBOOT.mk
    boot medium(启动介质): EMMC
    power solution(电源方案): NONE
    hardware version(硬件版本): RV1106B_EVB2_V10_V11
    application(应用场景): BAT_IPC_FASTBOOT
-----

-----

2. BoardConfig_BatteryIPC/BoardConfig-SPI_NAND-NONE-RV1106B_EVB2_V10_V11-
BAT_IPC.mk
    boot medium(启动介质): SPI_NAND
    power solution(电源方案): NONE
    hardware version(硬件版本): RV1106B_EVB2_V10_V11
    application(应用场景): BAT_IPC
-----

-----

3. BoardConfig_BatteryIPC/BoardConfig-SPI_NAND-NONE-RV1106B_EVB2_V10_V11-
BAT_IPC_FASTBOOT.mk
    boot medium(启动介质): SPI_NAND
    power solution(电源方案): NONE
    hardware version(硬件版本): RV1106B_EVB2_V10_V11
    application(应用场景): BAT_IPC_FASTBOOT
-----

-----

4. BoardConfig_BatteryIPC/BoardConfig-SPI_NOR-NONE-RV1106B_EVB2_V10_V11-
BAT_IPC.mk
    boot medium(启动介质): SPI_NOR
    power solution(电源方案): NONE
    hardware version(硬件版本): RV1106B_EVB2_V10_V11
```

application(应用场景): BAT_IPC

5. BoardConfig_BatteryIPC/BoardConfig-SPI_NOR-NONE-RV1106B_EVB2_V10_V11-BAT_IPC_DUAL_CAMERA.mk

boot medium(启动介质): SPI_NOR
power solution(电源方案): NONE
hardware version(硬件版本): RV1106B_EVB2_V10_V11
application(应用场景): BAT_IPC_DUAL_CAMERA

6. BoardConfig_BatteryIPC/BoardConfig-SPI_NOR-NONE-RV1106B_EVB2_V10_V11-BAT_IPC_DUAL_CAMERA_AOV.mk

boot medium(启动介质): SPI_NOR
power solution(电源方案): NONE
hardware version(硬件版本): RV1106B_EVB2_V10_V11
application(应用场景): BAT_IPC_DUAL_CAMERA_AOV

7. BoardConfig_BatteryIPC/BoardConfig-SPI_NOR-NONE-RV1106B_EVB2_V10_V11-BAT_IPC_FASTBOOT.mk

boot medium(启动介质): SPI_NOR
power solution(电源方案): NONE
hardware version(硬件版本): RV1106B_EVB2_V10_V11
application(应用场景): BAT_IPC_FASTBOOT

8. BoardConfig_BatteryIPC/BoardConfig-SPI_NOR-NONE-RV1106B_EVB2_V10_V11-BAT_IPC_SC200AI_AOV.mk

boot medium(启动介质): SPI_NOR
power solution(电源方案): NONE
hardware version(硬件版本): RV1106B_EVB2_V10_V11
application(应用场景): BAT_IPC_SC200AI_AOV

9. BoardConfig_BatteryIPC/BoardConfig-SPI_NOR-NONE-RV1106B_EVB2_V10_V11-BAT_IPC_SC450AI.mk

boot medium(启动介质): SPI_NOR
power solution(电源方案): NONE
hardware version(硬件版本): RV1106B_EVB2_V10_V11
application(应用场景): BAT_IPC_SC450AI

10. BoardConfig_BatteryIPC/BoardConfig-SPI_NOR-NONE-RV1106B_EVB2_V10_V11-BAT_IPC_SC450AI_AOV.mk

boot medium(启动介质): SPI_NOR
power solution(电源方案): NONE
hardware version(硬件版本): RV1106B_EVB2_V10_V11
application(应用场景): BAT_IPC_SC450AI_AOV

11. BoardConfig_BatteryIPC/BoardConfig-SPI_NOR-NONE-RV1106B_EVB2_V10_V11-IPC_AOV.mk

boot medium(启动介质): SPI_NOR
power solution(电源方案): NONE
hardware version(硬件版本): RV1106B_EVB2_V10_V11
application(应用场景): IPC_AOV

12. BoardConfig_BatteryIPC/BoardConfig-SPI_NOR-NONE-RV1106B_EVB2_V10_V11-IPC_DUAL_CAMERA_AOV.mk

boot medium(启动介质): SPI_NOR
power solution(电源方案): NONE
hardware version(硬件版本): RV1106B_EVB2_V10_V11
application(应用场景): IPC_DUAL_CAMERA_AOV

13. BoardConfig_BatteryIPC/BoardConfig-SPI_NOR-NONE-RV1106B_EVB3_V11-DOORBELL.mk

boot medium(启动介质): SPI_NOR
power solution(电源方案): NONE
hardware version(硬件版本): RV1106B_EVB3_V11
application(应用场景): DOORBELL

14. BoardConfig_IPC/BoardConfig-EMMC-NONE-RV1103B_EVB1_V10_V11-IPC.mk

boot medium(启动介质): EMMC
power solution(电源方案): NONE
hardware version(硬件版本): RV1103B_EVB1_V10_V11
application(应用场景): IPC

15. BoardConfig_IPC/BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-BAT_IPC.mk

boot medium(启动介质): SPI_NAND
power solution(电源方案): NONE
hardware version(硬件版本): RV1103B_EVB1_V10_V11
application(应用场景): BAT_IPC

16. BoardConfig_IPC/BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-IPC.mk

boot medium(启动介质): SPI_NAND
power solution(电源方案): NONE
hardware version(硬件版本): RV1103B_EVB1_V10_V11
application(应用场景): IPC

17. BoardConfig_IPC/BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-IPC_4K.mk

boot medium(启动介质): SPI_NAND
power solution(电源方案): NONE
hardware version(硬件版本): RV1103B_EVB1_V10_V11
application(应用场景): IPC_4K

```

-----
18. BoardConfig_IPC/BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-IPC_AIISP.mk
    boot medium(启动介质): SPI_NAND
    power solution(电源方案): NONE
    hardware version(硬件版本): RV1103B_EVB1_V10_V11
    application(应用场景): IPC_AIISP
-----

-----
19. BoardConfig_IPC/BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-
IPC_DUAL_CAMERA.mk
    boot medium(启动介质): SPI_NAND
    power solution(电源方案): NONE
    hardware version(硬件版本): RV1103B_EVB1_V10_V11
    application(应用场景): IPC_DUAL_CAMERA
-----

-----
20. BoardConfig_IPC/BoardConfig-SPI_NOR-NONE-RV1103B_EVB1_V10_V11-BAT_IPC.mk
    boot medium(启动介质): SPI_NOR
    power solution(电源方案): NONE
    hardware version(硬件版本): RV1103B_EVB1_V10_V11
    application(应用场景): BAT_IPC
-----

-----
21. BoardConfig_IPC/BoardConfig-SPI_NOR-NONE-RV1103B_EVB1_V10_V11-IPC.mk
    boot medium(启动介质): SPI_NOR
    power solution(电源方案): NONE
    hardware version(硬件版本): RV1103B_EVB1_V10_V11
    application(应用场景): IPC
-----

Which would you like? [0]:

```

Enter the corresponding number to select the corresponding reference board level.

BoardConfig: BoardConfig_BatteryIPC	Hardware Description	Support FASTAE function (i.e., camera quick start)	Support AOV (i.e. Always On Video)
BoardConfig-EMMC-NONE-RV1106B_EVB2_V10_V11-BAT_IPC.mk	Use for RV1106B EVB2 V10 or V11, Camera is SC200AI	YES	NO
BoardConfig-EMMC-NONE-RV1106B_EVB2_V10_V11-BAT_IPC_FASTBOOT.mk	Use for RV1106B EVB2 V10 or V11, Camera is SC200AI	NO	NO
BoardConfig-SPI_NAND-NONE-RV1106B_EVB2_V10_V11-BAT_IPC.mk	Use for RV1106B EVB2 V10 or V11, Camera is SC200AI	YES	NO
BoardConfig-SPI_NAND-NONE-RV1106B_EVB2_V10_V11-BAT_IPC_FASTBOOT.mk	Use for RV1106B EVB2 V10 or V11, Camera is SC200AI	NO	NO
BoardConfig-SPI_NOR-NONE-RV1106B_EVB2_V10_V11-BAT_IPC.mk	Use for RV1106B EVB2 V10 or V11, Camera is SC200AI	YES	NO
BoardConfig-SPI_NOR-NONE-RV1106B_EVB2_V10_V11-BAT_IPC_DUAL_CAMERA.mk	Use for RV1106B EVB2 V10 or V11, Camera is Dual SC200AI	YES	NO
BoardConfig-SPI_NOR-NONE-RV1106B_EVB2_V10_V11-BAT_IPC_DUAL_CAMERA_AOV.mk	Use for RV1106B EVB2 V10 or V11, Camera is Dual SC200AI	YES	YES
BoardConfig-SPI_NOR-NONE-RV1106B_EVB2_V10_V11-BAT_IPC_FASTBOOT.mk	Use for RV1106B EVB2 V10 or V11, Camera is Dual SC200AI	NO	NO
BoardConfig-SPI_NOR-NONE-RV1106B_EVB2_V10_V11-BAT_IPC_SC200AI_AOV.mk	Use for RV1106B EVB2 V10 or V11, Camera is Dual SC200AI	YES	YES
BoardConfig-SPI_NOR-NONE-RV1106B_EVB2_V10_V11-BAT_IPC_SC450AI.mk	Use for RV1106B EVB2 V10 or V11, Camera is SC450AI	YES	NO
BoardConfig-SPI_NOR-NONE-RV1106B_EVB2_V10_V11-BAT_IPC_SC450AI_AOV.mk	Use for RV1106B EVB2 V10 or V11, Camera is SC450AI	YES	YES

BoardConfig: BoardConfig_BatteryIPC	Hardware Description	Support FASTAE function (i.e., camera quick start)	Support AOV (i.e. Always On Video)
BoardConfig-SPI_NOR-NONE- RV1106B_EVB2_V10_V11- IPC_AOV.mk	Use for RV1106B EVB2 V10 or V11, Camera is SC200AI or SC450AI	NO	YES
BoardConfig-SPI_NOR-NONE- RV1106B_EVB2_V10_V11- IPC_DUAL_CAMERA_AOV.mk	Use for RV1106B EVB2 V10 or V11, Camera is Dual SC200AI	NO	YES
BoardConfig-SPI_NOR-NONE- RV1106B_EVB3_V11- DOORBELL.mk	Use for RV1106B EVB3 V11, Camera is PS5458	YES	NO

BoardConfig: BoardConfig_IPC	Hardware Description	Support FASTAE function (i.e., camera quick start)
BoardConfig-EMMC-NONE-RV1103B_EVB1_V10_V11-IPC.mk	Use for RV1103B EVB1 V10 or V11, Camera is SC200AI or SC450AI	NO
BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-BAT_IPC.mk	Use for RV1103B EVB1 V10 or V11, Camera is SC200AI	YES
BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-IPC.mk	Use for RV1103B EVB1 V10 or V11, Camera is SC200AI or SC450AI	NO
BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-IPC_4K.mk	Use for RV1103B EVB1 V10 or V11, Camera is SC200AI or SC450AI	NO
BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-IPC_AIISP.mk	Use for RV1103B EVB1 V10 or V11, Camera is SC200AI or SC450AI	NO
BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-IPC_DUAL_CAMERA.mk	Use for RV1103B EVB1 V10 or V11, Camera is SC200AI or SC450AI	NO
BoardConfig-SPI_NOR-NONE-RV1103B_EVB1_V10_V11-BAT_IPC.mk	Use for RV1103B EVB1 V10 or V11, Camera is SC200AI	YES
BoardConfig-SPI_NOR-NONE-RV1103B_EVB1_V10_V11-IPC.mk	Use for RV1103B EVB1 V10 or V11, Camera is SC200AI or SC450AI	NO

3.2 Check the SDK Version and Build Configuration

```
./build.sh info
```

The command to check the sdk version `sdkinfo`

```
# sdkinfo
```

3.3 One-click Automatic Compilation

```
./build.sh lunch      # Select reference board level
./build.sh            # One-click automatic compilation
```

3.4 Build U-Boot

```
./build.sh clean uboot
./build.sh uboot

# The detailed uboot compilation command
# ./build.sh info
```

Generate image files: output/image/download.bin output/image/adbblock.img output/image/uboot.img

3.5 Build kernel

```
./build.sh clean kernel
./build.sh kernel

# The detailed kernel compilation command
# ./build.sh info
```

Generate image file: output/image/boot.img

3.6 Build rootfs

```
./build.sh clean rootfs
./build.sh rootfs
```

Make rootfs.img firmware by the command of `./build.sh firmware`

Generate image file: output/image/rootfs.img

3.7 Build media

```
./build.sh clean media
./build.sh media
```

The storage directory of the generated files: output/out/media_out

3.8 Build Reference Applications

```
./build.sh clean app
./build.sh app
```

The storage directory of the generated file: output/out/app_out

Note: app depends on media

3.9 Build Kernel's Driver

```
./build.sh clean driver
./build.sh driver
```

The storage directory of the generated file: output/out/sysdrv_out/kernel_drv_ko/

3.10 Packaging env.img

```
./build.sh env
```

Use uboot's mkenvimage tool to package env.img

The command format: `mkenvimage -s $env_partition_size -p 0x0 -o env.img env.txt`

NOTICE: The different storage medium, **\$env_partition_size** is different, please refer to [Partition Table Introduction](#) for details.

To see the details of env.img: `strings env.img`

```
# for example, the content of eMMC's env.txt
blkdevparts=mmcblk0:32K(env),512K@32K(idblock),256K(uboot),32M(boot),2G(rootfs)
,1G(oem),2G(userdata),-(media)
```

NOTICE: The different storage medium, env.img is different. Use `strings env.img` to see the details.

The blkdevparts will be transferred from the uboot to kernel and rewrite kernel's bootargs parameter.

3.11 Firmware Packaging

```
./build.sh firmware
```

The path of the generated files: output/image

3.12 SDK Directory Structure

Directory Path	Introduction
build.sh	SDK compilation script soft-link to project/build.sh
media	Multimedia codec, ISP, etc. Algorithm related
sysdrv	U-Boot, kernel, rootfs directory
project	Reference applications, build configuration and script directory
docs	SDK Documents directory
tools	Image packaging tools and Burning tools
output	The directory where the image files are stored after SDK compilation
output/image	Firmware images directory
output/out	Files generated after compilation
output/out/app_out	Files generated after reference applications compilation
output/out/media_out	Files generated after media files compilation
output/out/sysdrv_out	Files generated after sysdrv files compilation
output/out/sysdrv_out/kernel_drv_ko	Ko files for peripherals and multimedia
output/out/rootfs_XXX	Rootfs source
output/out/S20linkmount	Partition mount script
output/out/userdata	userdata

Notice: media and sysdrv can be compiled independently of the SDK.

3.12.1 Sysdrv Directory Introduction

If config the file of `sysdrv/cfg/cfg.mk`, sysdrv can be built independently of the SDK and includes U-Boot, kernel, rootfs, and some image packaging tools.

Build Command:

```
# Build all default
make all

# Build U-Boot
make uboot_clean
make uboot

# Build kernel
make kernel_clean
make kernel

# Build rootfs
```

```

make rootfs_clean
make rootfs

# Clean
make clean

# Clean and remove out directory
make distclean

# Get the information of Build configuration, e.g. uboot or kernel compilation
command
make info

```

sysdrv subdir	Introduction
cfg	config kernel and U-Boot
out	sysdrv compilation files output directory
out/bin/board_XXX	programs which run on the board
out/bin/pc	programs which run on PC
out/bin/image_XXX	the output directory of generated firmware images
out/bin/rootfs_XXX	rootfs source files
source/busybox	busybox compilation directory, the source is located in sysdrv/tools/board/busybox
source/busybox/objs_config_normal	the objects of busybox
source/busybox/objs_config_tiny_arm	the busybox building objects for Recovery
source/objs_kernel	the directory of linux kernel objects
source/kernel	the directory of linux kernel source
source/uboot	the directory of U-Boot and rkbin (ddr init)
tools/board	the directory of tools source of board
tools/pc	Tools used on PC

3.12.2 Media Directory Introduction

The media directory can be compiled independently of the SDK and includes multi-media encode and decode, ISP image algorithm.

Compiling Command:

```
# Compile all default
make

# Clean
make clean

# Get the information of compilation configure
make info
```

media subdir	Introduction
cfg	config whether to build the module
alsa-lib	Advanced Linux Sound Architecture (ALSA) library
common_algorithm	audio 3A algorithm, move detect, occlusion detect
isp	Image Signal Processing
iva	Intelligence Video Analysis
libdrm	Direct Rendering Manager
libv4l	video4linux2 library
mpp	encode and decode interface, used for rkmedia and rockit, Not Recommended to call MPP directly
rga	Raster Graphic Acceleration Unit
rockit	Interface of multi-media (Recommend)
sysutils	Peripheral interface (ADC/GPIO/TIME/WATCHDOG)
samples	Test samples
out	The output directory of media compilation

3.12.3 App Directory Introduction

App directory: project/app

project/app subdir	Introduction
component/rkadk	rkadk has packaged basic commonly used interfaces, such as video recording, photography, playback, preview, etc., which simplifies the difficulty of application development.
component/rkfsmk	Optimize storage-related libraries (include FAT32 format, FAT32 filesystem repair, MP4 file repair)
component/lvgl	Light and Versatile Embedded Graphics Library
rkipc	The reference application for IPC
wifi_app	The reference application for Wi-Fi
uvc_app_tiny	The reference application for UVC
ipweb	The reference application for Web

3.13 The Output Directory of Images

The images after compiling SDK are located in output/image directory.

Image name	Introduction
download.bin	Will Only be downloaded to the DDR of the board
env.img	include partiton table and boot parameter(The env partiton is located at address 0 in the SDK by default)
idblock.img	loader image (include DDR init), used to load U-Boot image
uboot.img	uboot image
boot.img	Linux kernel image
rootfs.img	rootfs image
oem.img	oem image (Optional)
userdata.img	userdata image (Optional)

3.14 Debugging Tools

The images compiled by the SDK support adb and tftp tools for PC and board file transfer.

3.14.1 Transfer Files via Network tftp

```

### Get the IP address 192.168.1.159 of the PC
### Download files from the tftp server on the PC to the board
cd /tmp
tftp 192.168.1.159 -g -r test-file

### Upload files from the board to the tftp server on the PC
tftp 192.168.1.159 -p -l test-file

```

NOTICE: tftp server configuration refers to [How to Use TFTP Upgrade in U-Boot Terminal](#)

3.14.2 Debug via ADB

```

### Get the IP address 192.168.1.159 of the EVB board
adb connect 192.168.1.159

adb devices
List of devices attached
192.168.1.159:5555      device

### adb login EVB board to debug
adb -s 192.168.1.159:5555 shell

### Upload the file test-file from the PC to the directory /userdata of the EVB
board
adb -s 192.168.1.159:5555 push test-file /userdata/

### Download the file /userdata/test-file on the EVB board to the PC
adb -s 192.168.1.159:5555 pull /userdata/test-file test-file

```

3.14.3 Debug via Telnet

```

### Set the IP address
udhcpc -i eth0

### Run telnetd on board
telnetd

```

```

### Get the IP address 192.168.1.159 of the EVB board
### Run telnet on PC
telnet 192.168.1.159
### username: root
### password: rockchip

```

3.14.4 Debug via Serial Port

Chip Name	Serial Port Configure
RV1106B/RV1103B	Baudrate:115200, Data Bits:8, Parity:None, Stop Bits:1, Flow Type:None

4. Documents Introduction

```
docs/
├── zh ----- SDK documents in Chinese
│   ├── bsp
│   ├── isp
│   ├── npu_iva
│   ├── media
│   ├── security
│   └── ipc/Rockchip_RV1106B_RV1103B_Quick_Start_Linux_IPC_SDK_CN.pdf --- SDK
quick start in Chinese
└── en ----- SDK documents in English
    ├── bsp
    ├── isp
    ├── npu_iva
    ├── media
    ├── security
    └── ipc/Rockchip_RV1106B_RV1103B_Quick_Start_Linux_IPC_SDK_EN.pdf ---SDK
quick start in English
```

5. Tools Introduction

Tools which are used for debugging and mass production are released with Rockchip Linux IPC SDKs for development. Tools version will be continuously updated with the SDK update. If you have any questions or requirements on the tools, please contact our FAE window fae@rock-chips.com.

There are two versions of tools: linux (tools used in Linux operating system) and windows (tools used in Windows operating system in the tools directory of Rockchip Linux IPC SDK).

- Windows Tools

Tools documents: tools/windows/ToolsRelease.txt

The Name of Tools	Usage of Tools
SocToolKit	Firmware update and used for entire update
DriverAssitant	Driver installation tool

- Linux tools

Tools related document: tools/linux/ToolsRelease.txt

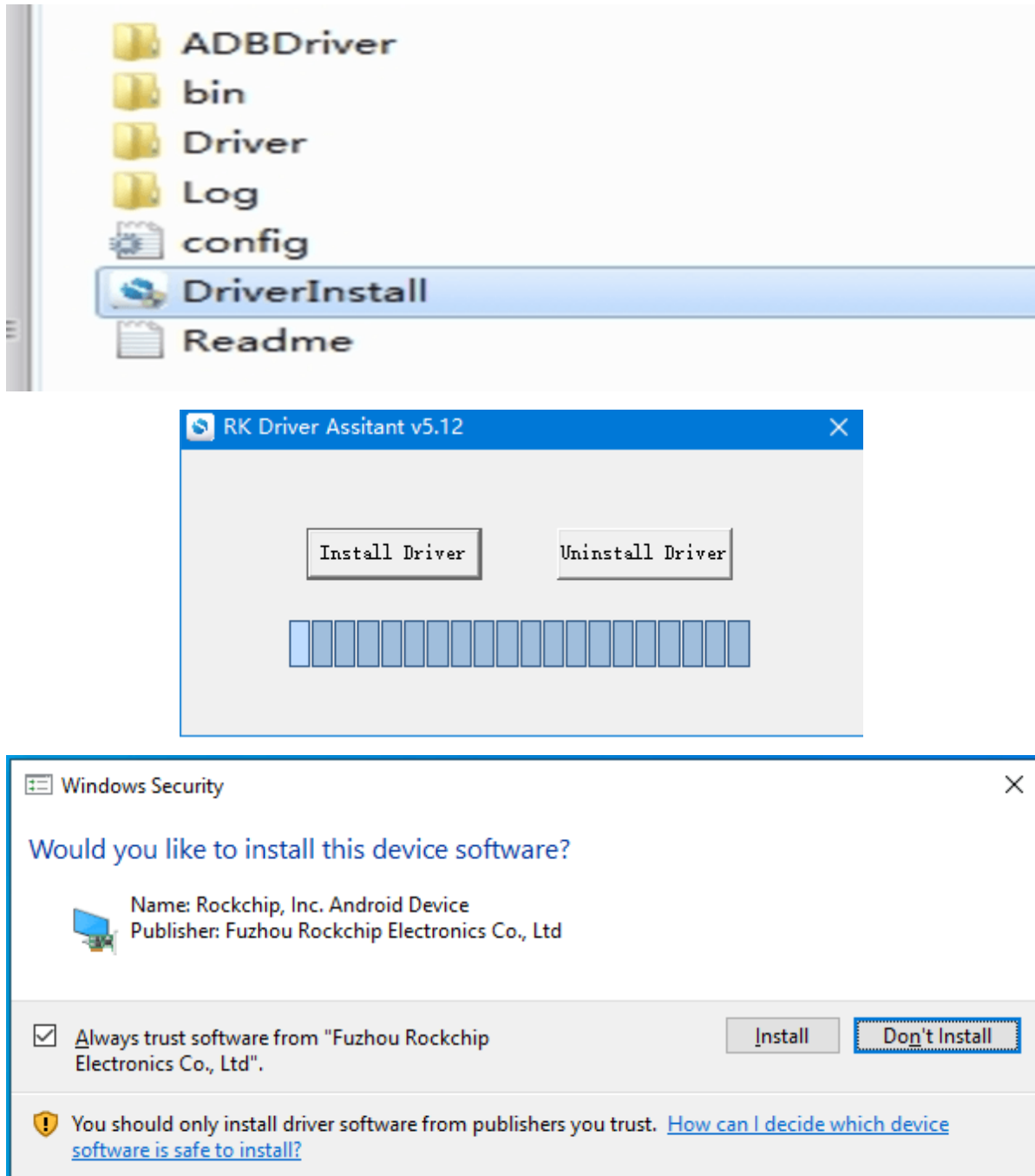
Tool Name	Tool Purpose
SocToolKit	Firmware update and used for entire update
Linux_Upgrade_Tool	Firmware update for command-line interface (only support USB)

5.1 Driver Installation Tool

Rockchip USB driver installation assistant is stored in

`<SDK>/tools/windows/DriverAssitant_<version>.zip`. support
win7_64, win10_64 and other operating systems.

The installation steps are as follows:



5.2 Firmware Burning

- The method of switching burn mode:

Long press the "Update" button and press "RESET" button at the same time and then release, device will enter Maskrom mode.

- The SDK provides Windows flashing tools , the tool is located in the project root directory:

If the board has flashed firmware, we can enter U-Boot to upgrade firmware. For detailed instructions, please refer to the following chapters:

[How to Use TFTP Upgrade in U-Boot Terminal](#)

[How to Use SDcard Upgrade in U-Boot Terminal](#)

```
<SDK>/Tools/windows/SocToolKit/SocToolKit.exe
```

Rockchip Soc ToolKit v1.98

功能 帮助

固件下载 固件制作 SD卡工具

☒ USB设备 ☐ 串口设备 波特率:

名字	地址	路径
DownloadBin		

选择芯片

支持的芯片列表

RV1103B

OK Cancel

下载 高级功能

☒ 重启设备 下载

搜索路径...

固件... 升级

Rockchip Soc ToolKit v1.98(RV1103B)

Tool Help

Download Firmware SDTool

Step1: Select Download

Step2: Select USB or COM

Step3: Select the firmware directory

Step4: Start to Download

USB Maskrom 12 Baudrate:

COM

Name	Address	Path
DownloadBin		download.bin
env	0x00000000	\\10.10.10.190\cww\ipc-sdk\rv1103b\output\image\env.img
idblock	0x00040000	idblock.img
uboot	0x00140000	uboot.img
boot	0x00240000	boot.img
rootfs	0x00A40000	rootfs.img
oem	0x02240000	oem.img
userdata	0x04240000	userdata.img

Download Advanced Feature

reset Download

\\10.10.10.190\cww\ipc-sdk\rv1103b\output\image Search Path...

Firmware... Upgrade

5.3 Update.img Related Tool

5.3.1 Packaging

When the SDK automatically compiles (`./build.sh`) with one click, the firmware to be flashed will be packaged into `update.img` and stored in the `<SDK>/output/image` directory. Alternatively, you can also run the following command to manually package the firmware in the preceding directory:

```
./build.sh updateimg
```

To customize the firmware directory, you can manually run the package script. View the help (-h) or enter the following options:

```
<SDK>/tools/linux/Linux_Pack_Firmware/mk-update_pack.sh -id <RK_CHIP> -i  
<IMAGE_DIR>
```

5.3.2 Unpacking

This function needs to be run manually. You can unpack the `<SDK>/output/image/update.img` into discrete firmware and save it in the `<SDK>/output/image/unpack` directory. The unpacking command is as follows:

```
./build.sh unpackimg
```

To customize the firmware path, you can manually run the unpacking script. View the help (-h), or enter the following options:

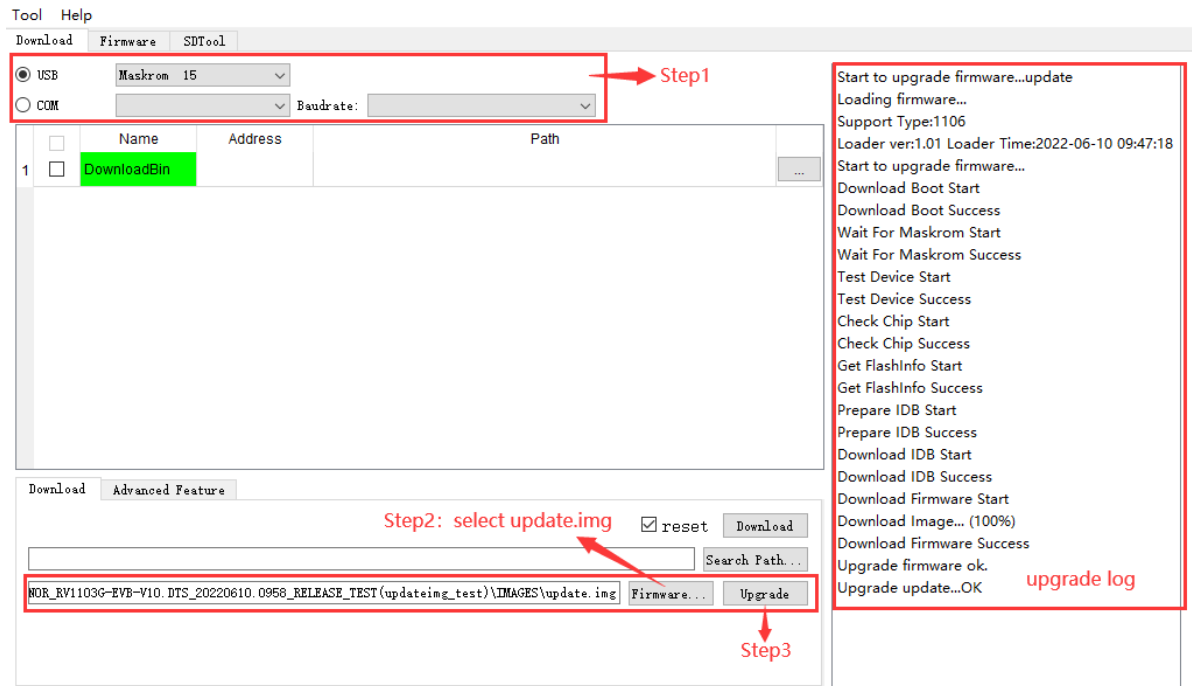
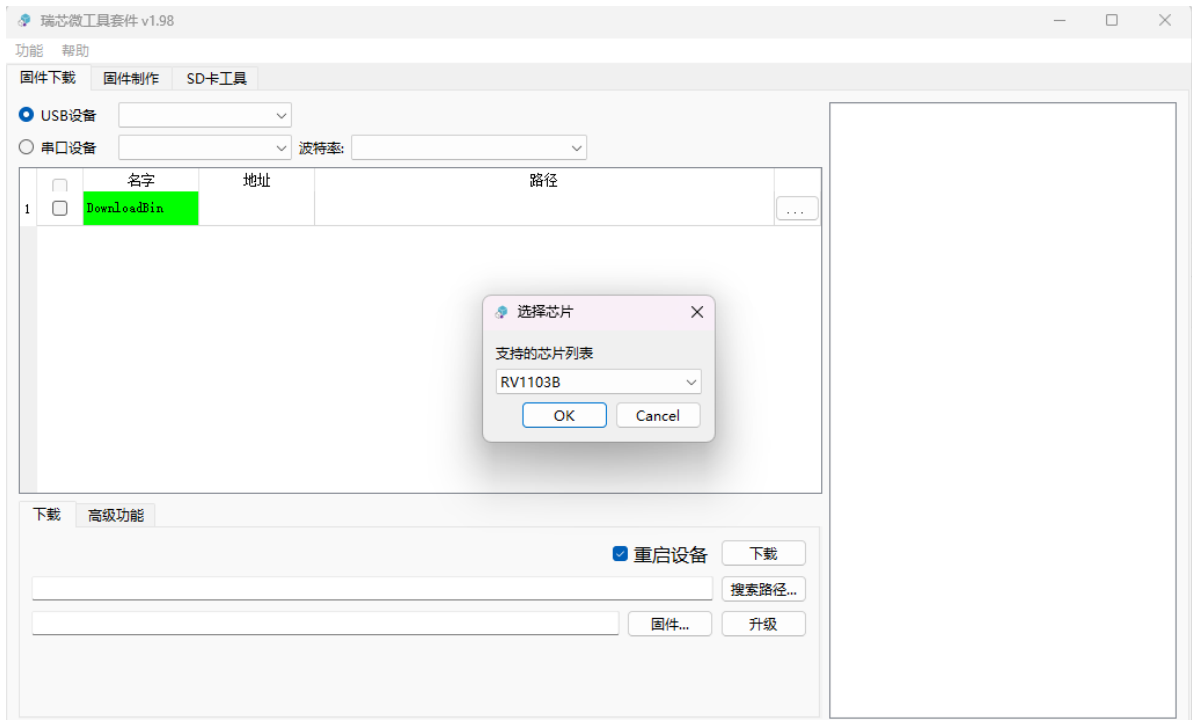
```
<SDK>/tools/linux/Linux_Pack_Firmware/mk-update_unpack.sh -i <IMAGE_PATH> -o  
<UNPACK_DIR>
```

5.3.3 Update.img Flashing

- The SDK provides Windows flashing tools , the tool is located in the project root directory:

```
<SDK>/Tools/windows/SocToolKit/SocToolKit.exe
```

NOTICE: SocToolKit 1.99 or later is required to support the `update.img` flashing function.



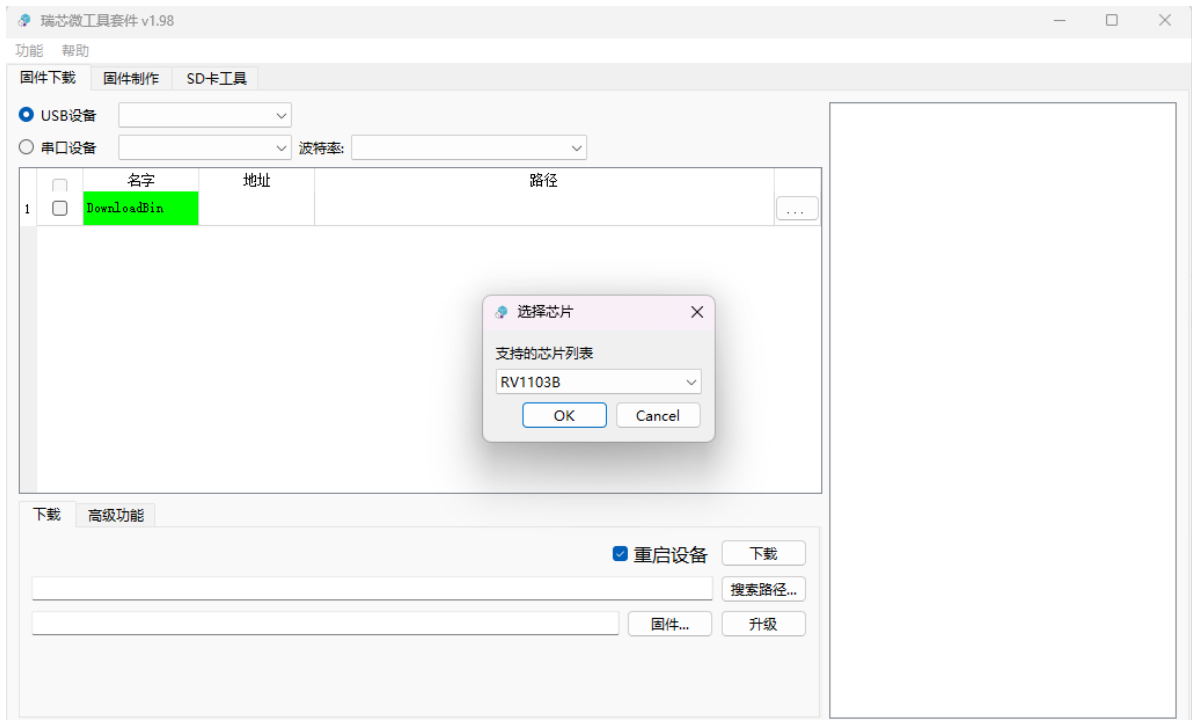
5.4 SD Card Upgrade and Booting Making Tool

- The SDK provides Windows SD card upgrade and booting making tool, the tool is located in the project root directory.

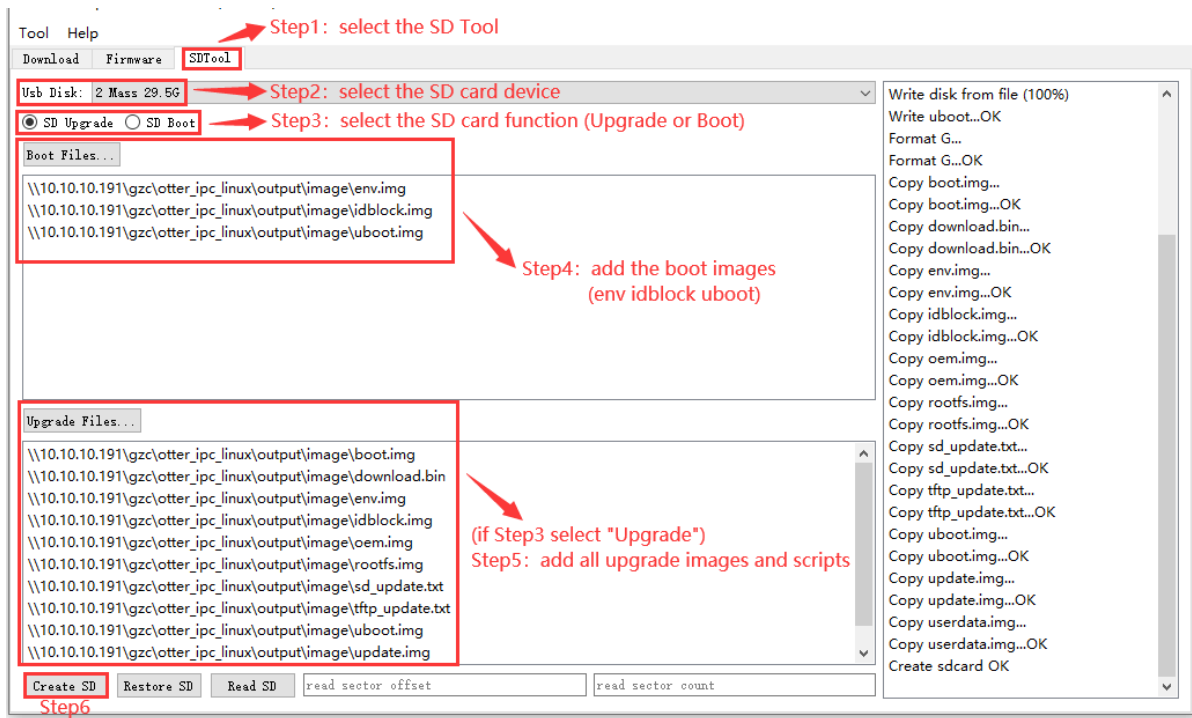
NOTICE: **SD Booting** ONLY support eMMC firmware.

```
<SDK>/Tools/windows/SocToolKit/SocToolKit.exe
```

NOTICE-1: SocToolKit 1.99 or later is required to support the SD card upgrade and booting function.



NOTICE-2: This function requires the user to run SocToolKit.exe as an administrator (asked by default when the tool is started).



- After inserting the SD card and restarting the device, the device preferentially accesses the U-boot terminal in the SD card.
- If the SD card has the upgrade function, the device is automatically upgraded.
- After the upgrade, remove the SD card and restart the device to access the device system.

5.5 Factory Firmware Introduction

The `programmer_image_tool` is required to make factory firmware. The tool is located in the `<SDK>/tools/linux/SocToolKit/bin/linux` directory. The input image is `update.img`. For details, see [Update.img Related Tool](#).

The doc of programmer_image_tool is located in

```
<SDK>/tools/linux/SocToolKit/bin/linux/programmer_image_tool.pdf.
```

The SDK supports building part of the flash type factory firmware. The generated firmware is stored in the

```
<SDK>/output/image/factory
```

 directory. **For other flash types of firmware, see the document `Rockchip_Developer_Guide_Linux_Nand_Flash_Open_Source_Solution_CN.pdf` and use the tool `programmer_image_tool` to generate them.**

- For SPI NAND, block size and page size require to be configured:

Config on the BoardConfig file:

```
export RK_NAND_BLOCK_SIZE=0x20000
export RK_NAND_PAGE_SIZE=2048
```

- SDK compilation command:

```
# building firstly
./build.sh all

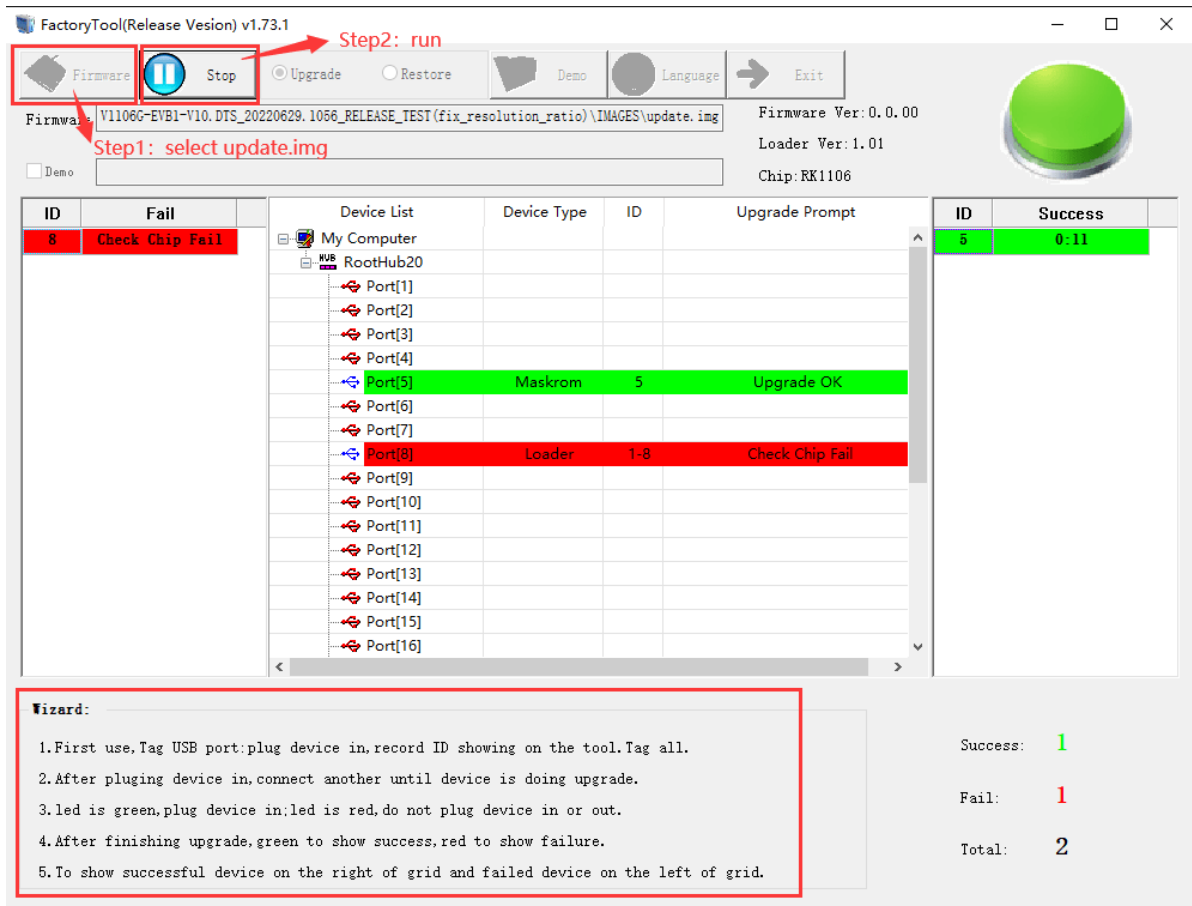
# packge udpatch.img
./build.sh updateimg

# generate factory's firmware: output/image/factory
./build.sh factory
```

For more information about debugging and handling storage-related issues, please refer to <https://redmine.rock-chips.com/documents/108>

5.6 Mass Production Upgrade Tool

Mass production upgrading needs the `FactoryTool`. The tool is located in the `<SDK>/tools/windows` directory. The input image is update.img. For details, see [Update.img Related Tool](#).



When the tool is running, the upgrade can be started automatically as long as the computer and the devices are connected via USB and the devices are in Maskrom or Loader mode. No further operations are required.

Other content can be found in the wizard below the tool.

6. CVR Framework Introduction

6.1 Overview of Software Functions

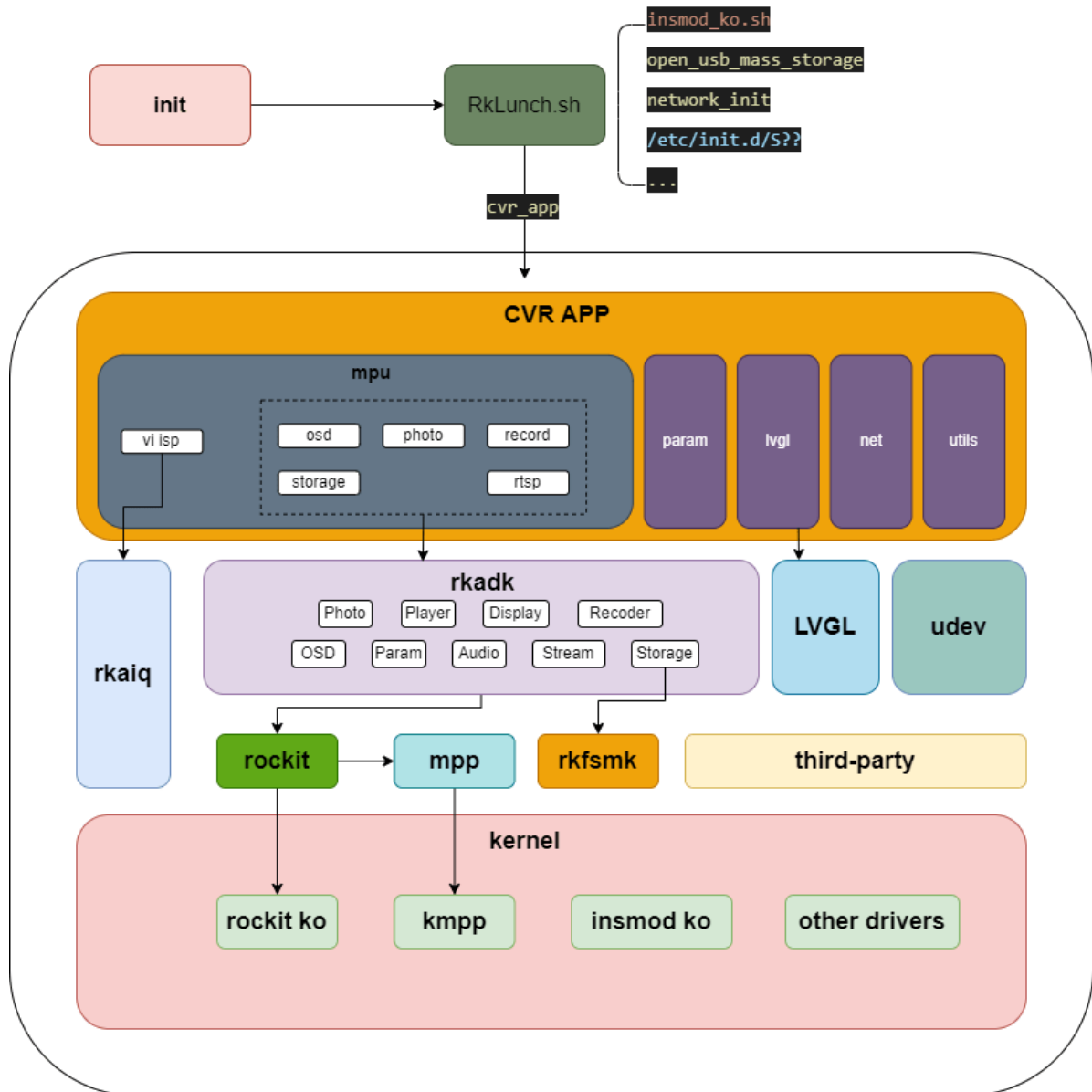
The CVR SDK developed by Rockchip provides the following basic functions for the development of CVR-related products:

- **Video Recording:** loop recording, file segmentation, pre-recording, playback, time-lapse recording, multi-channel recording;
- **Photo Shooting:** photographing, thumbnail generation, photo playback;
- **Player:** playback of video and audio files, support for playback control operations such as play, pause, and seek;
- **Live Streaming:** provide basic services for standard RTSP live streaming, and support RTMP live streaming. RTSP and RTMP cannot be started at the same time;
- **Storage:** pre-allocation of storage space, write optimization, file repair, formatting, and media file management;
- **Preview:** real-time video preview, providing access to video and audio information, enabling and disabling VENC and AENC, and registering a callback interface for processing audio and video frame data;
- **OSD:** providing basic watermark functions;
- **Quick Start:** fast startup, snapshot, quick streaming (to be supported in later versions);
- **UI-related:** LVGL (Lightweight and Versatile Graphics Library) is a free and open-source graphics library;

- **Image Effects:** optimization of image effects for CVR scenarios;
- **Algorithm:** support for commonly used car-related algorithms such as ADAS, DMS, and BSD (not included in the SDK, third-party algorithm adaptation is required);
- **Audio:** support for audio noise reduction and echo cancellation.

6.2 Software Framework

As shown in the figure, after the system enters the user layer from kernel, the init process is started. The init process will run RkLunch.sh ((project/app/cvr/cvr/RkLunch.sh sets up the runtime environment required by cvr_app), and then start cvr_app. cvr_app package calls rkaiq, rkadk, lvgl, udev, and other modules, and packages data stream processing and various interfaces required by CVR business logic.



Module Name	Function Description
kernel	Version 5.10
rkadk	Core framework that provides APP function APIs, document path: /project/app/component/rkadk/Rockchip_Developer_Guide_Linux_RKADK_CN.pdf
rockit	Multimedia framework API
rkaik	ISP image processing API
rkfsmk	Formatting and file repair of the memory card
LVGL	UI framework
cvr_app	Demo APP

6.3 APP Introduction

`cvr_app` is a demo program included with the CVR SDK, designed specifically for CVR products, implementing standard functionalities. It is divided into versions with and without a screen.

The screen-display version adds UI display, video playback, and recording playback features on top of the version without a screen. The SDK defaults to the version without a screen. To enable the screen-display version, please refer to the following modifications:

```
diff --git a/BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-CVR.mk
b/BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-CVR.mk
index 94ba391..dbd6121 100644
--- a/BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-CVR.mk
+++ b/BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-CVR.mk
@@ -92,7 +92,7 @@ export RK_NAND_PAGE_SIZE=2048
 export RK_PRE_BUILD_OEM_SCRIPT=rv1103b-pre-oem-post.sh

# enable lvgl
-export RK_ENABLE_LVGL=n
+export RK_ENABLE_LVGL=y

# enable libdrm
export RK_ENABLE_LIBDRM=y
```

6.3.1 Version Without Screen

- `cvr_app` starts during boot via `RKLunch.sh`, reads the INI configuration file located in the `/userdata/rkadk/` directory, and upon completion of initialization, begins recording. Recorded files are stored in the `/mnt/sdcard` directory. If an TF card is inserted for the first time, formatting is required using the `KEY2` button on the EVB. Upon successful formatting, the following log information will be displayed:

```
Formatting is complete, time-consuming: 4719.220000 ms.
```

- Triggering `cvr_app` via the `KEY1` button on the EVB allows for capturing photos, which are saved in the `/mnt/sdcard/photo` directory.
- Adjustments to the initialization parameters during the runtime of `cvr_app` can be made by modifying the INI configuration file. For detailed configurations, please refer to the rkadk documentation.

`<SDK>/project/app/component/rkadk/Rockchip_Developer_Guide_Linux_RKADK_CN.pdf`

6.3.2 Version with Screen Display

The version with screen display currently does not support touch functionality and can only be operated via buttons. Below are the functions corresponding to the buttons on the EVB:

- `KEY1` button corresponds to the `TAB` function, used to focus on different controls.
- `KEY2` button corresponds to the `Enter` function, used to select the focused control.

If you need to format the TF card automatically, you can use the following command: `cvr_remote_cfg_tool -f`.

6.3.3 Functional Module Introduction

```
.
└─cvr
    │
    │ └─ res
    │   │
    │   │ └─ conf          # APP configuration file
    │   │ └─ font          # Font resource
    │   │ └─ sound
    │   │ └─ string        # Multilingual text
    │   │ └─ ui            # icon
    │   └─ src
    │     │
    │     │ └─ algorithm    # Integrated algorithm
    │     │ └─ app          # Application processing module: including buttons,
    │     │                 hot plugging, and automated test processing
    │     │ └─ conf          # User parameter management
    │     │ └─ core          # System management
    │     │ └─ lvgl          # UI framework: driver for display and input
    │     └─ devices required by LVGL
    │       │
    │       │ └─ mpu          # Multimedia module, based on rkadk: including ISP,
    │       │                 OSD, photography, video, storage, RTSP related applications
    │       │ └─ net
    │       │ └─ ui            # UI source code
    │       │ └─ utils        # Common components of code
    │       └─ tools
    │         │
    │         │ └─ ntp          # NTP time synchronization tool
    │         │ └─ remote_config # Automated testing tool
    │         │ └─ script_job
    │         └─ sound</left>
```

6.4 Introduction to LVGL

LVGL is the most popular free, open source embedded graphics library that can be used to create beautiful user interfaces for any MCU, MPU and display type related product. The UI of `cvr_app` is built using LVGL, and the LVGL source code is provided in the SDK. For detailed documentation on using the interface, please refer to the official website <https://lvgl.io/>.

6.4.1 Enable LVGL

The SDK disables LVGL by default. If you need to enable it, please refer to the following modifications:

```
diff --git a/BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-CVR.mk
b/BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-CVR.mk
index 94ba391..dbd6121 100644
--- a/BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-CVR.mk
+++ b/BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-CVR.mk
@@ -92,7 +92,7 @@ export RK_NAND_PAGE_SIZE=2048
 export RK_PRE_BUILD_OEM_SCRIPT=rv1103b-pre-oem-post.sh

# enable lvgl
-export RK_ENABLE_LVGL=n
+export RK_ENABLE_LVGL=y

# enable libdrm
export RK_ENABLE_LIBDRM=y
```

6.4.2 LVGL Configuration File

To configure LVGL as required, it can effectively reduce resource usage. Copy the `/project/app/component/lvgl/lvgl/lv_conf_template.h` file located next to the LVGL directory and rename it to `lv_conf_XXX.h`. Open the file and change the `#if 0` at the beginning to `#if 1` to enable its content. The comments in the configuration file explain the meanings of the options. After the configuration is completed, set the `LVGL_SRC_CONF` in the `project/app/component/lvgl/Makefile` as the path where `lv_conf_XXX.h` is stored.

```
# project/app/component/lvgl reference code
...
PKG_NAME := lvgl
PKG_BIN ?= out
PKG_BUILD ?= build
PKG_CFG := rockchip-conf #配置文件存放目录

LVGL_DEF_CONF := $(CURRENT_DIR)/$(PKG_NAME)/$(PKG_CFG)/lv_conf_default.h #默认
配置文件路径
LVGL_DST_CONF := $(CURRENT_DIR)/$(PKG_NAME)/lv_conf.h

# 根据选中板级设置不同的配置文件，更改配置文件路径修改LVGL_SRC_CONF
ifneq (, $(filter RKCVR_RV1106, $(RK_APP_TYPE)))
LVGL_SRC_CONF := $(CURRENT_DIR)/$(PKG_NAME)/$(PKG_CFG)/lv_conf_rv1106_cvr.h
else ifneq (, $(filter RKCVR_RV1103B, $(RK_APP_TYPE)))
LVGL_SRC_CONF := $(CURRENT_DIR)/$(PKG_NAME)/$(PKG_CFG)/lv_conf_rv1103b_cvr.h
```

```
else ifneq (, $(filter FACIAL_GATE_RV1106_NORMAL, $(RK_APP_TYPE)))
LVGL_SRC_CONF := $(CURRENT_DIR)/$(PKG_NAME)/$(PKG_CFG)/lv_conf_facialgate.h
else ifneq (, $(filter RKCVR_RV1126, $(RK_APP_TYPE)))
LVGL_SRC_CONF := $(CURRENT_DIR)/$(PKG_NAME)/$(PKG_CFG)/lv_conf_rv1126_cvr.h
else
LVGL_SRC_CONF := $(LVGL_DEF_CONF)
endif

...
```

6.5 rkadc Introduction

The rkadc has further packaged on Rockit. It provides a set of common components for multimedia recording, including video recording, photography, playback, preview, media file management, etc. These components help simplify the development process and allow for quick development of multimedia applications. With the interface of the rkadc module, customers can quickly develop products with functions such as video recording and photography.

Please note that this component only supports single-process functionality. Unless specifically stated otherwise, it does not support simultaneous use across multiple processes.

6.5.1 rkadc Feature List

Feature Name	Feature Description
Recording	1. Create and destroy recording tasks; 2. Start and stop recording tasks; 3. Manually split recording files; 4. Pre-recording; 5. Time-lapse recording; 6. Multi-channel recording; 7. MP4 file thumbnail
Photography	1. Single-shot mode; 2. Continuous shooting mode; 3. Photo thumbnail;
Media Stream	1. RTSP; 2. RTMP;
Player	1. Play videos; 2. Play images; 3. Play audio; 4. Support for play, pause, and seek;
Media File Management	1. Scan multimedia files; 2. Repair multimedia files; 3. Get multimedia file list; 4. Format storage devices, query device capacity and status;
Watermark Overlay	1. Overlay watermark on the main and sub-stream recordings; 2. Overlay watermark on photos; 3. Overlay watermark on remote and local previews; 4. Overlay watermark on RTSP and RTMP;
Parameter Management	1. Set and get module parameters; 2. Save specified parameters; 3. Restore default parameters;
local preview	1. Provide local preview function;

6.5.2 rkadk API Description

For detailed description of the rkadk module API interface, please refer to the flollowing documentation:

`<SDK>/project/app/component/rkadk/Rockchip_Developer_Guide_Linux_RKADK_CN.pdf`

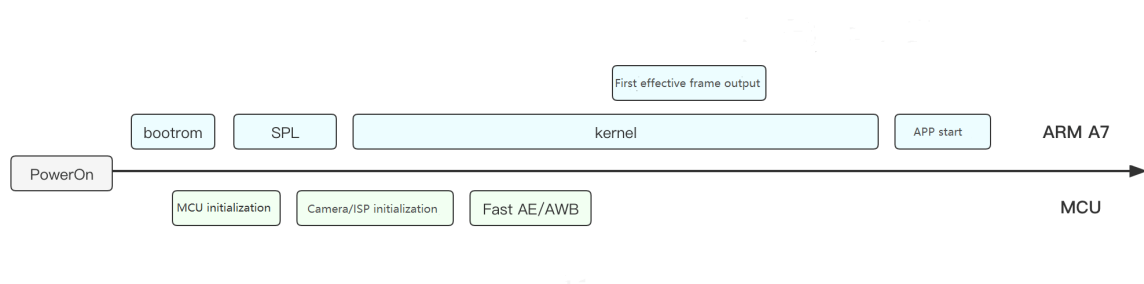
7. Instruction of Battery Fastboot IPC

7.1 Fastboot Process Introduction

7.1.1 Basic Framework of Fastboot Process

There is a high-performance MCU inside the Chip, during booting process, the high-performance MCU will initialize camera and ISP instead of ARM, to quickly capture the effective first frame.

The basic framework of fastboot process of Chip is shown as follow:



During the process, Uboot is disabled and load kernel by SPL directly. If Uboot is enabled, the boot speed Will be affected.

7.1.2 FastBoot FW config and compile

The BoardConfig directory is project/cfg/BoardConfig_BatteryIPC/. You can refer to this directory for customizing the configuration firmware. This section uses BoardConfig-SPI_NOR-NONE-RV1106B_EVB2_V10_V11-BAT_IPC.mk as an example to describe how to enable fast startup for RV1103B+SC200AI.

1.RTT MCU firmware

```
# Uboot Loader ini overlay
# build.sh mcu info --> rv1103b_evb2-SC200AI-ADC
export RK_UBOOT_RKBIN_MCU_CFG=rv1103b_evb2-SC200AI-ADC
```

2.sensor iqbin file

```
# Config sensor IQ files
# RK_CAMERA_SENSOR_IQFILES format:
#     "iqfile1 iqfile2 iqfile3 ..."
# ./build.sh media and copy <SDK root
dir>/output/out/media_out/isp_iqfiles/$RK_CAMERA_SENSOR_IQFILES
export RK_CAMERA_SENSOR_IQFILES="thunderboot/sc200ai_default_default.bin"
```

3.fastboot app

```
# app config
export RK_APP_TYPE="RK_FASTBOOT_SINGLE_PROCESS"
#export RK_APP_TYPE="RK_FASTBOOT_DUAL_PROCESS"
```

4.enable fastboot

```
# enable fastboot
export RK_ENABLE_FASTBOOT=y
```

5.meta parameter configuration, used to configure the camera operating parameters in rtt and kernel state.

```
export RK_META_PARAM=" \
--rk_cam_mirror_flip=0 \
--rk_cam_fps=15 \
--rk_cam1_max_fps=60 \
--rk_cam_hdr=0 \
--rk_led_value=20 \
--rk_night_mode=2 \
--rk_rtt_stream_on_threshold=15 \
--rk_rtt_stream_buf_cnt=2 \
"
```

7.1.3 Camera Fastboot Process Introduction

7.1.3.1 Dual-camera Fastboot Demo Introduction

The Dual-camera Fastboot demo is a single process.

Code path: project/app/fastboot_client/fastboot_dual_camera_demo.c

Install script: project/cfg/BoardConfig_BatteryIPC/tb-dual-cam-spi_nor-post.sh

Boot process: /etc/init.d/rcS

7.1.3.2 Single Camera Boot-up Demo Introduction

The Singal-camera Fastboot supports single-process and dual-process.

1.single-process code path: project/app/fastboot_client/fastboot_demo.c

2.dual-process Code path: project/app/component/fastboot_server/fastboot_server.c,
project/app/fastboot_client/fastboot_client.c

In BoardConfig, specify the application type through RK_APP_TYPE, where

RK_FASTBOOT_SINGLE_PROCESS indicates a single process, and RK_FASTBOOT_DUAL_PROCESS indicates a dual process.

Boot process: /etc/init.d/rcS

7.1.3.3 View the stream video

1. Use streaming software to preview the stream video

```
# rndis.sh
serialnumber is 0000000000000000
config usb0 IP..

# ifconfig
usb0      Link encap:Ethernet  HWaddr 4A:FC:84:D6:A4:1A
          inet addr:192.168.1.100  Bcast:192.168.1.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:80 errors:0 dropped:63 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:12637 (12.3 KiB)  TX bytes:0 (0.0 B)
```

Visit server ip addr

rtsp://192.168.1.100/live/0

2.View the first 30 frames of the stream

The SDK's built-in fastboot application saves the first 30 frames of the stream data by default, which is stored in /tmp/venc0.bin. You can pull the stream data via TFTP or SZ."

```
tftp -pl /tmp/venc0.bin 192.168.1.101
```

Or

```
sz /tmp/venc0.bin
```

7.1.4 Fastboot Log Introduction

The MCU's logs aren't printed into the UART, need to run the command `make_meta --rtt-log` to print MCU log.

```
##### DDR Version Info #####
DDR a0d2414c29 cym 24/10/14-16:17:47, fwver: v1.05
...

##### MCU Version Info #####
version info: 2024-10-15 10:02:47.814183 71b560cee3(rtt) 8fae34de(hal)
...

##### idblock (SPL) Version Info #####
U-Boot SPL 2017.09-gac6d8253981-240913 #lzw (Oct 15 2024 - 10:03:01)
...

##### idblock Total Elapsed Time #####
Total: 93.118/102.148 ms
...

##### MCU fastAE Log #####
[STREAM]: L 0, 1, 0, 0x869000, 783360, 0x22e, 0x1ac40, tick:37
[AELIB]:
1,luma=245.60,setP=28.00,curE=79.90051,newE=14.47106,T=0.00827,G=1713.000,ispG=
1.021,regT=558,regG=109632,dcg=0
```

```
[STREAM]: L 1, 2, 0, 0x929000, 783360, 0x22e, 0x1ac40, tick:45
[AELIB]:
2,luma=245.60,setP=28.00,curE=79.90051,newE=2.27621,T=0.00827,G=275.228,ispG=1.
000,regT=558,regG=17615,dcg=0
[STREAM]: L 2, 3, 0, 0x9e9000, 783360, 0x22e, 0x1ac40, tick:53
[AELIB]:
3,luma=239.00,setP=28.00,curE=56.62541,newE=4.42213,T=0.00827,G=534.702,ispG=1.
000,regT=558,regG=34221,dcg=0

##### MCU fastAE Matching Log #####
=====fastae is match=====

...

##### MCU System Exit Log, the log mean that MCU exit on
244ms #####
[STREAM]: will go to wfi, tick:244!
```

7.1.5 Introduction to the time-consuming statistics

Hardware Conditions: The LED/PA DIP switch needs to be turned ON.

The time consumption of each stage of fastboot is saved in the /tmp/thunderboot_time.txt file. The following is a detailed explanation.

1.rtt Application startup sequence and streamOn time statistics.

```
[thunderboot_time] rtt app enter tick: 0(ms)
[thunderboot_time] set camera initial regs enter tick: 0(ms)
[thunderboot_time] set camera initial regs exit tick: 6(ms)
[thunderboot_time] camera stream on enter tick: 10(ms)
[thunderboot_time] camera stream on exit tick: 11(ms)
```

2.rtt AE match time statistics

```
[thunderboot_time] fastae 4 is match tick: 52(ms)
```

3.rtt switch resolution time statistics

```
[thunderboot_time] camera switch resolution enter tick: 52(ms)
[thunderboot_time] camera switch resolution exit tick: 58(ms)
```

4.init process launch finish

```
[ 0.176237] Run /sbin/init as init process
```

5.app launch

```
[ 0.200728] [app]: [thunderboot_time] fastboot_demo enter
```

6.codec start

```
[ 0.286846] [thunderboot_time] venc run task begin.
```

7.codec finish

```
[ 0.306572] [thunderboot_time] venc done.
```

8.app process get first frame

```
[ 0.358040] [app]: [thunderboot_time] get venc all reserved frames
```

7.2 Ambient Light Sensor Introduction

7.2.1 Ambient Light Sensor Type

Ambient Light Sensor Type	Introduction
Software Light Sensor	There is no hardware ambient light sensor. Use software obtaining image brightness continuously to calculate environment brightness.
Analog Light Sensor	Default solution of the SDK. There are different MCU firmwares for different camera sensor. For details, please refer to How to Modify Camera Sensors Supported by the Firmware
Digital Light Sensor	Need to rebuild MCU firmware. The SDK support LTR308.

Ambient light sensor types include software light sensor and hardware light sensor. Hardware light sensor is divided into analog light sensor and digital light sensor . Software light sensor is to obtain the brightness statistical value of ISP and use software to calculate the ambient brightness according to the AE calibration parameters. Analog light sensor reads the hardware ADC value and calculates the ambient brightness based on the calibrated ADC value, and the measurement speed is faster. The digital light sensor can directly output the ambient brightness value without calibration, but it requires a certain exposure integration time. The measurement value range and accuracy are related to the exposure gain and integration time. The digital light sensor has high measurement value range and accuracy compared to analog light sensor.

7.2.2 Ambient Light Sensor Parameters

For detailed meta partition introduction, please refer to [The meta partition Introduction](#).

The parameters related to light sensor in the meta partition mainly include ambient light sensor types (als_type), day-night switching threshold (als_value), and calibration parameters (ae_init_cfg and awb_init_cfg). For specific configuration, please refer to `project/make_meta/sensor_ae_awb_table/XX_2a.c`.

According to the ambient light sensor type, the XXsensor_init.c in the meta partition can be configured as ALS_TYPE_NONE (0: Software Light Sensor), ALS_TYPE_ANALOG (1: Analog Light Sensor), ALS_TYPE_DIGITAL (2: Digital Light Sensor). als_value indicates the ambient illumination threshold (unit: lux). When the ambient illumination is lower than this value, night vision mode will enter. It is noteworthy that

als_value in software light sensor is no longer the original ambient illuminance threshold value, and the software light sensor is the threshold value to judge the ambient brightness according to the reflection of objects. The default value is 0. After the ambient light sensor parameters are configured, the log of >>>als type: 0, als vlaue: xx, night mode: xx will be printed as a prompt in the RTT stage.

The calibration parameters of ambient light sensor parameters are introduced in detail below, The structure definition of ae_init_cfg and awb_init_cfg can be queried in

project/make_meta/include/sensor_2a_common.h.

```
#define ADC_CALIB_NO            4
#define ADC_CALIB_TBL_LEN      15
#define AE_START_EXP_NO        3
#define AE_RESERVE_NO          4
#define AWB_RESERVE_NO         60
#define FIX2INT16(x)            (int)(x * (1 << 16))
#define AE_GRID_STATS          113

struct ae_init_cfg {
    /* black level, 12bit
     * Used for mcu stage ae statistical calculation, the value can be obtained
    from IQ
     */
    uint32_t black_lvl;
    /*
     * Analog light sensor ADC value calibration type
     *    0: Formula
     *    1: LUT(look-up-table)
     */
    uint32_t adc_calib_type;
    /*
     * Analog light sensor ADC value range type
     *    0: dual
     *    1: single
     */
    uint32_t adc_range_type;
    /*
     * Analog light sensor ADC formula method calibration parameters
     * A total of 4 calibration parameters, all 10bits data
     * For dual range type:
     *    adc_calib[0] is the corresponding adc value when the low-range
    ambient brightness = 0 lux
     *    adc_calib[1] is the corresponding adc value when the low-range
    ambient brightness = 100 lux
     *    adc_calib[2] is the corresponding adc value when the high-range
    ambient brightness = 0 lux
     *    adc_calib[3] is the corresponding adc value when the high-range
    ambient brightness = 1000 lux
     * For single range type:
     *    adc_calib[0] is the corresponding adc value when the ambient
    brightness = 0 lux
     *    adc_calib[1] is the corresponding adc value when the ambient
    brightness = 100 lux
     *    adc_calib[2] is the corresponding adc value when the ambient
    brightness = 0 lux
     *    adc_calib[3] is the corresponding adc value when the ambient
    brightness = 1000 lux
     */
    uint32_t adc_calib[ADC_CALIB_NO];
```

```

/*
 * Analog light sensor ADC look-up-table method calibration parameter number
 * The maximum value is 15
 */
uint32_t adc_table_len;
/*
 * Analog light sensor ADC look-up-table method calibration parameter table
 * That is a two-dimensional array
 * adc_table_low[i][0] is the brightness value of each environment
 * adc_table_low[i][1] is the corresponding adc value
 * Environment brightness requirements are filled in ascending order
according to the actual calibration situation
 * It is recommended that the environment brightness in the first line is
0lux.
 * Note: For dual-range type, adc_table_low represents the low-range look-
up-table calibration parameter
 */
uint32_t adc_table_low[ADC_CALIB_TBL_LEN][2];
/*
 * Analog light sensor ADC look-up-table method calibration parameter table
 * That is a two-dimensional array
 * adc_table_high[i][0] is the brightness value of each environment
 * adc_table_high[i][1] is the corresponding adc value
 * Environment brightness requirements are filled in ascending order
according to the actual calibration situation
 * It is recommended that the environment brightness in the first line is
0lux.
 * Note: For the dual-range type, adc_table_high represents the high-range
look-up-table calibration parameter. For the single-range type, adc_table_high
is invalid and does not need to be filled in.
 */
uint32_t adc_table_high[ADC_CALIB_TBL_LEN][2];
/*
 * set three groups of ambient illuminance (uint: lux)
 * set the RTT initial exposure value according to the ambient illuminance
 * only for software light sensor
 */
uint32_t start_exp[AE_START_EXP_NO];
};

/*
 * The awb calibration value of night to day, which is used for the software
light sensor algorithm
 */
struct awb_init_cfg {
    /*
     * rgain reference value of night to day
     */
    uint32_t rg_gain_base;
    /*
     * bgain reference value of night to day
     */
    uint32_t bg_gain_base;
    /*
     * awb filtering radius of night to day
     */
    uint32_t awb_gain_rad;
    /*

```

```

* awb dispersion threshold night to day
*/
uint32_t awb_gain_dis;
};

```

7.2.3 Analog Light Sensor Calibration Method

Analog light sensor can be divided into single range and dual range. Single range analog light sensor, the starting point of its range can be adjusted by selecting different resistors, and the linearity is generally poor, so look-up-table method is often used for calibration. Dual range refers to the ability to achieve different precision measurements by switching between the high and low positions of GPIO. It is generally used for testing in dark and bright environments, with good linearity and can be calibrated using formula methods. High and low ranges need to be calibrated separately.

The following is an introduction to the calibration methods for analog light sensor.

- **Calibration Equipment**

Illuminometer, uniform surface light source (DNP light box or integrating ball and other equipment that can provide uniform lighting), with adjustable brightness and a brightness adjustment range of 0-5000lux.

- **Calibration Process**

The prototype should be placed close to a uniform surface light source, and the photosensitive sensing surface should be consistent with the lens and aligned with the light source. The illuminometer is placed next to the prototype, parallel to the lens, with the sensing surface aligned with the orientation of the photosensitive and lens, and aligned with the light source. Adjust the brightness of the light source in ascending order. After the adc value stabilizes, record the illuminance value and adc value separately. The recommended brightness level (unit: lux) can be tested as follows: 0, 5, 10, 50, 100, 300, 500, 1000, 3000, 5000. The specific level can be adjusted according to the linearity of the data during the actual measurement process. If the light sensor is a dual range type, it is necessary to switch the range and record the ambient brightness and its corresponding adc values separately.

Based on the recorded brightness levels and corresponding ADC values, the trend of the currently used photosensitive ADC values with environmental brightness can be determined, which can be divided into linear and nonlinear. It is recommended to use formula calibration method for linear trend light sensor, and lookup look-up-table method for nonlinear trend light sensor.

- **Formula Calibration Method (Linear)**

For the light sensor whose adc value shows a linear trend with the change of ambient brightness, the formula calibration method can be used.

There are four calibration parameters of the formula calibration method in adc_calib. For dual range type: adc_calib[0] is the corresponding adc value when the low range ambient brightness is equal to 0 lux. adc_calib[1] is the corresponding adc value when the low range ambient brightness is equal to 100 lux. adc_calib[2] is the corresponding adc value when the high range ambient brightness is equal to 0 lux. adc_calib[3] is the corresponding adc value when the high range ambient brightness is equal to 1000 lux.

For single range type: adc_calib[0] is the corresponding adc value when the ambient brightness is equal to 0 lux. adc_calib[1] is the corresponding adc value when the ambient brightness is equal to 100 lux. adc_calib[2] is the corresponding adc value when the ambient brightness is equal to 0 lux (That is, adc_calib[2] equals adc_calib[0]). adc_calib[3] is the corresponding adc value when the ambient brightness is equal to 1000 lux.

$$\begin{cases} lux1 = a * adc1 + b, \\ lux2 = a * adc2 + b \end{cases}$$

In the algorithm, a and b of the formula are calculated according to the filled calibration parameters.

- **Look-up-table Method (Non-linear)**

For the light sensor whose adc value shows a nonlinear trend with the change of ambient brightness, the look-up-table method can be used.

The calibration parameters of the look-up-table method are two-dimensional arrays, which are divided into two columns. The first column is filled with the ambient brightness gear (unit: lux), and the second column is filled with the adc value corresponding to the ambient brightness. Currently, it supports filling up to 15 lines, that is, 15 ambient brightness gears. Ambient brightness gears are required to be filled in ascending order. It is recommended that the first ambient brightness is 0 lux. The principle of the look-up-table method will not be repeated here. The gear range will be judged according to the input adc value, and the specific ambient brightness value will be calculated according to the interpolation method. The calibration example is as follows:

```
.adc_table_low = {
    {0,1000},
    {6,458},
    {13,446},
    {55,427},
    {102,419},
    {209,410},
    {597,396},
    {1042,388},
    {3017,343},
    {5033,253},
    {7043,254},
},
```

7.2.4 Software Light Sensor Introduction

The main differences between the software light sensitive fast boot process and the hardware light sensitive fast boot process are:

1. The average convergence time is slower than the hardware light sensitive fast boot solution.
2. Since there is no hardware light sensor judgment of ambient illumination, add smartIr algorithm to the application for day-night switching when rk_night_mode=2 (automatic infrared fill light night vision mode).
3. In order to avoid frequent switching of ircut during fast boot, the rk_color_mode is added in the process of software light sensitive fast boot. The rk_color_mode parameter is used to record the actual position of the ircut.

The following is an introduction to the smartIr algorithm parameters, rk_night_mode parameters, and rk_color_mode parameters.

- **SmartIr Parameter**

When rk_night_mode=2, smartIr algorithm is added to the application for automatic day-night switching. Among them, rk_smart_ir_autoled parameter is only valid when the IR fill light supports adjustable brightness (setting PWM duty cycle). SmartIr algorithm can refer to the `release_camera_engine_rkaiq_rv1106_arm-rockchip830-linux-uclibcgnueabi/rkisp_demo/demo/sample/sample_smartIr.cpp` in the SDK, for detailed parameter configuration, please refer to `project/app/fastboot_client/fastboot_client.c`.

```
typedef struct rk_smart_ir_params_s {
    /*
```

```

    * The brightness threshold of day to night, the value should match the
    brightness threshold of RTT stage
    * This value can be calculated and obtained from AecEnvLvCalib in IQ JSON
    */
    float d2n_envL_th;
    /*
    * The brightness threshold of night to day, this value needs debugging
    */
    float n2d_envL_th;
    /*
    * rgain reference value of day to night, the default value is 1
    */
    float rggain_base;
    /*
    * bgain reference value of day to night, the default value is 1
    */
    float bggain_base;
    /*
    * awb filtering radius of day to night, the default value is 0.1
    */
    float awbgain_rad;
    /*
    * awb dispersion threshold of day to night, this value needs debugging
    */
    float awbgain_dis;
    /*
    * smartIr day-night switching sensitivity, the default value is 60
    */
    uint16_t switch_cnts_th;
} rk_smart_ir_params_t;

typedef struct rk_smart_ir_autoled_s {
    /*
    * When the initial state of the fast boot is in night mode, this parameter
    needs to be configured to true to achieve a smooth transition
    */
    bool is_smooth_convert;
    /*
    * Value of automatic infrared fill light in night mode
    * Normalized to 0~100%, the unit is percentage, the precision is 0.1%
    * The value range is (auto_irled_min, auto_irled_max), and the default
    value is 100
    */
    float auto_irled_val;
    /*
    * Minimum value of automatic infrared fill light in night mode
    * Normalized to 0~100%, the unit is percentage, the precision is 0.1%
    * The value range is (0,100), and the default value is 10
    * This value cannot be set too small, otherwise poor linearity will cause
    the screen to flicker
    */
    float auto_irled_min;
    /*
    * Maximum value of automatic infrared fill light in night mode
    * Normalized to 0~100%, the unit is percentage, the precision is 0.1%
    * The value range is (0,100), and the default value is 100
    */
    float auto_irled_max;

```

```
} rk_smart_ir_autoled_t;
```

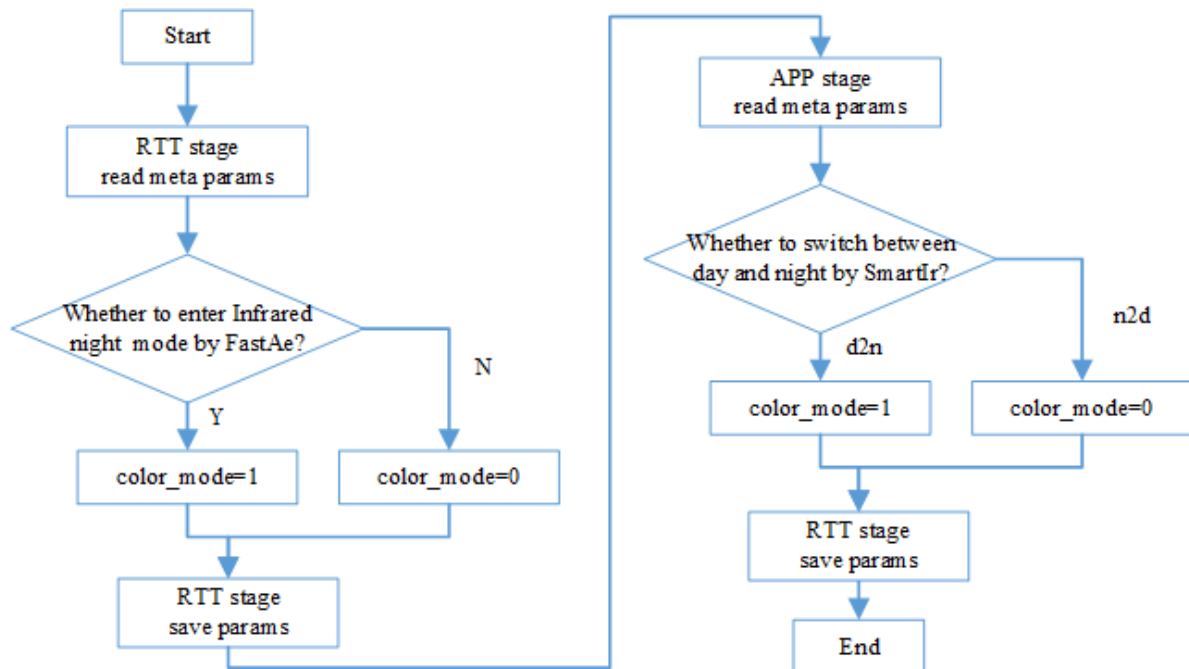
- **rk_night_mode**

This parameter contains five night vision modes, which can be set to a particular night vision mode in the meta partition.

Night mode	Phenomenon
0: no fill light	Always work in daytime mode, infrared cut-off, full color image
1: auto-on white light fill light	When entering the night vision, turn on the white fill light, infrared cut-off, full color image
2: auto-on infrared fill light	When entering the night vision, turn on the infrared fill light, infrared cut-on, gray image. Smart Ir automatic day-night switching
3: always-on white fill light	Always-on white fill light, infrared cut-off, full color image
4: always-on infrared fill light	Always-on infrared fill light, infrared cut-on, full color image

- **rk_color_mode**

This parameter is used to record the current ircut state in the software light sensor fast boot project, so as to avoid the problem of frequent switching of ircut under particular environment. This parameter does not need to be set, but is automatically set by the software light sensor solution. When rk_color_mode=1, it means that the current sensor (or before the last power outage) is working in gray mode, and the full spectral filter corresponding to the ircut starts to work. When rk_color_mode=0, it means that the current sensor (or before the last power outage) is working in full color mode, and the infrared cut-off filter corresponding to the ircut starts to work.



7.3 The meta partition Introduction

The meta partition is only used for RV1103/RV1106 which support fastboot, meta partition is used for saving sensor initialization parameters and image quality files.

meta partition allocation	introduction
32KB	meta data: META_HEAD PARAM_SHARE2KERNEL_OFFSET SENSOR_INIT_OFFSET CMDLINE_OFFSET AE_TABLE_OFFSET APP_PARAM_OFFSET SECONDARY_SENSOR_INIT_OFFSET RESERVE
32KB backup	meta data backup: BACKUP
320KB	Sensor IQ file
320KB	For dual camera, it uses for the secondary Sensor IQ file

Detailed allocation of meta partition	introduction	size
META_HEAD	The mark of meta	1KB
PARAM_SHARE2KERNEL_OFFSET	The share memory for RISCv and Kernel	1KB
SENSOR_INIT_OFFSET	Sensor initialization parameters	1KB
CMDLINE_OFFSET	The parameters of linux kernel CMDLINE	1KB
AE_TABLE_OFFSET	The parameters of AE/AWB	1KB
APP_PARAM_OFFSET	The parameters of application	1KB
SECONDARY_SENSOR_INIT_OFFSET	For dual camera, the secondary sensor initialization parameters	1KB
RESERVE	Reserved	25KB
BACKUP	Backup	32KB
SENSOR_IQ_BIN_OFFSET	Sensor IQ file	320KB
SECONDARY_SENSOR_IQ_BIN_OFFSET	For dual camera, the secondary sensor IQ file	320KB

Notice: the partition of meta is defined by `project/make_meta/include/rk_meta.h`. There may be differences between SDK versions, so you need to pay attention to the version code matching issue.

7.3.1 Meta Introduction

```
struct meta_head {
    uint32_t tag;                // The mark of meta
    uint32_t load;               // Load address of meta, DO NOT
MODIFY
    uint32_t size;               // The partition size of meta,
DO NOT MODIFY
    uint32_t comp_type;          // Camera Sensor IQ file
compress type (it is gzip by default)
    uint32_t comp_size;          // The compress size of camera
sensor IQ file
    uint32_t comp_off;           // The offset of camera sensor
IQ file
    uint32_t iq_item_size;       // The uncompress size of
camera sensor IQ file
    uint32_t total_part_num;     // The num of part
    uint32_t part_flag;          // flag of part
    uint32_t part_reserved_size; // reserved size of part
    uint8_t reserved[META_HEAD_RESERVED_SIZE]; // Reserved
    uint32_t crc32;              // meta partition CRC32
    uint32_t meta_flags;         // meta partition is read by
SPL and meta_flag is set to notify MCU to read
};
```

The meta_head is generated automatically when packaging meta.img without manual modification. While system is booting, SPL will read meta to DDR and notify MCU to read. For detailed instructions, please refer to the codes `arch/arm/mach-rockchip/rk_meta.c`

7.3.2 Sensor Information Parameters Introduction

```
struct gpio_info
{
    /*
     * e.g.
     *   GPIO2_D3 (2*32 + 3*8 + 3 = 91)
     *   GPIO3_A2 (3*32 + 0*8 + 2 = 98)
     */
    int gpio_index;
    /*
     * bit25: FLGA_GPIO_INPUT --> gpio is input
     * bit25: FLGA_GPIO_OUTPUT --> gpio is output
     *
     * bit26: FLAG_GPIO_DEFAULT_LOW --> gpio default level is low
     * bit26: FLAG_GPIO_DEFAULT_HIGH --> gpio default level is high
     *
     * bit0~bit3: gpio's iomux --> define in sensor_define.h eIOMUX_FUNC
     */
    int gpio_func;

    /*
     * PWM channel number
     * e.g. PWM11_IR_M1 --> pwm_channel = 11
     */
};
```

```

int pwm_channel;

/* PWM period, units nanosecond */
int pwm_period;
/* PWM Duty Ratio */
int pwm_pulse;

/*
 * ADC channel number
 * e.g. SARADC_IN1 --> adc_channel = 1
 */
int adc_channel;

/*
 * 0: The larger the ADC value, the darker the environment
 * 1: The larger the ADC value, the brighter the environment
 */
int adc_direction;
};

struct sensor_init_cfg
{
    uint32_t head;
    int32_t len;
    uint32_t crc32;

    /**
     * IRCUT
     */
    struct gpio_info ircut_a;
    struct gpio_info ircut_b;

    /**
     * IR_LED
     */
    struct gpio_info led_ir_enable;
    struct gpio_info led_ir;

    /**
     * White_LED
     */
    struct gpio_info led_white_enable;
    struct gpio_info led_white;

    /* ADC */
    struct gpio_info gpio_adc_enable;
    struct gpio_info gpio_adc;

    /* Sensor resolution */
    int32_t cam_w;
    int32_t cam_h;

    /* ambient light sensor: 0: none; 1: analog; 2: digital */
    int32_t als_type;
    /* Ambient brightness threshold for day-night vision switching. typical:
    FIX2INT16(10.0)*/
    int32_t als_value;
};

```

sensor_init_cfg is used to configure the following information:

- The resolution of camera sensor(cam_w/cam_h)
- Two GPIO of IRCUT(ircut_a/ircut_b)
- Infrared fill light(led_ir)
- Enable GPIO of infrared fill light(led_ir_enable)
- White light fill light(led_white)
- Enable GPIO of white light fill light(led_white_enable)
- Ambient light sensor type(als_type)
- Ambient brightness threshold of the ambient light sensor for day and night vision switching(als_value)

For detailed sensor_init_cfg instructions, please refer to

project/make_meta/sensor_init/rv1106_evb2_gpio.c

7.3.3 Camera Sensor IQ Parameter Introduction

```
#define SENSOR_IQ_INFO_UPPER_ALIGN(size, align_size)    (((size) + align_size -  
1) & ~(align_size - 1))  
#define SENSOR_IQ_INFO_UPPER_ALIGN_16(size)            (SENSOR_IQ_INFO_UPPER_ALIGN(size,  
16))  
#define SENSOR_IQ_INFO_UPPER_ALIGN_64(size)            (SENSOR_IQ_INFO_UPPER_ALIGN(size,  
64))  
  
struct sensor_iq_info  
{  
    unsigned int tag;                                // Identify of sensor_iq_info  
    unsigned int main_sensor_iq_offset;               // The offset of main camera  
sensor IQ  
    unsigned int main_sensor_iq_size;                 // The size of main camera sensor  
IQ  
    unsigned int secondary_sensor_iq_offset;          // The offset of secondary camera  
sensor IQ  
    unsigned int secondary_sensor_iq_size;            // The size of secondary camera  
sensor IQ  
    int len;  
    /*  
     * Ensure data's offset align with 16 byte  
     */  
    * 4 = sizeof(unsigned int), sizeof(int)  
    * 7 = count of (tag + main_sensor_iq_offset + main_sensor_iq_size  
    *      + secondary_sensor_iq_offset + secondary_sensor_iq_size + len +  
crc32)  
    */  
    uint8_t reserved[SENSOR_IQ_INFO_UPPER_ALIGN_16(7*4)-7*4];  
    unsigned int crc32;  
    uint8_t data[0]; /* Just a way for get data */  
};
```

7.3.4 AE/AWB Parameter Introduction

```
struct ae_init_cfg {
    uint32_t black_lvl;    // Black level value(12bit)
    uint32_t adc_calib_type; //0: Formula 1: LUT(look up table)
    uint32_t adc_range_type; //0: dual 1: single

    /*
     * adc_calib[0]=lowadc_lux0
     * adc_calib[1]=lowadc_lux100
     * adc_calib[2]=highadc_lux0
     * adc_calib[3]=highadc_lux1000, 10bit
     */
    uint32_t adc_calib[ADC_CALIB_NO];

    uint32_t adc_table_len;
    uint32_t adc_table_low[ADC_CALIB_TBL_LEN][2];
    uint32_t adc_table_high[ADC_CALIB_TBL_LEN][2];
    uint32_t start_exp[AE_START_EXP_NO]; //typical: start_exp[0]=1lux,
    start_exp[1]=10lux, start_exp[2]=100lux
};

struct awb_init_cfg {
    uint32_t rg_gain_base; //used for soft light sensor, need float =>
    FIX2INT16
    uint32_t bg_gain_base;
    uint32_t awb_gain_rad;
    uint32_t awb_gain_dis;
};

struct rtt_2a_res {
    uint32_t day_or_night; // 0: invalid 1:day 2:night
    uint32_t awb_gain; //0~-15bit: rgain 16~31bit: bgain
    uint32_t exp;
    uint32_t isp_dgain;
    uint32_t reg_time;
    uint32_t reg_gain;
    uint32_t dcg_mode;
    uint32_t ae_stats[AE_GRID_STATS]; // only G channel (16bit)
    uint32_t ae_diff_th; //unit:%
    uint32_t nr_diff_th; //unit:%
    uint32_t last_lodif_0;
    uint32_t last_lodif_1;
    uint32_t last_hidif_0;
    uint32_t last_hidif_1;
};

struct ae_awb_init_cfg {
    uint32_t head;
    int32_t len;
    uint32_t crc32;
    struct ae_init_cfg ae;
    struct awb_init_cfg awb;
    struct rtt_2a_res rtt_res;
};
```

For detailed ae_awb_init_cfg instructions, please refer to

```
project/make_meta/sensor_ae_awb_table/sc230ai_2a.c
```

7.3.5 Application Parameters Introduction

```
struct app_param_info
{
    uint32_t head; // app_param_info's mark
    int32_t len;    // app_param_info's size

    /*
     * Set up horizontal and vertical mirroring of the main camera
     * 0: NONE
     * 1: Mirror Horizontally
     * 2: Mirror Vertically
     * 3: Mirror Horizontally and Vertically
     */
    int32_t cam_mirror_flip;

    int32_t cam_fps; // Set the camera frame rate
    int32_t cam2_fps;

    /*
     * Set the fill light
     * 0: no fill light
     * 1: auto-on white light fill light
     * 2: auto-on infrared fill light
     * 3: always-on white fill light
     * 4: always-on infrared fill light
     */
    int32_t night_mode;

    int32_t led_value; // Set the brightness of fill light
    int32_t venc_w;    // Set the main camera's width of encoding resolution
    int32_t venc_h;    // Set the main camera's height of encoding
    resolution
    int32_t venc_type; // Set the main camera's encoding type
    RK_VENC_TYPE_H264/RK_VENC_TYPE_H265
    int32_t venc_bitrate; // Set the main camera's encodeing bitrate (it is
    venc_w x venc_h by default)

    /*
     * Set whether the HDR function of the camera is enabled
     * 0: HDR is disabled
     * 5: enable 2 frame HDR
     */
    int32_t cam_hdr;
    int32_t cam2_hdr;

    /*
     * Set whether the camera sensor work in gray mode or full color mode
     * 1: gray mode
     * 0: full color mode
     * The Physical address rk_color_mode of color_mode will be recorded in the
    kernel cmdline
```

```

    * For details, please refer to
project/app/fastboot_client/fastboot_client.c
    */
    int32_t color_mode;

    /* The index of the sensor which boot firstly */
    int32_t cam_index;

    /* The max frames of the sensor fast AE (Automatic Exposure) while MCU run
*/
    int32_t fastae_max_frame;

    int32_t rtt_stream_on_threshold;
    int32_t rtt_stream_buf_cnt;

    /*
    * The define of dual camera or singular camera
    * 1: singular camera
    * 2: dual camera
    */
    int32_t cam_num;

    int32_t venc2_w;          // Set the secondary camera's width of encoding
resolution
    int32_t venc2_h;          // Set the secondary camera's height of encoding
resolution
    int32_t venc2_type;       // Set the secondary camera's encoding type
RK_VENC_TYPE_H264/RK_VENC_TYPE_H265
    int32_t venc2_bitrate; // Set the secondary camera's encodeing bitrate (it
is venc_w x venc_h by default)

    /*
    * Set up horizontal and vertical mirroring of the secondary camera
    * 0: NONE
    * 1: Mirror Horizontally
    * 2: Mirror Vertically
    * 3: Mirror Horizontally and Vertically
    */
    int32_t cam2_mirror_flip;

    int32_t cam1_max_fps;
    int32_t cam2_max_fps;

    uint32_t cam_skip_frame_interval;

    int32_t fastae_tolerance_coef;

    uint32_t pr_exp_gain;
    uint32_t pr_exp_time;

    uint32_t crc32; //The CRC32 of app_param_info
};

```

7.3.6 Kernel CMDLINE Parameters Introduction

```
struct cmdline_info {  
    uint32_t tag; // The mark of cmdline_info  
    uint8_t data[MAX_CMDLINE_LENGTH]; // The parameter of kernel cmdline  
    uint32_t crc32; // The CRC32 of cmdline_info  
};
```

The variables currently passed to the kernel cmdline are as follows:

```
meta_load_addr=0x00800000  
rk_app_param_offset=0x1400  
meta_part_size=0x00060000  
rk_cam_mirror_flip=0  
rk_cam2_mirror_flip=0  
rk_cam_fps=15  
rk_cam2_fps=15  
rk_cam1_max_fps=60  
rk_cam2_max_fps=60  
rk_cam_skip=0  
rk_cam_hdr=0  
rk_cam2_hdr=0  
rk_cam_index=0  
rk_night_mode=2  
rk_led_value=20  
rk_cam_num=1  
rk_color_mode=0x801434  
rk_iqbin_addr=0x00810040  
rk_iqbin_size=0x0004b7d8  
rk_cam_h=1080  
rk_cam_w=1920
```

The rk_color_mode value of this address record is stand for the camera sensor work mode. (1 is stand for gray mode and 0 is stand for full color mode)

For detailed instructions, please refer to fastboot_server.c

The value of rk_iqbin_addr/rk_iqbin_size is stand for the camera sensor's IQ file address and size.

For detailed instructions, please refer to fastboot_server.c

7.3.7 Generate meta.img

The tool which is used to generate meta.img is located in the directory of project/make_meta.

The building steps are as follows:

- Add the meta partition in BoardConfig.mk (the max size of meta partition is 384KB)
- Building

```
./build.sh meta
```

7.3.8 Update meta Data on Board

7.3.8.1 Dump meta Partition

```
make_meta -d /dev/block/by-name/meta
```

Notice: `/dev/mtdblock4` is the device of meta partition

7.3.8.2 Update meta Partition Data

To update meta partition, we can use the command of `make_meta` or refer to the code in:

```
project/make_meta/make_meta/sample_read_write_meta.c
```

This is an example for update meta with the command of `make_meta` (`/dev/block/by-name/meta` is the device of meta partition):

7.3.8.3 Update the cmdline of meta. For example, enable kernel boot log

```
make_meta --update --meta_path=/dev/block/by-name/meta --cmdline="loglevel=8"
```

7.3.8.4 Update the cmdline of meta. Enable the camera debug AIQ log

```
# log on /tmp/rkaiq0.log /tmp/rkaiq1.log
make_meta --update --meta_path /dev/block/by-name/meta --cmdline
"persist_camera_engine_log=0x3ff4"

# enable smart IR log, /tmp/rkaiq1.log
make_meta --update --meta_path /dev/block/by-name/meta --cmdline
"persist_camera_engine_log=0x80000000"
```

7.3.8.5 Update sensor initialization parameters

- Building will select the sensor based on IQ file which defined in BoardConfig.mk
- Sensor initialization bin file: `project/make_meta/sensor_init/init_sensor.bin`
- Copy `init_sensor.bin` to the board (e.g. `/userdata`)

```
make_meta --update --meta_path=/dev/mtdblock4 --
sensor_init="/userdata/init_sensor.bin"
```

Notice: For detailed instructions, please refer to the document `/project/make_meta/readme.txt`

7.3.8.6 Update Sensor IQ file

For example, copy IQ files to the board (e.g. the dir of /userdata)

```
# for single camera
make_meta --update --meta_path=/dev/block/by-name/meta --sensor_iq_bin
/userdata/sc3338_FK01_30IRC-F16.bin

# for dual camera, update the main and secondary cameras's IQ files at the same
time
make_meta --update --meta_path=/dev/block/by-name/meta --sensor_iq_bin
/userdata/sc3338_FK01_30IRC-F16.bin --secondary_sensor_iq_bin
/userdata/sc4336_OT01_40IRC_F16.bin

# for dual camera, update the main camera's IQ file only
make_meta --update --meta_path=/dev/block/by-name/meta --sensor_iq_bin
/userdata/sc3338_FK01_30IRC-F16.bin

# for dual camera, update the secondary camera's IQ file only
make_meta --update --meta_path=/dev/block/by-name/meta --
secondary_sensor_iq_bin /userdata/sc4336_OT01_40IRC_F16.bin
```

7.3.8.7 Other parameters introduction

Parameter	Description
--create	Create meta.img
--update	Update the image which input from --meta_path
-d	Dump meta partition data
-r	Recover data from backup (IQ files is not supported)
--cmdline	Add parameters into kernel CMDLINE
--meta_path	The path of meta partition
--sensor_init	The path of init_sensor.bin
--ae_awb_tab	The path of ae_awb_table.bin
--sensor_iq_bin	The path of sensor IQ file
--meta_load_addr	The load address of meta (DO NOT MODIFY)
--meta_part_size	The size of meta partition (max size 384KB)
--rk_cam_w	The width of sensor resolution
--rk_cam_h	The height of sensor resolution
--rk_cam2_w	The width of secondary sensor resolution
--rk_cam2_h	The height of secondary sensor resolution
--rk_cam_mirror_flip	The mirror and flip of main sensor 0: NONE 1: Mirror 2: Flip 3: Mirror and Flip
--rk_cam2_mirror_flip	The mirror and flip of secondary sensor 0: NONE 1: Mirror 2: Flip 3: Mirror and Flip
--rk_cam_fps	Sensor frame rate
--rk_led_value	The luminance of led<size: 0~100>
--rk_night_mode	The selection of led 0: NONE 1: white led auto 2: Infrared led auto 3: white led always 4: Infrared led always
--rk_venc_w	The width of venc
--rk_venc_h	The height of venc

Parameter	Description
--rk_venc_type	Type of venc coding <1: h264, 2: h265>
--rk_venc_bitrate --	Size of venc bit rate value KB (is rk_venc_w x rk_venc_h by default)
--rk_venc2_w	The width of venc2
--rk_venc2_h	The height of venc2
--rk_venc2_type	Type of venc2 coding <1: h264, 2: h265>
--rk_venc2_bitrate --	Size of venc2 bit rate value KB (is rk_venc_w x rk_venc_h by default)
--rk_cam_hdr --	HDR mode of camera sensor 0: no HDR(default) 5: 2 frame HDR
--rk_cam_index	The index of the sensor which boot firstly
--rk_color_mode	Status of full color or gray <0: full color, 1: gray>
--rk_fastae_max_frame	The max frames of the sensor fast AE (Automatic Exposure) while MCU run

7.4 Modify System Memory Allocation

During product development, the kernel and file system may become larger than the size of the SDK's default configuration. At this time, you have to adjust the memory allocation.

The default size of the SDK after ramdisk is 10MB, so we will take modifying the ramdisk size as an example.

7.4.1 Modify the Kernel DTS Memory Allocation

The SDK's default memory allocation is located at **sysdrv/source/kernel/arch/arm/boot/dts/rv1106-thunder-boot.dtsi**

Notice: the offset and size of the node rtos and meta can not be modified, and the offset of rkisp_thunderboot node also can not be modified, but it's size can modify according to image resolution.

```
reserved-memory {
    #address-cells = <1>;
    #size-cells = <1>;
    ranges;

    rtos@40000 {
        reg = <0x40000 0x40000>;
    };

    meta: meta@800000 {
        /* reg's offset MUST match with RTOS */
        reg = <0x00800000 0x60000>;
    };
};
```

```
rkisp_thunderboot: rkisp@860000 {
    /* reg's offset MUST match with RTOS */
    /*
     * vicap, capture raw10, ceil(w*10/8/256)*256*h *4(buf num)
     * e.g. 1920x1080: 0xa8c000
     */
    reg = <0x00860000 0xa8c000>;
};

ramdisk_r: ramdisk_r {
    reg = <0x12ec000 (10 * 0x00100000)>;
};

ramdisk_c: ramdisk_c {
    reg = <0x1cec000 (5 * 0x00100000)>;
};
};
```

Add codes at the end of dts file as follows(e.g. /sysdrv/source/kernel/arch/arm/boot/dts/rv1106b-evb2-v10.dts):

```
/* ramdisk_r is the max size of output/out/fastboot/rootfs_romfs.img */
&ramdisk_r {
    reg = <0x12ec000 (15 * 0x00100000)>;
};

/* ramdisk_c is the max size of output/out/fastboot/rootfs_romfs.img.gz */
&ramdisk_c {
    // 0x21ec000 = 0x12ec000 + (15 * 0x00100000)
    reg = <0x21ec000 (5 * 0x00100000)>;
};
```

7.4.2 Modify FIT Script

Modify these files: /project/scripts/rv1103b-boot-tb-ramdisk.its

- the load value of ramdisk node is ramdisk_r
- the comp value of ramdisk node is ramdisk_c
- the comp value of kernel node is kernel_c, it will be auto calculated by ramdisk_c when building

```

/* ramdisk_r is the max size of output/out/fastboot/rootfs_romfs.img */
&ramdisk_r {
    reg = <0x12ec000 (15 * 0x00100000)>;
};

/* ramdisk_c is the max size of output/out/fastboot/rootfs_romfs.img.gz */
&ramdisk_c {
    // 0x21ec000 = 0x12ec000 + (15 * 0x00100000)
    reg = <0x21ec000 (5 * 0x00100000)>;
};

```

```

ramdisk {
    data = /incbin/("./images-tb/ramdisk");
    compression = "gzip";
    type = "ramdisk";
    arch = "arm";
    os = "linux";
    preload = <1>;
    comp = <0x21ec000>;
    load = <0x12ec000>;
    decomp-async;
    hash {
        algo = "sha256";
        uboot-ignore = <1>;
    };
};

```

7.4.3 Rebuild

```

./build.sh kernel
./build.sh firmware

```

8. IPC Linux SDK Q&A

8.1 How to Modify Partition Table, Add Customized Partition and Partition Read/Write

8.1.1 Storage Medium and File System Type

Storage Medium	Supported Readable and Writable File System Formats	Supported Read-Only File System Formats
eMMC	ext4	squashfs
spi nand or slc nand	ubifs	squashfs
spi nor	jffs2	squashfs

File System Format	Script to Create Burning Image Files
ext4	output/out/sysdrv_out/pc/mkfs_ext4.sh
jffs2	output/out/sysdrv_out/pc/mkfs_jffs2.sh
ubifs	output/out/sysdrv_out/pc/mkfs_ubi.sh
squashfs	output/out/sysdrv_out/pc/mkfs_squashfs.sh

Note: Nand Flash hardware has different page size and block size, and requires to burn the corresponding image file, so mkfs_ubi.sh will package the burning image with different page size and block size by default. For detailed Nand Flash instructions, please refer to the document [Rockchip_Developer_Guide_Linux_Nand_Flash_Open_Source_Solution_CN.pdf](#)

8.1.2 Partition Table Introduction

The SDK uses env partition to set partition table. Partition table information is configured by the parameter `RK_PARTITION_CMD_IN_ENV` in `<SDK>/project/cfg/BoardConfig_IPC/BoardConfig*.mk`.

The partition table is stored as a string in the configuration. Here is each medium partition table.

Medium	Partition Table
eMMC	<code>RK_PARTITION_CMD_IN_ENV="32K(env),512K@32K(idblock),4M(uboot),32M(boot),2G(rootfs),-(userdata)"</code>
spi nand 或 slc nand	<code>RK_PARTITION_CMD_IN_ENV="256K(env),256K@256K(idblock),1M(uboot),8M(boot),32M(rootfs),-(userdata)"</code>
spi nor	<code>RK_PARTITION_CMD_IN_ENV="64K(env),128K@64K(idblock),128K(uboot),3M(boot),6M(rootfs),-(userdata)"</code>

The format for each partition is `<size>[@<offset>] (part-name)`. The partition size and partition name are required, and the offset depends (see the Note 3 below).

There are the following points to note when configuring a partition table:

1. Partitions are separated by characters ",".
2. The unit of the partition size is K, M, G, T, P, and E, case insensitive. If there is no unit, the default unit is byte. "-" indicates that the partition size is the remaining capacity.
3. If the first partition starts at address 0x0, no offset is added. Otherwise, an offset must be added. The offsets of subsequent partitions can be added or not.
4. **idblock** partition offset is fixed. **non-modifiable**

5. It is not advised to change the env partition name. (If you want to modify env's partition offset and size, The defconfig CONFIG_ENV_OFFSET and CONFIG_ENV_SIZE of U-Boot must be modified, re-build firmware, erase board's flash, and update firmware finally)

8.1.3 Add Customized Partition

The following is an example of eMMC adding a readable and writable partition custom_part with a size of 64MB.

- Modify partition table parameters in the boardconfig using the preceding method [Partition Table Introduction](#). For example: 64M(custom_part)
- Add a partition naming "custom_part" in `RK_PARTITION_CMD_IN_ENV` and modify the `RK_PARTITION_FS_TYPE_CFG` partition mount configuration in

```
<SDK>/project/cfg/BoardConfig*.mk
```

```
# config partition's filesystem type (squashfs is readonly)
# emmc:      squashfs/ext4
# nand:      squashfs/ubifs
# spi nor:   squashfs/jffs2
# RK_PARTITION_FS_TYPE_CFG format:
#      AAAA@/BBBB/CCCC@DDDD
#      AAAA -----> partition name
#      /BBBB/CCCC ----> partition mount point
#      DDDD -----> partition filesystem type
(squashfs/ext4/ubifs/jffs2)
export
RK_PARTITION_FS_TYPE_CFG=rootfs@IGNORE@ext4,custom_part@/opt/custom_part@ext4

# config partition in environment
# RK_PARTITION_CMD_IN_ENV format:
#      <partdef>[,<partdef>]
#      <partdef> := <size>[@<offset>] (part-name)
#
export
RK_PARTITION_CMD_IN_ENV="32K(env),512K@32K(idblock),4M(uboot),32M(boot),2G(rootfs),64M(custom_part),-(userdata)"
```

- Make custom_part partition image

```
mkdir -p custom_part
./output/out/sysdrv_out/pc/mkfs_ext4.sh custom_part custom_part.img 64*0x100000
# Note: When using the default mount script of the SDK, the partition image
file name should be named with the partition name
# For example: the partition name is custom_part, and the partition image name
is custom_part.img
```

- Create a mount directory for the custom_part partition in the filesystem

```
mkdir -p output/out/rootfs_uclibc_rv1103b/opt/custom_part
```

- Repackage root filesystem (rootfs.img) `./build.sh firmware`
- Burn rootfs.img env.img and custom_part.img

8.1.4 OEM Partition Mounting Introduction

To mount OEM partition, the following configurations need to be modified:

- Add OEM partition to partition table (see [Partition Table Introduction](#) for details), for example:

```
RK_PARTITION_CMD_IN_ENV="32K(env),512K@32K(idblock),4M(uboot),32M(boot),2G(rootfs),64M(oem),-(userdata)"
```

- Add OEM configurations to file system type configuration (see [Add Customized Partition](#) for details), for example:

```
RK_PARTITION_FS_TYPE_CFG=rootfs@IGNORE@ext4,oem@/oem@ext4
```

- Enable the following configuration:

```
# Enable app installation in OEM partition
export RK_BUILD_APP_TO_OEM_PARTITION=y
```

8.2 How to Use OTA

The IPC SDK support two OTA scheme, [Recovery](#) and [A/B System](#).

8.2.1 How to Use Recovery

There is one more recovery partition on the device in recovery mode, which consists of kernel+resource+ramdisk and is mainly used for upgrade operations. u-boot will determine whether the system to be booted is the Normal system or the recovery system according to the fields stored in the misc partition. Due to the independence of the system, the recovery mode can ensure the integrity of the upgrade, that is, if the upgrade process is interrupted, such as an abnormal power failure, the upgrade can still continue.

This chapter mainly introduces the process and technical details of running the Recovery local upgrade program through the userdata partition and SD card. The following is the way to use recovery.

- Add the following configuration to `<SDK>/project/cfg/BoardConfig*.mk`:

```
export RK_RECOVERY_KERNEL_DEFCONFIG_FRAGMENT="rv1103b-recovery.config"

#misc image
export RK_MISC=recovery-misc.img

# enable build recovery
export RK_ENABLE_RECOVERY=y

# select image to update
# export RK_OTA_RESOURCE="uboot.img boot.img rootfs.img userdata.img"
```

Note: If RK_OTA_RESOURCE is not enabled, uboot.img, boot.img and rootfs.img are packaged by default.

- Modify the partition table

Add two partitions: `misc` and `recovery` to the partition table `RK_PARTITION_CMD_IN_ENV` which located on `BoardConfig.mk`. The size and order of the partitions can be adjusted within a reasonable range based on actual requirements.

Storage Medium	the partition size of misc
eMMC	64KB
spi nand	512KB (recommendation)
spi nor	64KB

- Build recovery

```
./build.sh recovery
```

- Build the firmware that required to be upgraded

Manually build the firmware included in `RK_OTA_RESOURCE`. If it is not enabled, build the default partition firmware (please refer to the previous section for details of firmware compilation).

- Package OTA upgrade package

```
./build.sh ota
```

- Copy the OTA upgrade package

Copy the generated OTA upgrade package (`<SDK>/output/image/update_ota.tar`) to the root directory of the SD card.

Note: This step is not required if the program downloads the OTA upgrade package to the userdata partition in the device.

- Insert the SD card to the device

Note: This step is not required if the program downloads the OTA upgrade package to the userdata partition in the device.

- Enter the Recovery system on the device side to start the upgrade

```
reboot recovery
# Note: This command requires using busybox in the SDK or the corresponding patch
```

Enter the above code on the device side (board side) to enter the recovery system to start the upgrade.

Note: After the upgrade is complete, the device will restart and enter the normal system. If it fails, the device will stay in the recovery system and print log. If no SD card or upgrade package is found, the system will also restart to normal system.

8.3 How to Use TFTP Upgrade in U-Boot Terminal

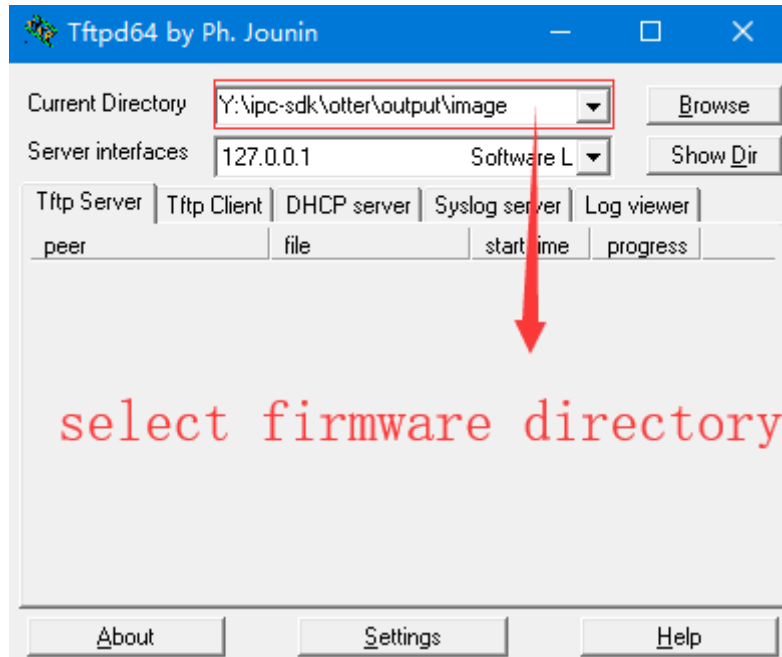
The TFTP upgrade file is compiled in the `<SDK>/output/image/` directory along with the firmware. The file name is `tftp_update.txt`. The usage method is as follows:

- Configure the TFTP Server

Tftpd64 download address <https://pjo2.github.io/tftpd64>

NOTICE:

1. Customers should follow relevant open source agreements when using Tftpd64
2. Customers shall take all legal risks and consequences from using Tftpd64



- Save the upgrade file `tftp_update.txt` and all firmware files with the file name extension `.img` to the specified directory on the server

(Note: SLC Nand does not support upgrading `idblock` partition by downloading firmware.)

- Set the IP address on the U-Boot terminal

```
=> setenv ipaddr 192.168.1.111
=> setenv serverip 192.168.1.100
=> saveenv
Saving Environment to envf...
=>
```

The preceding IP addresses are for reference only. Set them based on actual conditions and ensure that the client and server reside on the same network segment.

- Run the `tftp_update` command on the U-Boot terminal

```
=> tftp_update
ethernet@fffc40000 Waiting for PHY auto negotiation to complete. done
Using ethernet@fffc40000 device
TFTP from server 192.168.1.100; our IP address is 192.168.1.111
Filename 'tftp_update.txt'.
Load address: 0x3be24c00
Loading: *•#
      203.1 KiB/s
done
Bytes transferred = 1250 (4e2 hex)
...
```

8.4 How to Use SDcard Upgrade in U-Boot Terminal

The SD card upgrade file is compiled in the `<SDK>/output/image/` directory along with the firmware. The file name is `sd_update.txt`. The usage method is as follows:

- Save the upgrade file `sd_update.txt` and all firmware files with the file name extension `.img` to the specified directory in the root directory of the SD card

(Note: 1. SLC Nand does not support upgrading `idblock` partition by downloading firmware. 2. The SD card only supports FAT file systems.)

- Insert the SD card to the device
- Run the `sd_update` command on the U-Boot terminal

```
=> sd_update
PartType: ENV
reading sd_update.txt
1511 bytes read in 2 ms (737.3 KiB/s)
...
```

- After the upgrade, restart the device

8.5 How to Use a Key to Trigger SDcard Upgrade during U-Boot Phase

ADC is required for this function. After the corresponding defconfig are configured in the U-Boot, the `sd_update` is triggered during the U-Boot phase by holding down the key and restarting the device. The configuration process is as follows:

- According to the hardware circuit diagram, the voltage value of the key is obtained for ADC configuration

Taking the key2 of RV1103B EVB1 V10 as an example, the adc value of the selected key is 230 according to the circuit diagram.

8.6 Cannot Detect Device When SD Card Inserted

The defconfig corresponding to the kernel requires to enable the configuration that supports SD card.

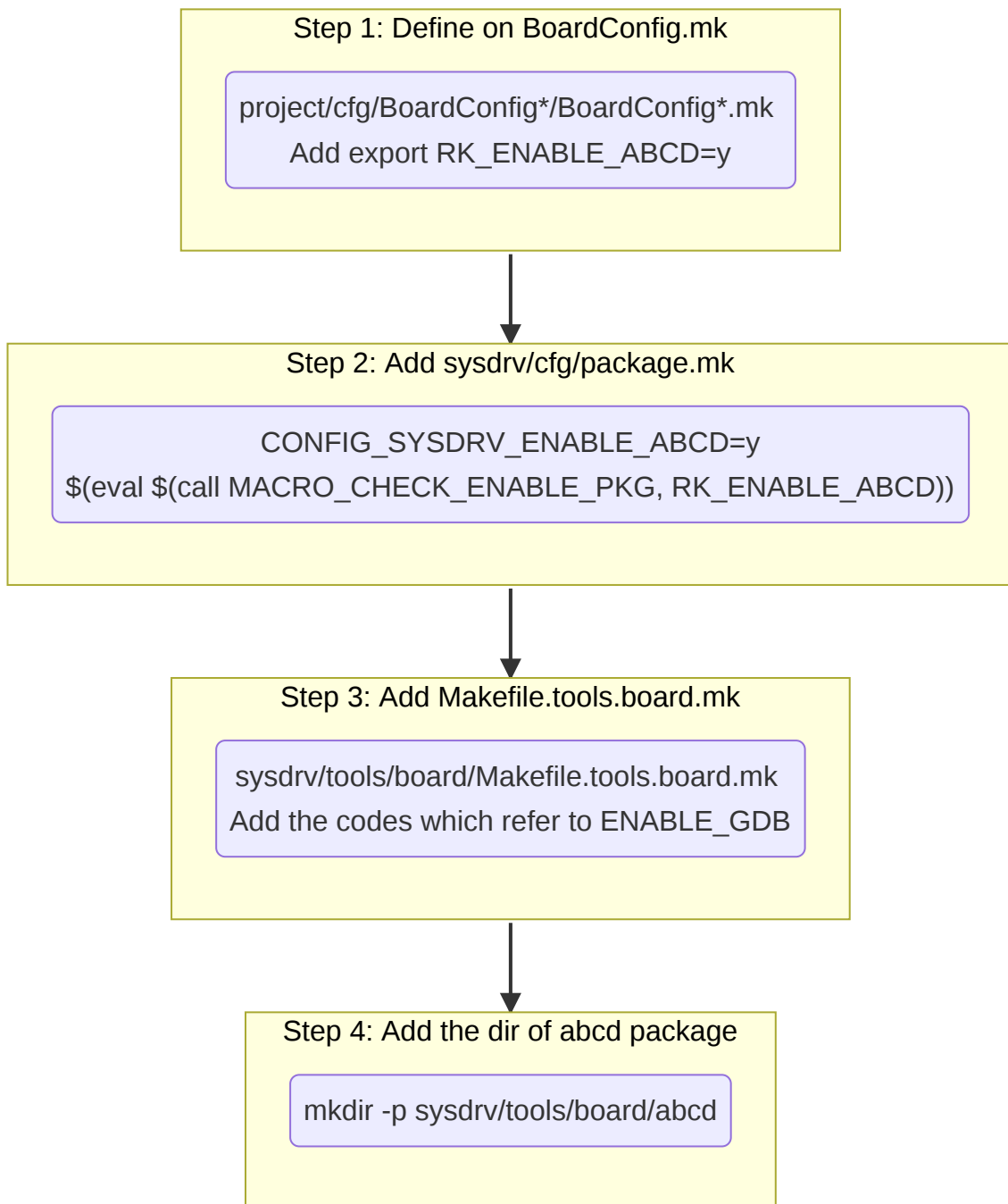
```
CONFIG_MMC_DW=y  
CONFIG_MMC_DW_PLTFM=y  
CONFIG_MMC_DW_ROCKCHIP=y
```

Add vfat filesystem support.

```
CONFIG_VFAT_FS=y  
CONFIG_MSDOS_PARTITION=y  
CONFIG_EFI_PARTITION=y
```

8.7 How to Add Third-Party Libraries to the sysdrv Directory

- sysdrv/tools/board is the program running on the board, now is going to introduce how to add a third-party program (for example, the third-party program is named ABCD).



- touch sysdrv/tools/board/abcd/Makefile

```
# sysdrv/tools/board/abcd/Makefile reference code
ifeq ($(SYSDRV_PARAM), )
SYSDRV_PARAM:=../../../../../Makefile.param
include $(SYSDRV_PARAM)
endif

export LC_ALL=C
SHELL:=/bin/bash

CURRENT_DIR := $(shell pwd)
PKG_TARBALL := abcd.tar.xz
PKG_NAME := abcd
PKG_BIN := out

all:
rm -rf $(CURRENT_DIR)/$(PKG_NAME); \
```

```

tar -xf $(PKG_TARBALL); \
mkdir -p $(CURRENT_DIR)/$(PKG_NAME)/$(PKG_BIN); \
mkdir -p $(CURRENT_DIR)/$(PKG_BIN); \
pushd $(CURRENT_DIR)/$(PKG_NAME)/; \
    ./configure --host=$(SYSDRV_CROSS) \
    --target=$(SYSDRV_CROSS) CFLAGS="$(SYSDRV_CROSS_CFLAGS)" \
    LDFLAGS="$(SYSDRV_CROSS_CFLAGS)" \
    --prefix=$(CURRENT_DIR)/$(PKG_NAME)/$(PKG_BIN); \
    make -j$(SYSDRV_JOBS) > /dev/null || exit -1; \
    make install > /dev/null; \
popd; )
$(call MAROC_COPY_PKG_TO_SYSDRV_OUTPUT, $(SYSDRV_DIR_OUT_ROOTFS),
$(PKG_BIN))

clean: distclean

distclean:
    -rm -rf $(PKG_NAME) $(PKG_BIN)

```

- Test and build

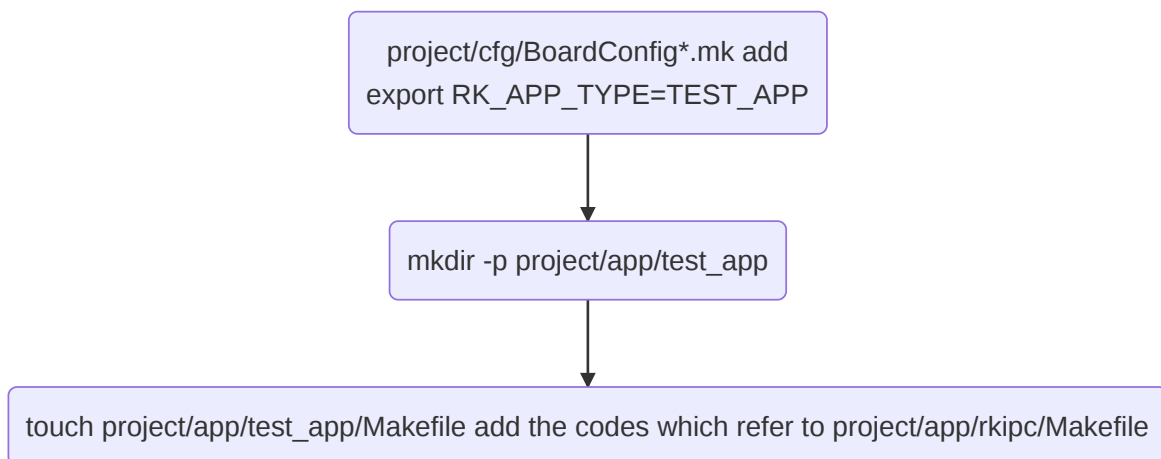
```

cd sysdrv/tools/board/abcd/
make
cd ../../..
make

```

8.8 How to Add New Applications in Project/app

- Take adding application test_app as an example



```

# project/app/test_app/Makefile reference code

ifeq ($(APP_PARAM), )
APP_PARAM:=../Makefile.param
include $(APP_PARAM)
endif

export IC_ALL=C

```

```

SHELL:=/bin/bash

CURRENT_DIR := $(shell pwd)

PKG_NAME := test_app
PKG_BIN ?= out
PKG_BUILD ?= build

RK_APP_CFLAGS = -I $(RK_APP_MEDIA_INCLUDE_PATH)

RK_APP_LDFLAGS = -L $(RK_APP_MEDIA_LIBS_PATH)

RK_APP_OPTS += -Wl,-rpath-
link,$(RK_APP_MEDIA_LIBS_PATH):$(RK_APP_PATH_LIB_INCLUDE)/root/usr/lib
PKG_CONF_OPTS += -DCMAKE_C_FLAGS="$(RK_APP_CFLAGS) $(RK_APP_LDFLAGS)
$(RK_APP_OPTS)" \
    -DCMAKE_CXX_FLAGS="$(RK_APP_CFLAGS) $(RK_APP_LDFLAGS)
$(RK_APP_OPTS)"

# define project/cfg/BoardConfig*.mk
ifneq ($(findstring $(RK_APP_TYPE),TEST_APP),)
PKG_TARGET := test_app_build
endif

ifeq ($(PKG_BIN),)
$(error ### $(CURRENT_DIR): PKG_BIN is NULL, Please Check !!!)
endif

all: $(PKG_TARGET)
    @echo "build $(PKG_NAME) done"

test_app_build:
    rm -rf $(PKG_BIN) $(PKG_BUILD); \
    mkdir -p $(PKG_BIN); \
    mkdir -p $(PKG_BUILD); \
    pushd $(PKG_BUILD)/; \
        rm -rf CMakeCache.txt; \
        cmake $(CURRENT_DIR)/$(PKG_NAME) / \
            -DCMAKE_C_COMPILER=$(RK_APP_CROSS)-gcc \
            -DCMAKE_CXX_COMPILER=$(RK_APP_CROSS)-g++ \
            -DCMAKE_INSTALL_PREFIX="$(CURRENT_DIR)/$(PKG_BIN)" \
            $(PKG_CONF_OPTS) ; \
        make -j$(RK_APP_JOBS) || exit -1; \
        make install; \
    popd;
    $(call MAROC_COPY_PKG_TO_APP_OUTPUT, $(RK_APP_OUTPUT), $(PKG_BIN))

clean:
    @rm -rf $(PKG_BIN) $(PKG_BUILD)

distclean: clean

```

- Test and build

```

./build.sh clean app
./build.sh app

```

8.9 How to Download the NPU Model Transformation Tool and Runtime Library

There are no NPU model transformation tool or NPU runtime library in the SDK, and it can be download from github.

The NPU model transformation tool download address:

```
https://github.com/rockchip-linux/rknn-toolkit2
```

NPU runtime library download address:

```
https://github.com/rockchip-linux/rknpu2
```

8.10 How to Change the CMA Size on Board

- Check the CMA size on board

```
# grep -i cma /proc/meminfo
CmaTotal:      24576 kB
CmaFree:       0 kB
```

- Enter the U-Boot terminal

After the device is restarted, press `Ctrl+C` always. Until `=> <INTERRUPT>` is displayed, it is indicating that the U-boot terminal is entered.

- Check the CMA size in the U-Boot environment variables

```
=> printenv
...
sys_bootargs=root=/dev/mtdblock4 rk_dma_heap_cma=24M rootfstype=squashfs
...
```

`rk_dma_heap_cma` is the CMA size.

NOTICE: This environment variable is named `sys_bootargs`, and `rk_dma_heap_cma` is only one of its parameters. When modifying this environment variable, all contents after `sys_bootargs` should be considered as one.

- Modify environment variables, save environment variables, and restart the device

Using the above environment variable as an example, change the CMA from 24M to 32M.

```
# setenv <name> <vars>
# saveenv
# reset
=> setenv sys_bootargs root=/dev/mtdblock4 rk_dma_heap_cma=32M
rootfstype=squashfs
=> saveenv
Saving Environment to envf...
=> reset
```

- Check the CMA size on board again

```
# grep -i cma /proc/meminfo
CmaTotal:          32768 kB
CmaFree:           0 kB
```

- Check the usage of the CMA size on board

```
# cat /proc/rk_dma_heap/alloc_bitmap
Total: 67584 KiB
Used: 56136 KiB

FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF
FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF
...
FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF
FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF 0003FFFF 00000000
00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
```

Notice: 1 bit stand for 4KiB

- Check the usage of the CMA size for details on board

```
# cat /proc/rk_dma_heap/dma_heap_info |grep Alloc
Alloc by (vmpi          ) [0x074c2000-0x074d1fff] 0x00010000 (64 KiB)
Alloc by (vmpi          ) [0x074b2000-0x074c1fff] 0x00010000 (64 KiB)
Alloc by (vmpi          ) [0x074b0000-0x074b1fff] 0x00002000 (8 KiB)
Alloc by (vmpi          ) [0x074ae000-0x074affff] 0x00002000 (8 KiB)
Alloc by (vmpi          ) [0x07391000-0x074adfff] 0x0011d000 (1140
KiB)
Alloc by (vmpi          ) [0x07390000-0x07390fff] 0x00001000 (4 KiB)
Alloc by (vmpi          ) [0x07310000-0x0738ffff] 0x00080000 (512
KiB)
Alloc by (vmpi          ) [0x0730e000-0x0730ffff] 0x00002000 (8 KiB)
Alloc by (vmpi          ) [0x0730c000-0x0730dfff] 0x00002000 (8 KiB)
Alloc by (vmpi          ) [0x0730a000-0x0730bfff] 0x00002000 (8 KiB)
Alloc by (vmpi          ) [0x07308000-0x07309fff] 0x00002000 (8 KiB)
Alloc by (vmpi          ) [0x07306000-0x07307fff] 0x00002000 (8 KiB)
Alloc by (vmpi          ) [0x07246000-0x07305fff] 0x000c0000 (768
KiB)
Alloc by (vmpi          ) [0x07186000-0x07245fff] 0x000c0000 (768
KiB)
Alloc by (vmpi          ) [0x070c6000-0x07185fff] 0x000c0000 (768
KiB)
Alloc by (vmpi          ) [0x06ald000-0x070c5fff] 0x006a9000 (6820
KiB)
Alloc by (vmpi          ) [0x0691d000-0x06a1cfff] 0x00100000 (1024
KiB)
Alloc by (vmpi          ) [0x068eb000-0x0691cfff] 0x00032000 (200
KiB)
Alloc by (vmpi          ) [0x068ea000-0x068eafff] 0x00001000 (4 KiB)
Alloc by (vmpi          ) [0x068e2000-0x068e9fff] 0x00008000 (32 KiB)
Alloc by (vmpi          ) [0x068e1000-0x068e1fff] 0x00001000 (4 KiB)
Alloc by (vmpi          ) [0x068d9000-0x068e0fff] 0x00008000 (32 KiB)
```

Alloc by (vmpi KiB)	) [0x0680f000-0x068d8fff]	0x000ca000	(808
Alloc by (vmpi KiB)	) [0x0677a000-0x0680efff]	0x00095000	(596
Alloc by (vmpi KiB)	) [0x066e5000-0x06779fff]	0x00095000	(596
Alloc by (vmpi	) [0x066d3000-0x066e4fff]	0x00012000	(72 KiB)
Alloc by (vmpi	) [0x066c1000-0x066d2fff]	0x00012000	(72 KiB)
Alloc by (vmpi KiB)	) [0x06488000-0x066c0fff]	0x00239000	(2276
Alloc by (vmpi	) [0x06486000-0x06487fff]	0x00002000	(8 KiB)
Alloc by (vmpi KiB)	) [0x0643c000-0x06485fff]	0x0004a000	(296
Alloc by (vmpi	) [0x0643a000-0x0643bfff]	0x00002000	(8 KiB)
Alloc by (vmpi KiB)	) [0x063f0000-0x06439fff]	0x0004a000	(296
Alloc by (vmpi KiB)	) [0x05c34000-0x063effff]	0x007bc000	(7920
Alloc by (vmpi KiB)	) [0x05587000-0x05c2ffff]	0x006a9000	(6820
Alloc by (vmpi KiB)	) [0x04ede000-0x05586fff]	0x006a9000	(6820
Alloc by ((null) KiB)	) [0x04633000-0x046f0fff]	0x000be000	(760
Alloc by (ff660000.npu KiB)	) [0x044e3000-0x04632fff]	0x00150000	(1344
Alloc by (ff660000.npu	) [0x044d5000-0x044e2fff]	0x0000e000	(56 KiB)
Alloc by (ff660000.npu	) [0x044c7000-0x044d4fff]	0x0000e000	(56 KiB)
Alloc by (ff660000.npu KiB)	) [0x044ab000-0x044c6fff]	0x0001c000	(112
Alloc by (ff660000.npu KiB)	) [0x04473000-0x044aafff]	0x00038000	(224
Alloc by (ff660000.npu KiB)	) [0x0443b000-0x04472fff]	0x00038000	(224
Alloc by (ff660000.npu KiB)	) [0x043cb000-0x0443afff]	0x00070000	(448
Alloc by (ff660000.npu KiB)	) [0x03e03000-0x043cafff]	0x005c8000	(5920
Alloc by (vmpi	) [0x03e00000-0x03e02fff]	0x00003000	(12 KiB)
Alloc by (ffa00000.rkisp	) [0x046f1000-0x04705fff]	0x00015000	(84 KiB)
Alloc by (ffa00000.rkisp	) [0x04706000-0x04706fff]	0x00001000	(4 KiB)
Alloc by (ffa00000.rkisp	) [0x04707000-0x04707fff]	0x00001000	(4 KiB)
Alloc by (ffa00000.rkisp KiB)	) [0x04708000-0x04e3efff]	0x00737000	(7388
Alloc by (ffa00000.rkisp KiB)	) [0x04e3f000-0x04ecdfff]	0x0008f000	(572
Alloc by (rkisp-vir0	) [0x04ece000-0x04ed1fff]	0x00004000	(16 KiB)
Alloc by (rkisp-vir0	) [0x04ed2000-0x04ed5fff]	0x00004000	(16 KiB)
Alloc by (rkisp-vir0	) [0x04ed6000-0x04ed9fff]	0x00004000	(16 KiB)
Alloc by (rkisp-vir0	) [0x04eda000-0x04eddfff]	0x00004000	(16 KiB)
Alloc by (ffa00000.rkisp	) [0x05c30000-0x05c33fff]	0x00004000	(16 KiB)

Notice: If security boot is enabled, sys_bootargs's parameter must be written into the bootargs parameter of kernel's dts file.

8.11 How to Use the coredump

- The defconfig corresponding to the kernel requires to enable the configuration that supports coredump.

```
CONFIG_ELF_CORE=y
CONFIG_CORE_DUMP_DEFAULT_ELF_HEADERS=y
```

- Set the size of coredump on board

```
ulimit -c unlimited
```

- Modify the coredump path on board

```
echo "/data/core-%p-%e" > /proc/sys/kernel/core_pattern
```

Notice: If write coredump file to the NFS or VFAT filesystem, we MUST run these codes on board. If not, the size of coredump file is 0.

```
# dump core file to /mnt/sdcard (which mount on vfat)
ulimit -c unlimited
echo "| /bin/coredump.sh %p %e" > /proc/sys/kernel/core_pattern
cat > /bin/coredump.sh << EOF
#!/bin/sh
exec cat - > "/mnt/sdcard/core-%$1-%$2"
EOF
chmod a+x /bin/coredump.sh
```

- List the stack of coredump

Copy the coredump to (e.g. /data/core-279-rkipc_get_nn_up) the directory of SDK and run these commands:

For example, the toolchain of RV1106B/RV1103B is **arm-rockchip831-linux-uclibcgnueabi-hf-gdb**

```
arm-rockchip831-linux-uclibcgnueabi-hf-gdb ./output/out/app_out/bin/rkipc
./core-279-rkipc_get_nn_up
# ...
(gdb) set solib-search-path output/out/media_out/lib/
Reading symbols from /home/rk/ipc-
sdk/output/out/media_out/lib/librkaiq.so...done.
Reading symbols from /home/rk/ipc-sdk/output/out/media_out/lib/librockiva.so...
(no debugging symbols found)...done.
Reading symbols from /home/rk/ipc-
sdk/output/out/media_out/lib/librockchip_mpp.so.0...(no debugging symbols
found)...done.
Reading symbols from /home/rk/ipc-
sdk/output/out/media_out/lib/libaec_bf_process.so...(no debugging symbols
found)...done.
Reading symbols from /home/rk/ipc-
sdk/output/out/media_out/lib/librkaudio_detect.so...(no debugging symbols
found)...done.
Reading symbols from /home/rk/ipc-sdk/output/out/media_out/lib/librga.so...(no
debugging symbols found)...done.
Reading symbols from /home/rk/ipc-sdk/output/out/media_out/lib/librknnmrt.so...
(no debugging symbols found)...done.
```

```
(gdb) bt
#0  0x00042080 in xx_list_pop ()
#1  0x0004229c in rkipc_xx_object_get ()
#2  0x0001a3c4 in rkipc_xx_update_osd ()
#3  0xa6c05390 in start_thread ()
    from /home/rk/ipc-sdk/tools/linux/toolchain/arm-rockchip831-linux-
uclibcgnueabi/hf/arm-rockchip831-linux-uclibcgnueabi/hf/sysroot/lib/libc.so.0
#4  0xa6bb8764 in clone ()
    from /home/rk/ipc-sdk/tools/linux/toolchain/arm-rockchip831-linux-
uclibcgnueabi/hf/arm-rockchip831-linux-uclibcgnueabi/hf/sysroot/lib/libc.so.0
Backtrace stopped: previous frame identical to this frame (corrupt stack?)
```

8.12 Kernel Driver Insmode Introduction

This chapter is used for RV1106B/RV1103B.

Please refer to `sysdrv/drv_ko/insmod_ko.sh`

```
#!/bin/sh
# if not install udevadm, ignore 'udevadm control'
udevadm control --stop-exec-queue

insmod rk_dvbm.ko

insmod video_rkcif.ko
insmod video_rkisp.ko
insmod phy-rockchip-csi2-dphy-hw.ko
insmod phy-rockchip-csi2-dphy.ko

insmod os04a10.ko
insmod sc4336.ko
insmod sc3336.ko
insmod sc530ai.ko

echo 1 > /sys/module/video_rkcif/parameters/clk_unready_dev
echo 1 > /sys/module/video_rkisp/parameters/clk_unready_dev

insmod rga3.ko

insmod mpp_vcodec.ko
insmod rockit.ko

insmod rknpu.ko
insmod snd-soc-rv1106.ko

# $sensor_height is the height of the camera sensor (e.g. os04a0/sc4336/sc3336
and so on)
insmod rockit.ko

udevadm control --start-exec-queue
```

8.13 Information About Libraries and Driver for RV1106B/RV1103B

- Libraries

Name	Usage	Is it necessary
ld-uClibc-1.0.31.so	Toolchain Library	YES
libatomic.so	Toolchain Library	YES
libgcc_s.so	Toolchain Library	YES
libgcc_s.so.1	Toolchain Library	YES
libstdc++.so	Toolchain Library	YES
libuClibc-1.0.31.so	Toolchain Library	YES
libitm.so	Toolchain Library	YES
librga.so	Raster Graphic Acceleration Unit	YES
librkaiq.so	Rockchip Auto Image Quality	YES
librockchip_mpp.so	Encode And Decode Library	YES
librockit.so	Interface of Multi-media Library	YES
librkaudio_common.so/librkaudio.so	Audio Basic Algorithm Library	NO
librkaudio_vqe.so	Audio 3A Algorithm Library	NO
librkaudio_detect.so	Audio Detect Library	NO
librockiva.so	NPU Algorithm Library	NO
librknnmrt.so	librockiva.so needed	NO
librkfsmk.so	Optimize Storage-related Library	NO
librkmuxer.so	Media File Stream Library	NO
libdrm_rockchip.so	Direct Rendering Manager For Rockchip	NO
libdrm.so	Direct Rendering Manager	NO

- Drivers

Some of Kernel Driver	Usage	Is it necessary
mpp_vcodec.ko	Video Encode Driver	YES
phy-rockchip-csi2-dphy-hw.ko	Mipi dphy rx Physics Driver	YES
phy-rockchip-csi2-dphy.ko	Mipi dphy rx Logic Driver	YES
video_rkcif.ko	Rockchip CIF Driver	YES
video_rkisp.ko	Rockchip ISP Driver	YES
rockit.ko	Interface of Multi-media	YES
rga3.ko	Raster Graphic Acceleration Unit	YES
os04a10.ko	os04a10 Sensor Driver	NO
sc3336.ko	sc3336 Sensor Driver	NO
sc4336.ko	sc4336 Sensor Driver	NO
sc530ai.ko	sc530ai Sensor Driver	NO
rknpu.ko	NPU Driver	NO

8.14 How to Use NFS

- Enable NFS for kernel defconfig

```

CONFIG_EXPORTFS_BLOCK_OPS=y
CONFIG_FILE_LOCKING=y
CONFIG_KEYS=y
CONFIG_NETWORK_FILESYSTEMS=y
CONFIG_ASSOCIATIVE_ARRAY=y
CONFIG_DNS_RESOLVER=y
# CONFIG_ECRYPT_FS is not set
# CONFIG_ENCRYPTED_KEYS is not set
CONFIG_FS_POSIX_ACL=y
CONFIG_GRACE_PERIOD=y
CONFIG_LOCKD=y
CONFIG_LOCKD_V4=y
CONFIG_MANDATORY_FILE_LOCKING=y
CONFIG_NFS_ACL_SUPPORT=y
CONFIG_NFS_COMMON=y
CONFIG_NFS_DISABLE_UDP_SUPPORT=y
CONFIG_NFS_FS=y
CONFIG_NFS_USE_KERNEL_DNS=y
# CONFIG_NFS_USE_LEGACY_DNS is not set
CONFIG_NFS_V2=y
CONFIG_NFS_V3=y
CONFIG_NFS_V3_ACL=y
CONFIG_NFS_V4=y
CONFIG_OID_REGISTRY=y
# CONFIG_PERSISTENT_KEYRINGS is not set
CONFIG_SUNRPC=y

```

```
CONFIG_SUNRPC_GSS=y
```

- Config NFS Server On PC

```
# Ubuntu 16.04 install NFS server
sudo apt-get install nfs-kernel-server
# Create /opt/rootfs
mkdir /opt/rootfs
# Enable nobody mount /opt/rootfs
chmod O+w -R /opt/rootfs
# Add the directory to exports
sudo echo "/opt/rootfs *(rw,sync,root_squash)" >> /etc/exports
# Update NFS configure
sudo exportfs -r
# Test NFS server
sudo mount -t nfs localhost:/opt/rootfs /mnt
# Ubuntu 16.04 disable firewall
sudo ufw disable
```

- Run NFS Mount Command On Board

```
# Get the IP address 192.168.1.123 of the PC
mount -t nfs -o nolock 192.168.1.123:/opt/rootfs /opt
```

8.15 How to Add a New User and Set Password for Login

For example, add the new user named testNewUser.

- Checking current users and get UID/GID

File Format:

Username:Password:User ID(UID):Group ID(GID):User ID Info (GECOS):Home directory:Login shell

Add /etc/passwd on board

```
testNewUser:x:1000:1000:testNewUser:/home:/bin/sh
```

- Use the command of mkpasswd to generate password on PC

```
sudo apt install whois
mkpasswd -m "md5" "test123" # --> $1$kprQOoLU$kOU2H.ecXkAw1ZJ0oplu/.
```

- Add /etc/shadow on board

```
testNewUser:$1$kprQOoLU$kOU2H.ecXkAw1ZJ0oplu/..:0:0:99999:7:::
```

- Add /etc/group on board

```
testNewUser:x:1000:
```

- Modify /etc/inittab on board

```
#::respawn:-/bin/sh
::sysinit:/etc/init.d/rcS

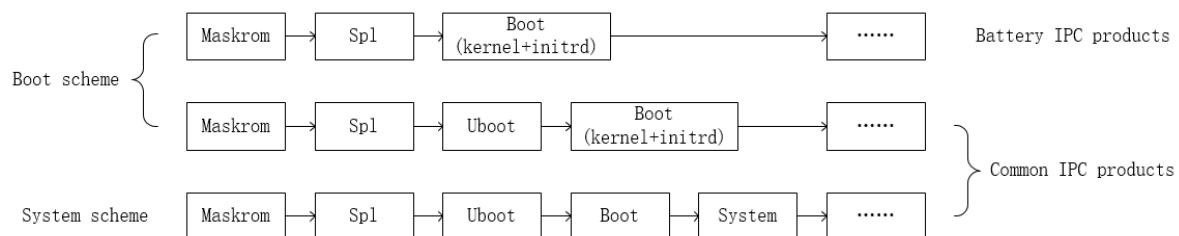
# Put a getty on the serial port
ttyFIQ0::respawn:/sbin/getty -L ttyFIQ0 0 vt100 # GENERIC_SERIAL
```

8.16 The Usage of A/B Systems

8.16.1 Introduction to Startup Schemes

The A/B systems support two startup schemes. The 'boot' scheme means that the boot partition contains the root file system (kernel+initrd). The 'system' scheme means that the root file system is separated (boot+system).

Currently, both startup schemes are supported in the common IPC products. Only the 'boot' scheme is supported in the battery IPC products.



8.16.2 Configuration

- Uboot

The Uboot configurations that need to be enabled for common IPC products and battery IPC products are different. The defconfig configurations are shown as follows respectively:

```
# Common IPC products
CONFIG_ANDROID_AB=y
CONFIG_SPL_MTD_WRITE=y
CONFIG_SPL_AB=y
CONFIG_EFI_PARTITION=y
CONFIG_SPL_EFI_PARTITION=y
CONFIG_AVB_LIBAVB=y
CONFIG_AVB_LIBAVB_AB=y
CONFIG_AVB_LIBAVB_USER=y
CONFIG_RK_AVB_LIBAVB_USER=y
```

```
# Battery IPC products
CONFIG_SPL_MTD_WRITE=y
CONFIG_SPL_AB=y
CONFIG_EFI_PARTITION=y
CONFIG_SPL_EFI_PARTITION=y
```

- Kernel

The following configuration should be enabled **only when the 'boot' scheme uses the initramfs file system**. In other cases, no additional configuration is required in kernel.

```
# The kernel configuration of initramfs file system
CONFIG_BLK_DEV_INITRD=y
```

- Root File System

When using SPI NAND as storage media, ensure that the following shell tools are in the file system:

Tool Name	Tool Functions
flash_erase	Erase blocks of the specified MTD device.
nandwrite	Writes to the specified MTD device.
nanddump	Dumps the contents of a nand mtd partition.
md5sum	Compute and check MD5 message digest.
dd	Convert and copy a file.

- BoardConfig

Enable OTA in BoardConfig to compile OTA upgrade tool and package images for A/B systems switching and upgrading.

```
# Enable OTA tool
export RK_ENABLE_OTA=y
# OTA package
export RK_OTA_RESOURCE="uboot.img boot.img system.img"
```

Modify the **partition Table** `RK_PARTITION_CMD_IN_ENV` and the **file system type** `RK_PARTITION_FS_TYPE_CFG`.

The followings provide examples of 'boot' and 'system' schemes for reference only, and parameters can be modified as required actually.

The 'boot' scheme:

```
# Partition table(Add the misc partition, remove the rootfs partition, and
change the boot partition to boot_a and boot_b partitions.)
export
RK_PARTITION_CMD_IN_ENV="64K(env),256K@64K(idblock),256K(uboot),64K(misc),384K(
meta),6M(boot_a),6M(boot_b),1M(userdata)"

# File system type(By default, erofs is used for battery IPC products and
initramfs is used for common IPC products.)
export RK_PARTITION_FS_TYPE_CFG=boot_a@IGNORE@erofs,userdata@/userdata@jffs2
```

The 'system' scheme:

```
# Partition table(Add the misc partition, remove the rootfs partition, change
the uboot partition to uboot_a and uboot_b partitions, change the boot
partition to boot_a and boot_b partitions, and add system_a and system_b
partitions.)
export
RK_PARTITION_CMD_IN_ENV="256K(env),256K@256K(idblock),256K(uboot_a),256K(uboot_
b),256K(misc),4M(boot_a),4M(boot_b),16M(system_a),16M(system_b),32M(oem),32M(us
erdata),-(media)"

# File system type
export
RK_PARTITION_FS_TYPE_CFG=system_a@IGNORE@ubifs,oem@oem@ubifs,userdata@/userdat
a@ubifs
```

8.16.3 OTA Upgrade Tool

AB system starts in RETRY_BOOT mode by default. If you need to start in SUCCESSFUL_BOOT mode, modify the configuration as follows:

Note: The difference between the two modes can be found in section 6.3.2 of the U-Boot documentation: [Rockchip_Developer_Guide_UBoot_Nextdev_CN.pdf](#)

```
diff --git a/tools/board/rk_ota/Makefile b/tools/board/rk_ota/Makefile
index 8bf67f5..21cf5e0 100644
--- a/tools/board/rk_ota/Makefile
+++ b/tools/board/rk_ota/Makefile
@@ -10,7 +10,7 @@ CURRENT_DIR := $(shell pwd)
PKG_BIN := out
TARGET_BIN := rk_ota
PKG_SRCS := src/mtutils.c src/md5sum.cpp src/bootloader.cpp src/main.cpp
-CONFIG_IS_SUCCESSFUL_BOOT := n
+CONFIG_IS_SUCCESSFUL_BOOT := y

PKG_DEP_OPENSSL := $(CURRENT_DIR)/../toolkits/openssl
PKG_DEP_ZLIB := $(CURRENT_DIR)/../toolkits/zlib
```

After adding the configuration `RK_ENABLE_OTA` to BoardConfig, the SDK will compile the OTA upgrade tool into the `/usr/bin` of the firmware file system. The command is called `rk_ota`. At the same time, the corresponding images are packaged to the `<SDK>/output/image/update_ota.tar`.

```

# rk_ota --help
[I/]RECOVERY *** rk_ota: Version V1.0.0 ***.
[I/]RECOVERY --misc=now                               Linux A/B mode: Setting the
current partition to bootable.
[I/]RECOVERY --misc=other                             Linux A/B mode: Setting
another partition to bootable.
[I/]RECOVERY --misc=update                             Linux A/B mode: Setting the
partition to be upgraded.
[I/]RECOVERY --misc=display                           Display misc info.
[I/]RECOVERY --tar_path=<path>                         Set upgrade firmware path.
[I/]RECOVERY --save_dir=<path>                         Set the path for saving the
image.
[I/]RECOVERY --partition=<uboot/boot/system/all>       Set the partition to be
upgraded. ('all' means 'uboot', 'boot' and 'system' are included.)
[I/]RECOVERY --extra_part=<name>                       Set the extra partition to be
upgraded.
[I/]RECOVERY --reboot                                 Restart the machine at the
end of the program.

```

8.16.4 A/B Systems Switching

The command for switching A/B systems is `rk_ota --misc=other`. The running result is as follows:

```

# rk_ota --misc=other --reboot
[I/]RECOVERY *** rk_ota: Version V1.0.0 ***.
[I/]RECOVERY Now is MTD.
A/B-slot: B, successful: 0, tries-remain: 6
[I/]RECOVERY Now is MTD.

```

`--reboot`: Restart the device.

Enter the other system. If the system boots successfully, you need to run the command `rk_ota --misc=now` to set the system as "the last boot system".

```

# rk_ota --misc=now
[I/]RECOVERY *** rk_ota: Version V1.0.0 ***.
[I/]RECOVERY Now is MTD.
A/B-slot: A, successful: 0, tries-remain: 6
info.mafic is 0
info.mafic is 41
info.mafic is 42
info.mafic is 30
[I/]RECOVERY Now is MTD.

```

8.16.5 A/B Systems Upgrading

Options for upgrading A/B systems are as follows:

`--misc=update`: Select 'Upgrade' mode;

`--tar_path=<path>`: Set the path of the OTA upgrading package (tar package that stores the images);

`--save_dir=<path>`: (Optional) Set the unpacking folder. If it is not set, the default folder is

`/mnt/sdcard/rk_update/`;

`--partition=<uboot/boot/system/all>`: (Optional) Set the partition to be upgraded. 'all' indicates that 'uboot', 'boot' and 'system' are upgraded. If it is not set, 'all' is the default value. (When there is no source file, the upgrade of the corresponding partition will be skipped.)

`--extra_part=<name>`: (Optional) Set a user-defined partition to be upgraded. If it is not set, this parameter is ignored by default.

Note : Currently, only the 'uboot', 'boot' and 'system' partitions and a user-defined partition can be upgraded.

For example:

```
# rk_ota --misc=update --tar_path=/mnt/sdcard/update_ota.tar --
save_dir=/mnt/sdcard/ --partition=all --reboot
# write 'boot'
[I/]RECOVERY *** rk_ota: Version V1.0.0 ***.
[I/]RECOVERY tar path = /mnt/sdcard/update_ota.tar
[I/]RECOVERY save path = /mnt/sdcard/
[I/]RECOVERY Now is MTD.
A/B-slot: A, successful: 0, tries-remain: 5
[I/]RECOVERY Now is MTD.
[I/]RECOVERY mtd_write src=/mnt/sdcard//boot.img dest=/dev/block/by-
name/boot_b.
Erasing 128 Kibyte @ 3e0000 -- 100 % complete
Writing data to block 0 at offset 0x0
Writing data to block 1 at offset 0x20000
...
2846720+0 records in
2846720+0 records out

[I/]RECOVERY Now is MTD.
[I/]RECOVERY [checkdata_mtd:30] offset [0] checksize [2846720]
ECC failed: 0
ECC corrected: 0
Number of bad blocks: 0
Number of bbt blocks: 0
Block size 131072, page size 2048, OOB size 128
Dumping data starting at 0x00000000 and ending at 0x002b7000...

read new md5: [6c60afda3ab31a49acd6d5d65e86a2e6]
new md5:6c60afda3ab31a49acd6d5d65e86a2e6
[I/]RECOVERY MD5Check is ok of /dev/block/by-name/boot_b
new md5:6c60afda3ab31a49acd6d5d65e86a2e6
[I/]RECOVERY MD5Check is ok for /mnt/sdcard//boot.img
[I/]RECOVERY check /dev/block/by-name/boot_b ok.
[I/]RECOVERY Write /mnt/sdcard//boot.img into /dev/block/by-name/boot_b
successfully.

# write 'system'
[I/]RECOVERY Now is MTD.
[I/]RECOVERY mtd_write src=/mnt/sdcard//system.img dest=/dev/block/by-
name/system_b.
Erasing 128 Kibyte @ fe0000 -- 100 % complete
Writing data to block 0 at offset 0x0
Writing data to block 1 at offset 0x20000
```

```

...
8126464+0 records in
8126464+0 records out

Writing data to block 62 at offset 0x7c0000
[I/]RECOVERY Now is MTD.
[I/]RECOVERY [checkdata_mtd:30] offset [0] checksize [8126464]
ECC failed: 0
ECC corrected: 0
Number of bad blocks: 0
Number of bbt blocks: 0
Block size 131072, page size 2048, OOB size 128
Dumping data starting at 0x00000000 and ending at 0x007c0000...
ECC: 1 corrected bitflip(s) at offset 0x004be000

read new md5: [9f9fad6f08cbdd210488ff544e14af25]
new md5:9f9fad6f08cbdd210488ff544e14af25
[I/]RECOVERY MD5Check is ok of /dev/block/by-name/system_b
new md5:9f9fad6f08cbdd210488ff544e14af25
[I/]RECOVERY MD5Check is ok for /mnt/sdcard//system.img
[I/]RECOVERY check /dev/block/by-name/system_b ok.
[I/]RECOVERY Write /mnt/sdcard//system.img into /dev/block/by-name/system_b
successfully.

[I/]RECOVERY Now is MTD.
[I/]RECOVERY Now is MTD.
mtd: successfully wrote block at 395a800000000
mtd: successfully wrote block at 395a800020000
reboot

```

8.17 Get the Camera Support Lists

The support list of camera sensor can be got from the Redmine <https://redmine.rock-chips.com/documents/53>

8.18 Get the Flash Support Lists

The support list of flash can be got from the Redmine <https://redmine.rock-chips.com/documents/46>

8.19 The Usage of the Stress Test

For the stress test function, the configuration needs to be enabled as follow:

```

# enable rockchip test
export RK_ENABLE_ROCKCHIP_TEST=y

```

The list of currently supported stress tests:

```

# ./rockchip_test/rockchip_test.sh
*****
***                                     ***

```

```

***          *****          ***
***      *ROCKCHIPS TEST TOOLS*      ***
***          *                  *          ***
***          *****          ***
***          ***          ***
***          *****          ***
*****
*****
ddr test :          1 (memtester & stressapptest)
cpufreq test:       2 (cpufreq stresstest)
flash stress test:  3
auto reboot test:   4
*****
please input your test moudle:

```

8.19.1 memtester Test

- Open the stress test list.

```
sh rockchip_test/rockchip_test.sh
```

- Start the test. (Select the test item No. 1 in the stress test list.)
- Select the `memtester test` number. (By default, the test uses half of the free memory.)

8.19.2 stressapptest

- Open the stress test list.

```
sh rockchip_test/rockchip_test.sh
```

- Start the test. (Select the test item No. 1 in the stress test list.)
- Select the `stressapptest` number. (By default, the test uses half of the free memory for 48 hours.)

8.19.3 cpufreq Test

- Open the stress test list.

```
sh rockchip_test/rockchip_test.sh
```

- Start the test. (Select the test item No. 2 in the stress test list.)
- Select the `cpu freq stress test` or `cpu freq test:(with out stress test)` number. (By default, the former uses half of the free memory for 24 hours. The latter default frequency conversion once a second.)

8.19.4 Flash Stress Test

- Open the stress test list.

```
sh rockchip_test/rockchip_test.sh
```

- Start the test. (Select the test item No. 3 in the stress test list.)

8.19.5 Reboot Test

- Open the stress test list.

```
sh rockchip_test/rockchip_test.sh
```

- Start the test. (Select the test item No. 4 in the stress test list.)
- Quit the test. (By default, the test is stopped after restarting for 10000 times. To exit early, enter the following command.)

```
echo off > /data/cfg/rockchip_test/reboot_cnt
```

8.20 How to Use rndis

Confirm linux kernel's dts configuration.

```
&u2phy {
    status = "okay";
};

&u2phy_otg {
    rockchip,vbus-always-on;
    status = "okay";
};

&usbdrd {
    status = "okay";
};

&usbdrd_dwc3 {
    extcon = <&u2phy>;
    status = "okay";
};
```

Confirm `rv1103b-rndis.config` is included in linux kernel's defconfig.

Confirm usb's vbus is supplied with 3.3V.

Modify BoardConfig.mk, add `export RK_ENABLE_RNDIS=y`, the command building as follow:

```
./build.sh sysdrv
./build.sh firmware
```

Flash firmware and run this command on board: `rndis.sh`

The details as follow:

```
# rndis.sh
serialnumber is f5bc7ed083b85dcf
config usb0 IP...
# ifconfig
usb0      Link encap:Ethernet  HWaddr C2:44:18:6D:9A:05
          inet addr:192.168.1.100  Bcast:192.168.1.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:133 errors:0 dropped:69 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:11360 (11.0 KiB)  TX bytes:0 (0.0 B)
```

Notice: If want to modify IP address default, we can modify the file of SDK named "sysdrv/tools/board/rndis/rndis.sh", and compiling and flash firmware again.

8.21 How to Add Non-root User Login

This section is going to introduce how to add non-root users to login in to a terminal when using busybox.

1. Modify /etc/passwd

```
test123:x:1010:1011:test123:/opt:/bin/sh
```

x represents user has set a password

1010 represents user's id

1011 represents user's group id

/opt represents the HOME directory of test123

Note: For detailed /etc/passwd instructions, please refer to <https://www.man7.org/linux/man-pages/man5/passwd.5.html>

2. Modify /etc/shadow

```
test123:$1$x.YlInZQ$N.kbNTIEOkBjmlfftSVFs0:10933:0:99999:7:::
```

`$1$x.YlInZQ$N.kbNTIEOkBjmlfftSVFs0:10933` is generated by the command of `mkpasswd -m "md5" "rockchip123"`.

Note: For detailed /etc/shadow instructions, please refer to <https://www.man7.org/linux/man-pages/man5/shadow.5.html>

3. Modify /etc/group

```
test123:x:1011:test123
```

Note: For detailed /etc/group instructions, please refer to <https://www.man7.org/linux/man-pages/man5/group.5.html>

4. Modify /etc/inittab

```
#::respawn:~/bin/sh
ttyFIQ0::respawn:/sbin/getty -L ttyFIQ0 0 vt100 # GENERIC_SERIAL
```

8.22 How to Add New Camera Sensor

Please refer to the documentation for detailed debugging:

<SDK>/docs/zh/media/Rockchip_Trouble_Shooting_Linux5.10_Camera_EN.pdf

8.23 How to Reboot into The Terminal of U-Boot

Take RV1103B fox example.

Directory: sysdrv/source/kernel

```
diff --git a/arch/arm/boot/dts/rv1103b.dtsi b/arch/arm/boot/dts/rv1103b.dtsi
index 4564909db8b2..99228b4b80cf 100644
--- a/arch/arm/boot/dts/rv1103b.dtsi
+++ b/arch/arm/boot/dts/rv1103b.dtsi
@@ -359,6 +359,7 @@ reboot_mode: reboot-mode {
                mode-ums = <BOOT_UMS>;
                mode-panic = <BOOT_PANIC>;
                mode-watchdog = <BOOT_WATCHDOG>;
+               mode-uboot = <BOOT_TO_UBOOT>;
+
        };

        rgb: rgb {
diff --git a/include/dt-bindings/soc/rockchip,boot-mode.h b/include/dt-
bindings/soc/rockchip,boot-mode.h
index 1436e1d32619..cc5a421aef26 100644
--- a/include/dt-bindings/soc/rockchip,boot-mode.h
+++ b/include/dt-bindings/soc/rockchip,boot-mode.h
@@ -21,4 +21,6 @@
/* enter usb mass storage mode */
#define BOOT_UMS                (REBOOT_FLAG + 12)

+#define BOOT_TO_UBOOT          (REBOOT_FLAG + 14)
+
#endif
```

Directory: sysdrv/source/uboot/u-boot

```
diff --git a/arch/arm/include/asm/arch-rockchip/boot_mode.h
b/arch/arm/include/asm/arch-rockchip/boot_mode.h
index bc1395ee2c..d68099a94e 100644
--- a/arch/arm/include/asm/arch-rockchip/boot_mode.h
+++ b/arch/arm/include/asm/arch-rockchip/boot_mode.h
@@ -26,6 +26,8 @@
/* enter bootrom download mode */
#define BOOT_BROM_DOWNLOAD      0xEF08A53C
```

```

#define BOOT_TO_UBOOT                (REBOOT_FLAG + 14)
+
#ifndef __ASSEMBLY__
int setup_boot_mode(void);
#endif
diff --git a/arch/arm/mach-rockchip/boot_mode.c b/arch/arm/mach-rockchip/boot_mode.c
index 61f0e85c1c..0d555314e2 100644
--- a/arch/arm/mach-rockchip/boot_mode.c
+++ b/arch/arm/mach-rockchip/boot_mode.c
@@ -189,6 +189,11 @@ int rockchip_get_boot_mode(void)
        boot_mode[PL] = BOOT_MODE_UMS;
        clear_boot_reg = 1;
        break;
+
+       case BOOT_TO_UBOOT:
+               printf("boot mode: uboot\n");
+               boot_mode[PL] = BOOT_MODE_UBOOT_TERMINAL;
+               clear_boot_reg = 1;
+               break;
+
+       case BOOT_CHARGING:
+               printf("boot mode: charging\n");
+               boot_mode[PL] = BOOT_MODE_CHARGING;
@@ -227,6 +232,8 @@ int setup_boot_mode(void)
{
        char env_preboot[256] = {0};

+       env_set("cli", NULL); /* removed by default */
+
        switch (rockchip_get_boot_mode()) {
        case BOOT_MODE_BOOTLOADER:
                printf("enter fastboot!\n");
@@ -259,6 +266,10 @@ int setup_boot_mode(void)
                printf("enter charging!\n");
                env_set("preboot", "setenv preboot; charge");
                break;
+
+       case BOOT_MODE_UBOOT_TERMINAL:
+               printf("enter uboot!\n");
+               env_set("cli", "yes");
+               break;
        }

        return 0;
diff --git a/common/autoboot.c b/common/autoboot.c
index c64d566d1c..9a6679aca9 100644
--- a/common/autoboot.c
+++ b/common/autoboot.c
@@ -220,7 +220,7 @@ static int __abortboot(int bootdelay)
#endif

#ifdef CONFIG_ARCH_ROCKCHIP
-       if (!IS_ENABLED(CONFIG_CONSOLE_DISABLE_CLI) && ctrlc()) { /* we
press ctrl+c ? */
+       if ((!IS_ENABLED(CONFIG_CONSOLE_DISABLE_CLI) && ctrlc()) ||
env_get("cli")) { /* we press ctrl+c ? */
        #else
        /*
        * Check if key already pressed
diff --git a/include/boot_rkimg.h b/include/boot_rkimg.h

```

```

index cb5781850e..d8ef3e6127 100644
--- a/include/boot_rkimg.h
+++ b/include/boot_rkimg.h
@@ -19,6 +19,7 @@ enum _boot_mode {
    BOOT_MODE_PANIC,
    BOOT_MODE_WATCHDOG,
    BOOT_MODE_DFU,
+   BOOT_MODE_UBOOT_TERMINAL,
    BOOT_MODE_UNDEFINE,
};

```

Notice: The value of **BOOT_TO_UBOOT** in the kernel MUST be matched with it in the U-Boot.

Rebuild kernel and U-Boot with command `./build.sh kernel; ./build.sh uboot`, then update boot.img and uboot.img.

After boot-up, run the command `reboot uboot`, the board will reboot and enter U-Boot Terminal.

8.24 How to Modify GPIO Register Configuration by Command

All the GPIO registers and configuration commands can be found in the document

`<SDK>/docs/zh/bsp/Rockchip_XXXX_User_Manual_GPIO.pdf`. The commands vary in the U-Boot terminal and the root file system, which will be described below.

Note: When using the pinmux function, confirm whether the pin has been occupied by a function in advance.

RV1103B

Name	Config	Description	Register	Position	Value	IO Command
GPIOA0	IOMUX	RTC_32K_OUT	0x201B0000	[0,3]	1	io -4 0x201B0000 0x000F0001
		CLK_32K	0x201B0000	[0,3]	2	io -4 0x201B0000 0x000F0002
		CLK_24M_OUT	0x201B0000	[0,3]	3	io -4 0x201B0000 0x000F0003
		FSPI1_CLK	0x201B0000	[0,3]	4	io -4 0x201B0000 0x000F0004
		GPIO0_A0	0x201B0000	[0,3]	0	io -4 0x201B0000 0x000F0000
	Pull	z	0x201B0200	[0,1]	0	io -4 0x201B0200 0x00030000
		up	0x201B0200	[0,1]	1	io -4 0x201B0200 0x00030001
		down	0x201B0200	[0,1]	2	io -4 0x201B0200 0x00030002
	I/O Mode	input	0x20520008	[0,0]	0	io -4 0x20520008 0x00010000
		output	0x20520008	[0,0]	1	io -4 0x20520008 0x00010001
	Output	low	0x20520000	[0,0]	0	io -4 0x20520000 0x00010000
		high	0x20520000	[0,0]	1	io -4 0x20520000 0x00010001
	Input Ctrl	disable	0x201B0300	[0,0]	0	io -4 0x201B0300 0x00010000
		enable	0x201B0300	[0,0]	1	io -4 0x201B0300 0x00010001
	Schmitt	disable	0x201B0400	[0,0]	0	io -4 0x201B0400 0x00010000
		enable	0x201B0400	[0,0]	1	io -4 0x201B0400 0x00010001
	Drive Strength	level0	0x201B0100	[0,5]	1	io -4 0x201B0100 0x003F0001
		level1	0x201B0100	[0,5]	3	io -4 0x201B0100 0x003F0003
		level2	0x201B0100	[0,5]	7	io -4 0x201B0100 0x003F0007
		level3	0x201B0100	[0,5]	15	io -4 0x201B0100 0x003F000F
		level4	0x201B0100	[0,5]	31	io -4 0x201B0100 0x003F001F
		level5	0x201B0100	[0,5]	63	io -4 0x201B0100 0x003F003F
	ExtPort	read gpio data in input mode	0x20520070	[0,0]		io -4 0x20520070

8.24.1 U-Boot Terminal

The U-Boot terminal uses the `md` command to display register values and the `mw` command to write register values.

- The following uses setting the output high level of GPIO0A0 as an example:

```
# IOMUX: GPIO (GPIO0A0->IOMUX->GPIO0_A0)
=> mw.l 0x201B0000 0x000F0000
# I/O mode: output (GPIO0A0->I/O Mode->output)
=> mw.l 0x20520008 0x00010001
# Output level: high level (GPIO0A0->Output->high)
=> mw.l 0x20520000 0x00010001
```

- The following uses reading the input data of GPIO0A0 as an example:

```
# IOMUX: GPIO (GPIO0A0->IOMUX->GPIO0_A0)
=> mw.l 0x201B0000 0x000F0000
# I/O mode: input (GPIO0A0->I/O Mode->input)
=> mw.l 0x20520008 0x00010000
# read data: 16 bytes (GPIO0A0->ExtPort) (4 long, which is 4*4=16 bytes)
=> md.l 0x20520070 0x4
```

8.24.2 Root File System

The root file system reads and writes register values using the `io` command.

- The following uses setting the output high level of GPIO0A0 as an example:

```
# IOMUX: GPIO (GPIO0A0->IOMUX->GPIO0_A0)
io -4 0x201B0000 0x000F0000
# I/O mode: output (GPIO0A0->I/O Mode->output)
io -4 0x20520008 0x00010001
# Output level: high level (GPIO0A0->Output->high)
io -4 0x20520000 0x00010001
```

- The following uses reading the input data of GPIO0A0 as an example:

```
# IOMUX: GPIO (GPIO0A0->IOMUX->GPIO0_A0)
io -4 0x201B0000 0x000F0000
# I/O mode: input (GPIO0A0->I/O Mode->input)
io -4 0x20520008 0x00010000
# read data: 16 bytes (GPIO0A0->ExtPort) (The unit is 4 bytes)
io -4 0x20520070
```

8.25 How to Modify the Uart Baudrate

The uart baudrate needs to be modified in both the DDR and kernel Device Tree Source (DTS) files.

8.25.1 DDR Bin File

1. Obtain the loader INI file

If there is a `RK_UBOOT_RKBIN_INI_OVERLAY` configuration in the board configuration, use the loader INI file specified by this configuration.

If unsure, compile U-Boot in advance and confirm the current loader INI file by examining the generated `.config` file. The result is as follows:

```
./build.sh uboot 2>&1 |grep "pack loader.*Input"
pack loader(SPL) okay! Input:
/opt/rv1103b/sysdrv/source/uboot/rkbin/rkboot/RV1103BMINIALL.ini
```

2. Obtain the path of the DDR bin file

From the loader INI file obtained above, we can get the path of the DDR bin file.

Take `RV1103BMINIALL.ini` as an example:

```
# cat /opt/rv1103b/sysdrv/source/uboot/rkbin/rkboot/RV1103BMINIALL.ini
[CHIP_NAME]
NAME=RV110E
[VERSION]
MAJOR=1
MINOR=1
[CODE471_OPTION]
NUM=1
Path1=bin/rv11/rv1103b_ddr_792MHz_v1.01.bin
Sleep=1
[CODE472_OPTION]
NUM=1
Path1=bin/rv11/rv1103b_usbplug_v1.00.bin
[LOADER_OPTION]
NUM=2
LOADER1=FlashData
LOADER2=FlashBoot
FlashData=bin/rv11/rv1103b_ddr_792MHz_v1.01.bin
FlashBoot=bin/rv11/rv1103b_spl_v1.00.bin
[OUTPUT]
PATH=rv1103b_download_v1.01.100.bin
IDB_PATH=rv1103b_idblock_v1.01.100.img
[SYSTEM]
NEWIDB=true
[FLAG]
471_RC4_OFF=true
RC4_OFF=true
CREATE_IDB=true
```

"FlashData" represents the configuration of the DDR bin file. Here, we can get the DDR bin file `bin/rv11/rv1103b_ddr_792MHz_v1.01.bin`, with the specific path being

`<SDK>/sysdrv/source/uboot/rkbin/bin/rv11/rv1103b_ddr_792MHz_v1.01.bin`.

3. Modify the uart baudrate in the DDR bin file

Use the DDR bin tool to modify the uart baudrate. Here's how to get the tool:

For Windows: Download from Redmine, link: <https://redmine.rock-chips.com/documents/49>

For Linux: `<SDK>/sysdrv/source/uboot/rkbin/tools/ddrbin_tool`

Below are the operation steps for the Windows tool:



After modification, recompile U-Boot.

8.25.2 Kernel DTS

The example board configuration uses the DTS rv1103b-evb1-v10.dts. Take this DTS as an example:

```
diff --git a/arch/arm/boot/dts/rv1103b-evb1-v10.dts
b/arch/arm/boot/dts/rv1103b-evb1-v10.dts
index dc6152543b20..bc8f062edc99 100644
--- a/arch/arm/boot/dts/rv1103b-evb1-v10.dts
+++ b/arch/arm/boot/dts/rv1103b-evb1-v10.dts
@@ -14,6 +14,10 @@ / {

+&fiq_debugger {
+    rockchip,baudrate = <1500000>;
+};
+
```

After modification, recompile the kernel.

8.26 Enabling the sysstat Test Suite

sysstat is an open-source Linux toolset that includes sar, pidstat, iostat, mpstat, sadf, and sadc. The sar is one of the most comprehensive system performance analysis tools on Linux. The sar tool samples the current state of the system and expresses the current operating state of the system by calculating data and ratios. Its feature is that it can continuously sample the system to obtain a large amount of sample data; sample data and analysis results can be stored in files, and the required load is small. It can report on the activity of the system in 14 aspects, including file read and write status, system call usage, serial ports, CPU efficiency, memory usage, process activity, and IPC-related activity.

It is highly recommended to enable this suite during debugging. The sar tool can be used to check the device data:

- Check CPU usage
- Check average load
- Check memory usage
- Check I/O device status
- Check network status
- Check page swap status
- ...

To enable the sysstat tool, the following two modifications are needed:

- Open the relevant configuration in sysdrv/source/kernel/arch/arm/configs/rv1103b-xxx.config:

```
CONFIG_TASKSTATS=y
CONFIG_SCHED_INFO=y
CONFIG_TASK_DELAY_ACCT=y
CONFIG_TASK_IO_ACCOUNTING=y
CONFIG_TASK_XACCT=y
CONFIG_DEBUG_USER=y
```

- Add the following in the BoardConfig:

```
export RK_ENABLE_SYSSTAT=y
```

The above can enable the sysstat tool, and the sar tool is used to count the system performance.

Commonly used commands:

```
CPU usage: Use sar -u 1 to check
Memory usage: Use sar -r 1 to check
I/O usage: Use sar -d 1 to check
```

Please search for other detailed commands on your own via a browser.

8.27 How to use DM-Verity and DM-Crypt

The SDK has only been verified on the eMMC of the constantly power IPC.

Configure as follows in BoardConfig.mk:

The defconfig of kernel add `rv1103b-device-mapper.config`

Enable DM-Verity feature separately: `RK_ENABLE_DM_V=y`

Enable DM-Crypt feature separately: `RK_ENABLE_DM_CRYPT=y`, Set `RK_MISC` and add `misc` partition

Enable both DM-Verity and DM-Crypt features: `RK_ENABLE_DM_V_DM_CRYPT=y`

Here is an example of enabling both DM-Verity and DM-Crypt features simultaneously:

The SDK defaults to using the rootfs as both the DM-Crypt partition and the DM-Verity partition.

```
diff --git a/BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-IPC.mk
b/BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-IPC.mk
index ac51b19..4c622a1 100644
--- a/BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-IPC.mk
```

```

+++ b/BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-IPC.mk
@@ -48,7 +48,11 @@ export RK_BOOTARGS_CMA_SIZE="48M"
#       <partdef> := <size>[@<offset>] (part-name)
# Note:
#   If the first partition offset is not 0x0, it must be added. Otherwise, it
needn't adding.
-export
RK_PARTITION_CMD_IN_ENV="256K(env),1M@256K(idblock),1M(uboot),8M(boot),24M(root
fs),48M(oem),40M(userdata)"
+export
RK_PARTITION_CMD_IN_ENV="256K(env),1M@256K(idblock),256K(misc),1M(uboot),8M(boo
t),24M(rootfs),48M(oem),40M(userdata)"
+
+export RK_MISC=blank-misc.img
+
+export RK_ENABLE_DM_V_DM_CRYPT=y

# Kernel defconfig fragment
-export RK_KERNEL_DEFCONFIG_FRAGMENT=rv1103b-evb.config
+export RK_KERNEL_DEFCONFIG_FRAGMENT="rv1103b-evb.config rv1103b-device-
mapper.config"

# config partition's filesystem type (squashfs is readonly)
# emmc:      squashfs/ext4
# nand:      squashfs/ubifs
# spi nor:   squashfs/jffs2
# RK_PARTITION_FS_TYPE_CFG format:
#   AAAA:/BBBB/CCCC@ext4
#       AAAA -----> partition name
#       /BBBB/CCCC ----> partition mount point
#       ext4 -----> partition filesystem type
-export
RK_PARTITION_FS_TYPE_CFG=rootfs@IGNORE@ext4,userdata@/userdata@ext4,oem@/oem@ex
t4
+export
RK_PARTITION_FS_TYPE_CFG=rootfs@IGNORE@squashfs,userdata@/userdata@ext4,oem@/oe
m@ext4

```

Note: The image for DM-Verity only supports read-only file system formats (e.g., squashfs).

```

./build.sh clean      # Clean sdk

./build.sh all        # Compile

./build.sh dm-key-gen # If you enable DM-Crypt, you will need to generate a dm-
crypt key, and running it once is sufficient.
                        # The directory where the key is saved is in the SDK root
                        # directory under dm-key.
                        # Compiling DM-Crypt requires sudo privileges. Use the
                        # command `echo "XXX" > dm-key/root_passwd`, where XXX is the sudo password.

./build.sh firmware   # Generate original firmware

./build.sh dm          # Build rootfs with DM-Verity or DM-Crypt

```

The image directory for DM-Verity and DM-Crypt is located at: /output/image-dm.

The DM-Crypt key is stored in the misc partition. In actual applications, you need to write the key to the eMMC's RPMB partition (refer to /project/scripts/init.in for writing RPMB).

8.28 How to Use UVC to Preview

The SDK provides UVC function, which can be used to preview and focus the camera.

- Open UVC's Compiling

Take project/cfg/BoardConfig_IPC/BoardConfig-SPI_NAND-NONE-RV1103B_EVB1_V10_V11-IPC.mk for example:

1. Add UVC_TINY for RK_APP_TYPE
2. Enable `RK_ENABLE_RNDIS` and build with kernel's drivers
3. Install `rk_mpi_uvc` and `usb_config.sh`

```
# app config
+export RK_APP_TYPE="UVC_TINY"

+ # enable rndis
+ export RK_ENABLE_RNDIS=y
```

Compiling Command

```
./build.sh all
```

- Preview on PC

Connect PC to the EVB board through an OTG cable, use the UVC preview tool on the PC to open UVC device.

9. Notices

When copying the source code package under Windows, the executable file under Linux may become a non-executable file, or the soft link fails and cannot be compiled and used.

Therefore, please be careful not to copy the source code package under Windows.